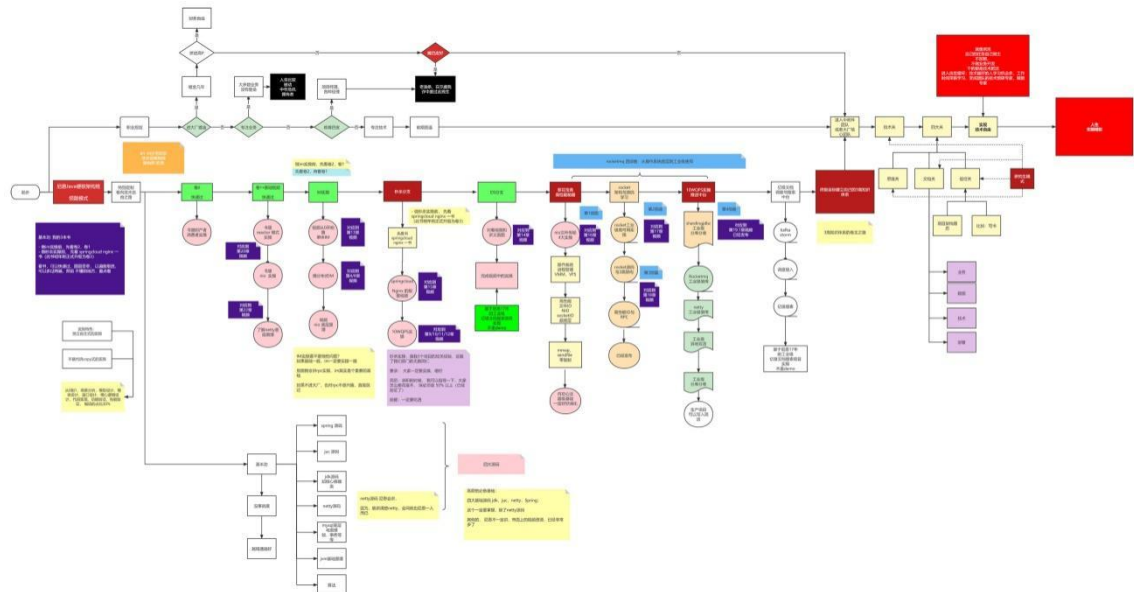


牛逼的职业发展之路

40 岁老架构尼恩用一张图揭秘：Java 工程师的高端职业发展路径，走向食物链顶端的之路

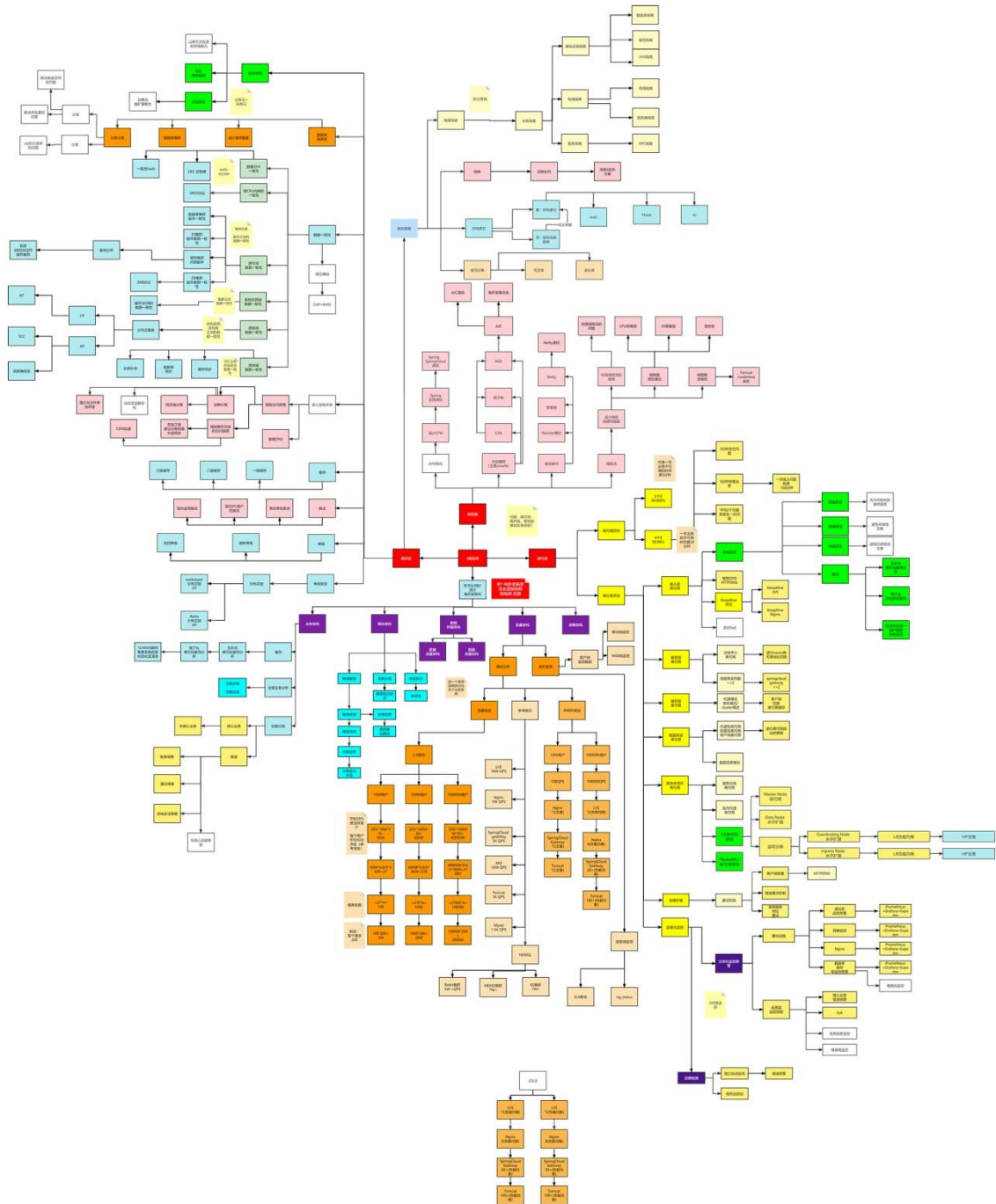
链接：<https://www.processon.com/view/link/618a2b62e0b34d73f7eb3cd7>



史上最全：价值10W的架构师知识图谱

此图梳理于尼恩的多个 3 高生产项目：多个亿级人民币的大型 SAAS 平台和智慧城市项目

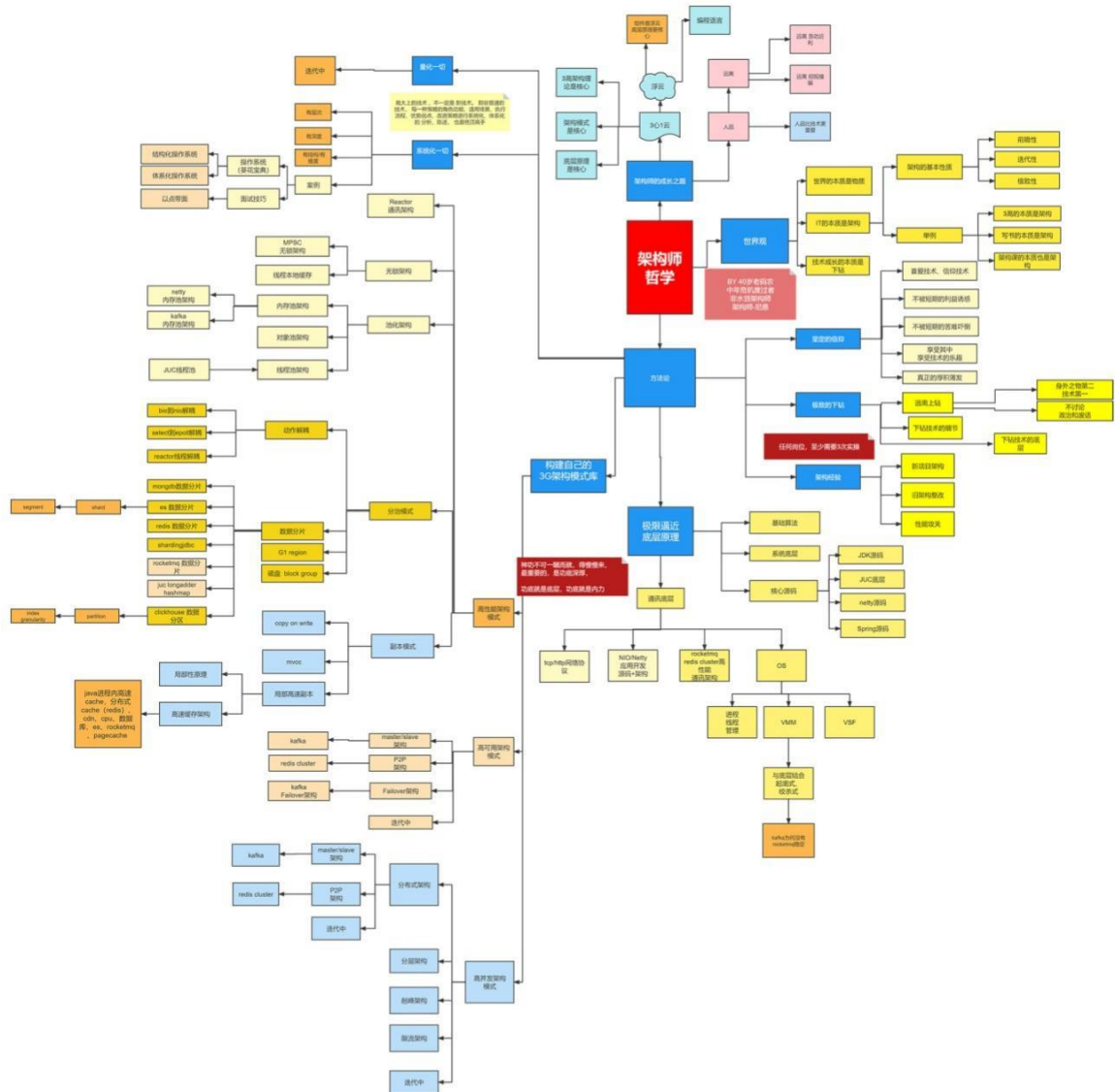
链接：<https://www.processon.com/view/link/60fb9421637689719d246739>



牛逼的架构师哲学

40 岁老架构师尼恩对自己的 20 年的开发、架构经验总结

链接: <https://www.processon.com/view/link/616f801963768961e9d9aec8>



牛逼的3高架构知识宇宙

尼恩 3 高架构知识宇宙，帮助大家穿透 3 高架构，走向技术自由，远离中年危机

链接: <https://www.processon.com/view/link/635097d2e0b34d40be778ab4>



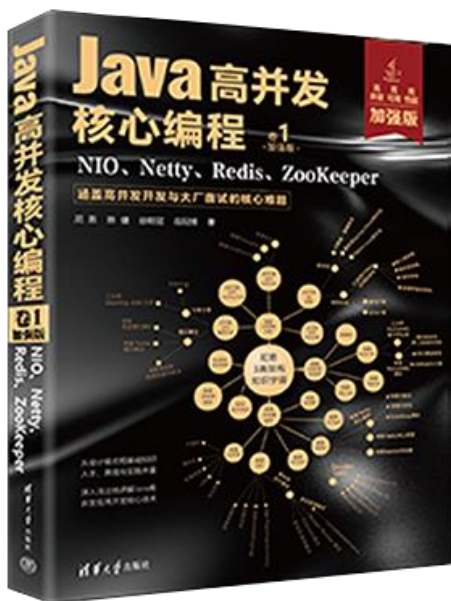
尼恩Java高并发三部曲（卷1加强版）

老版本：《Java 高并发核心编程 卷1：NIO、Netty、Redis、ZooKeeper》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷1 **加强版**：NIO、Netty、Redis、ZooKeeper》

- 由浅入深地剖析了高并发 IO 的底层原理。
- 图文并茂地介绍了 TCP、HTTP、WebSocket 协议的核心原理。
- 细致深入地揭秘了 Reactor 高性能模式。
- 全面介绍了 Netty 框架，并完成单体 IM、分布式 IM 的实战设计。
- 详尽地介绍了 ZooKeeper、Redis 的使用，以帮助提升高并发、可扩展能力

详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



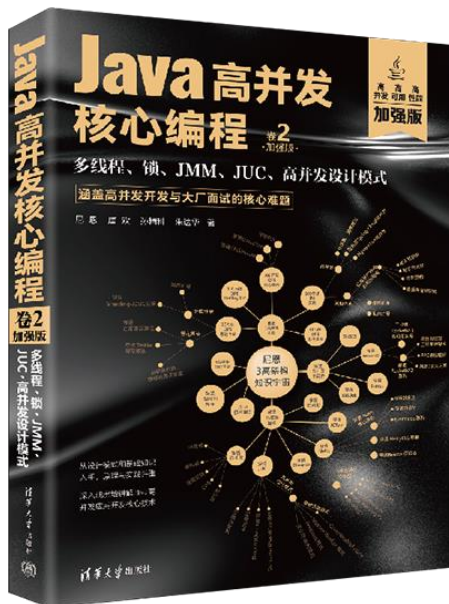
尼恩Java高并发三部曲（卷2加强版）

老版本：《Java 高并发核心编程 卷2：多线程、锁、JMM、JUC、高并发设计模式》
（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷2 **加强版**：多线程、锁、JMM、JUC、高并发设计模式》

- 由浅入深地剖析了 Java 多线程、线程池的底层原理。
- 总结了 IO 密集型、CPU 密集型线程池的线程数预估算法。
- 图文并茂地介绍了 Java 内置锁、JUC 显式锁的核心原理。
- 细致深入地揭秘了 JMM 内存模型。
- 全面介绍了 JUC 框架的设计模式与核心原理，并完成其高核心组件的实战介绍。
- 详尽地介绍了高并发设计模式的使用，以帮助提升高并发、可扩展能力

详情参阅：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



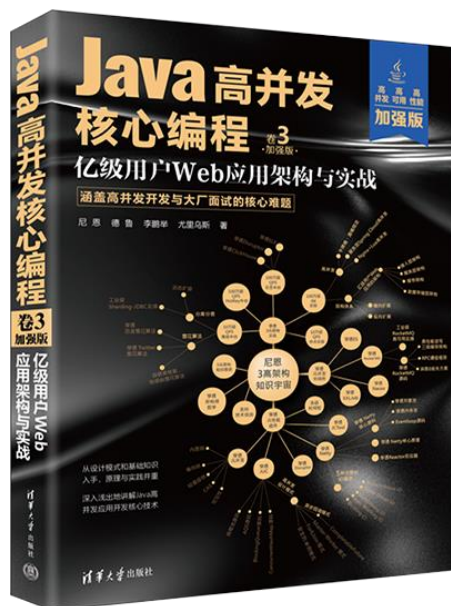
尼恩Java高并发三部曲（卷3加强版）

老版本：《SpringCloud Nginx 高并发核心编程》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷3 **加强版**：亿级用户 Web 应用架构与实战》

- 在当今的面试场景中，3 高知识是大家面试必备的核心知识，本书基于亿级用户 3 高 Web 应用的架构分析理论，为大家对 3 高架构系统做一个系统化和清晰化的介绍。
- 从 Java 静态代理、动态代理模式入手，抽丝剥茧地解读了 Spring Cloud 全家桶中 RPC 核心原理和执行过程，这是高级 Java 工程师面试必备的基础知识。
- 从Reactor 反应器模式入手，抽丝剥茧地解读了Nginx核心思想和各配置项的底层知识和原理，这是高级 Java 工程师、架构师面试必备的基础知识。
- 从观察者模式入手，抽丝剥茧地解读了 RxJava、Hystrix 的核心思想和使用方法，这也是高级 Java 工程师、架构师面试必备的基础知识。

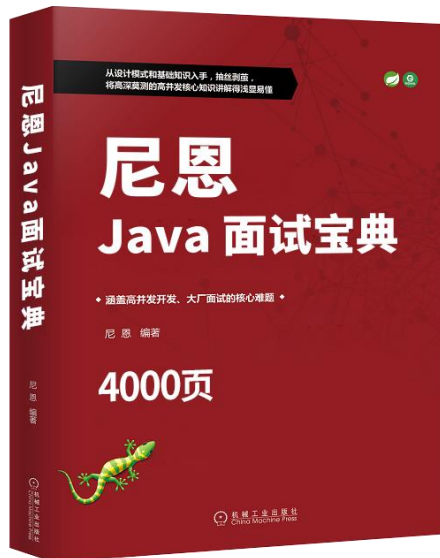
详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



尼恩Java面试宝典

40 个专题（卷王专供+ 史上最全 + 2023 面试必备）

详情：<https://www.cnblogs.com/crazymakercircle/p/13917138.html>



- 📖 专题01: JVM面试题 (卷王专供 + 史上最全 + 2023面试必备) -V20-from-尼恩Java面试宝典.pdf
- 📖 专题02: Java算法面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2 -from-Java面试红宝书-release.pdf
- 📖 专题03: Java基础面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题04: 架构设计面试题 (卷王专供 + 史上最全 + 2023面试必备) -V24-from-Java面试红宝书-release.pdf
- 📖 专题05: Spring面试题_专题06: SpringMVC_专题07: Tomcat面试题 (卷王专供 + 史上最全 + 2023面试必备) -V3-from-尼恩面试宝典-release.pdf
- 📖 专题08: SpringBoot面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题09: 网络协议面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题10: TCP/IP协议 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题11: JUC并发包与容器类 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题12: 设计模式面试题 (卷王专供 + 史上最全 + 2023面试必备) -V14-from-Java面试红宝书.pdf
- 📖 专题13: 死锁面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题14: Redis 面试题 (卷王专供 + 史上最全 + 2023面试必备) -V22-from-Java面试红宝书-release.pdf
- 📖 专题15: 分布式数据库面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题16: Zookeeper 面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题17: 分布式事务面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题18: 一致性协议 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题19: Zab协议 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题20: Paxos 协议 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题21: raft 协议 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题22: Linux面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题23: Mysql 面试题 (卷王专供 + 史上最全 + 2023面试必备) -V29-from-尼恩Java面试宝典-release.pdf
- 📖 专题24: SpringCloud 面试题 (卷王专供 + 史上最全 + 2023面试必备) -V12-from-Java面试红宝书-release.pdf
- 📖 专题25: Netty 面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题26: 消息队列面试题: RabbitMQ、Kafka、RocketMQ (卷王专供 + 史上最全 + 2023面试必备) -V10-from-Java面试红宝书-release.pdf
- 📖 专题27: 内存泄漏 内存溢出 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题28: JVM 内存溢出 实战 (卷王专供 + 史上最全 + 2023面试必备) -V17-from-Java面试红宝书-release.pdf
- 📖 专题29: 多线程面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题30: HR面试题: 过五关斩六将后, 小心阴沟翻船! (史上最全、避坑宝典) -V2-from-Java面试红宝书-release.pdf
- 📖 专题31: Hash链表面试题 (卷王专供 + 史上最全 + 2023面试必备) -V16-from-Java面试红宝书-release.pdf
- 📖 专题32: 大厂面试的基本流程和面试准备 (阿里、腾讯、网易、京东、头条.....) -V2-from-Java面试红宝书-release.pdf
- 📖 专题33: BST、AVL、RBT红黑树、三大核心数据结构 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题34: Elasticsearch面试题 (卷王专供 + 史上最全 + 2023面试必备) -V3-from-Java面试红宝书-release.pdf
- 📖 专题35: Mybatis面试题 (卷王专供 + 史上最全 + 2023面试必备) -V3-from-尼恩Java面试宝典-release.pdf
- 📖 专题36: Dubbo面试题 (卷王专供 + 史上最全 + 2023面试必备) -V21-from-尼恩Java面试宝典-release.pdf
- 📖 专题37: Docker面试题 (卷王专供 + 史上最全 + 2023面试必备) -V26-from-尼恩Java面试宝典-release.pdf
- 📖 专题38: K8S面试题 (卷王专供 + 史上最全 + 2023面试必备) -V26-from-尼恩Java面试宝典-release.pdf
- 📖 专题39: Nginx面试题 (卷王专供 + 史上最全 + 2023面试必备) -V27-from-尼恩Java面试宝典-release.pdf
- 📖 专题40: 操作系统面试题 (卷王专供 + 史上最全 + 2023面试必备) -V28-from-尼恩Java面试宝典-release.pdf

专题38：K8S面试题（史上最全、定期更新）

本文版本说明：V59

此文的格式，由markdown 通过程序转成而来，由于很多表格，没有来的及调整，出现一个格式问题，尼恩在此给大家道歉啦。

由于社群很多小伙伴，在面试，不断的交流最新的面试难题，所以，《尼恩Java面试宝典》，后面会不断升级，迭代。

本专题，作为 《尼恩Java面试宝典》专题之一，《尼恩Java面试宝典》一共**40个面试专题**，后续还会增加

V59 升级说明

聊聊：（支付宝真题）什么是pod，说说pod的创建流程？（重点）

聊聊：（支付宝真题）deployment的创建流程？（重点）

聊聊：（天翼云真题）Etcd的写数据流程是什么？

聊聊：简述Service的四种类型？ 四者之间有何不同？

《尼恩面试宝典》升级的规划为：

后续基本上，**每一个月，都会发布一次**，最新版本，可以扫描扫架构师尼恩微信，发送“领取电子书”获取。

尼恩的微信二维码在哪里呢？ 请参见文末

面试问题交流说明：

如果遇到面试难题，或者职业发展问题，或者中年危机问题，都可以来 疯狂创客圈社群交流，

入交流群，加尼恩微信即可，发送“**入群**”，

尼恩微信请打开语雀扫码 <https://www.yuque.com/crazymakercircle/gkkw8s/khigna>



什么是k8s？说出你的理解

K8s是kubernetes的简称，其本质是一个开源的容器编排系统，主要用于管理容器化的应用，其目标是让部署容器化的应用简单并且高效（powerful），Kubernetes提供了应用部署，规划，更新，维护的一种机制。

说简单点：k8s就是一个编排容器的系统，一个可以管理容器应用全生命周期的工具，从创建应用，应用的部署，应用提供服务，扩容缩容应用，应用更新，都非常的方便，而且还可以做到故障自愈，

所以，k8s是一个非常强大的容器编排系统。

k8s的组件有哪些，作用分别是什么？

k8s主要由master节点和node节点构成。

master节点负责管理集群，node节点是容器应用真正运行的地方。

master节点包含的组件有：kube-api-server、kube-controller-manager、kube-scheduler、etcd。

node节点包含的组件有：kubelet、kube-proxy、container-runtime。

kube-api-server:

以下简称api-server，api-server是k8s最重要的核心组件之一，它是k8s集群管理的统一访问入口，提供了RESTful API接口，实现了认证、授权和准入控制等安全功能；api-server还是其他组件之间的数据交互和通信的枢纽，其他组件彼此之间并不会直接通信，其他组件对资源对象的增、删、改、查和监听操作都是交由api-server处理后，api-server再提交给etcd数据库做持久化存储，只有api-server才能直接操作etcd数据库，其他组件都不能直接操作etcd数据库，其他组件都是通过api-server间接的读取，写入数据到etcd。

kube-controller-manager:

以下简称controller-manager，controller-manager是k8s中各种控制器的管理者，是k8s集群内部的管理控制中心，也是k8s自动化功能的核心；controller-manager内部包含replication controller、node controller、deployment controller、endpoint controller等各种资源对象的控制器，每种控制器都负责一种特定资源的控制流程，而controller-manager正是这些controller的核心管理者。

kube-scheduler:

以下简称scheduler，scheduler负责集群资源调度，其作用是将待调度的pod通过一系列复杂的调度算法计算出最合适的node节点，然后将pod绑定到目标节点上。scheduler会根据pod的信息，全部节点信息列表，过滤掉不符合要求的节点，过滤出一批候选节点，然后给候选节点打分，选分最高的就是最佳节点，scheduler就会把目标pod安置到该节点。

Etcd:

etcd是一个分布式的键值对存储数据库，主要是用于保存k8s集群状态数据，比如，pod，service等资源对象的信息；etcd可以是单个也可以有多个，多个就是etcd数据库集群，etcd通常部署奇数个实例，在大规模集群中，etcd有5个或7个节点就足够了；另外说明一点，etcd本质上可以不与master节点部署在一起，只要master节点能通过网络连接etcd数据库即可。

kubelet:

每个node节点上都有一个kubelet服务进程，kubelet作为连接master和各node之间的桥梁，负责维护pod和容器的生命周期，当监听到master下发到本节点的任务时，比如创建、更新、终止pod等任务，kubelet即通过控制docker来创建、更新、销毁容器；

每个kubelet进程都会在api-server上注册本节点自身的信息，用于定期向master汇报本节点资源的使用情况。

kube-proxy:

kube-proxy运行在node节点上，在Node节点上实现Pod网络代理，维护网络规则和四层负载均衡工作，kube-proxy会监听api-server中从而获取service和endpoint的变化情况，创建并维护路由规则以提供服务IP和负载均衡功能。简单理解此进程是Service的透明代理兼负载均衡器，其核心功能是将到某个Service的访问请求转发到后端的多个Pod实例上。

container-runtime：容器运行时环境，即运行容器所需要的一系列程序，目前k8s支持的容器运行时有很多，如docker、rkt或其他，比较受欢迎的是docker，但是新版的k8s已经宣布弃用docker。

简述Kubernetes相关基础概念？

答：

master:

k8s集群的管理节点，负责管理集群，提供集群的资源数据访问入口。拥有Etcd存储服务（可选），运行Api Server进程，Controller Manager服务进程及Scheduler服务进程；

node (worker) :

Node (worker) 是Kubernetes集群架构中运行Pod的服务节点，是Kubernetes集群操作的单元，用来承载被分配Pod的运行，是Pod运行的宿主机。运行docker engine服务，守护进程kuberneteslet及负载均衡器kube-proxy；

pod:

运行于Node节点上，若干相关容器的组合。Pod内包含的容器运行在同一宿主机上，使用相同的网络命名空间、IP地址和端口，能够通过localhost进行通信。Pod是Kubernetes进行创建、调度和管理的最小单位，它提供了比容器更高层次的抽象，使得部署和管理更加灵活。一个Pod可以包含一个容器或者多个相关容器；

label:

Kubernetes中的Label实质是一系列的Key/Value键值对，其中key与value可自定义。Label可以附加到各种资源对象上，如Node、Pod、Service、RC等。一个资源对象可以定义任意数量的Label，同一个Label也可以被添加到任意数量的资源对象上去。Kubernetes通过Label Selector（标签选择器）查询和筛选资源对象；

Replication Controller:

Replication Controller用来管理Pod的副本，保证集群中存在指定数量的Pod副本。

集群中副本的数量大于指定数量，则会停止指定数量之外的多余容器数量。反之，则会启动少于指定数量个数的容器，保证数量不变。

Replication Controller是实现弹性伸缩、动态扩容和滚动升级的核心；

Deployment:

Deployment在内部使用了RS来实现目的，Deployment相当于RC的一次升级，其最大的特色为可以随时获知当前Pod的部署进度；

HPA (Horizontal Pod Autoscaler) :

Pod的横向自动扩容，也是Kubernetes的一种资源，通过追踪分析RC控制的所有Pod目标的负载变化情况，来确定是否需要针对性的调整Pod副本数量；

Service:

Service定义了Pod的逻辑集合和访问该集合的策略，是真实服务的抽象。

Service提供了一个统一的服务访问入口以及服务代理和发现机制，关联多个相同Label的Pod，用户不需要了解后台Pod是如何运行；

Volume:

Volume是Pod中能够被多个容器访问的共享目录，Kubernetes中的Volume是定义在Pod上，可以被一个或多个Pod中的容器挂载到某个目录下；

Namespace:

Namespace用于实现多租户的资源隔离，可将集群内部的资源对象分配到不同的Namespace中，形成逻辑上的不同项目、小组或用户组，便于不同的Namespace在共享使用整个集群的资源的同时还能被分别管理；

Kubernetes 面试必问的核心知识点

Kubernetes 中的网络是一个需要理解的重要主题，因为它使运行在 Kubernetes 集群上的应用程序能够相互通信，以及与外部服务通信。

在 Kubernetes 集群中，每个 pod 都有自己唯一的 IP 地址，可在集群内路由。

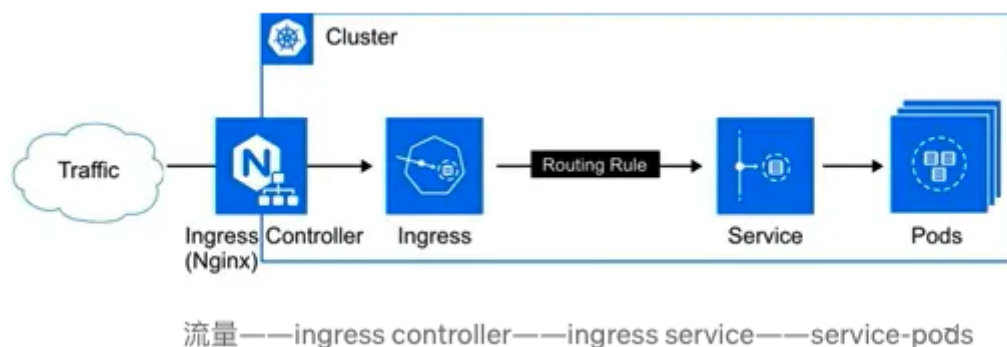
该 IP 地址通常由 Kubernetes 网络插件分配，该插件负责为集群配置网络。

网络插件是 Kubernetes 网络架构的关键组件，有多种选择，例如 Calico、Flannel 和 Weave Net。

除了网络插件之外，Kubernetes 还包括许多其他涉及网络的组件，例如负责跨多个 Pod 负载均衡网络流量的 kube-proxy 和提供稳定 IP 的 Service 对象—一组 pod 的地址和 DNS 名称。

了解网络在 Kubernetes 中的工作原理对于任何使用 Kubernetes 的人来说都很重要，因为它会对集群上运行的应用程序的性能和可靠性产生重大影响。

1 Ingress



在 Kubernetes 中，Ingress 是一个 API 对象，它提供了一种方法来配置对集群中服务的外部访问。

Ingress 资源通常定义一组规则，这些规则根据传入请求的主机和路径指定传入流量应如何路由到适当的服务。它将集群外部的 HTTP 和 HTTPS 路由暴露给集群内的服务。

当请求进入 Kubernetes 集群时，它首先会到达 Ingress 控制器，该控制器负责根据 Ingress 资源中定义的规则管理和路由流量。

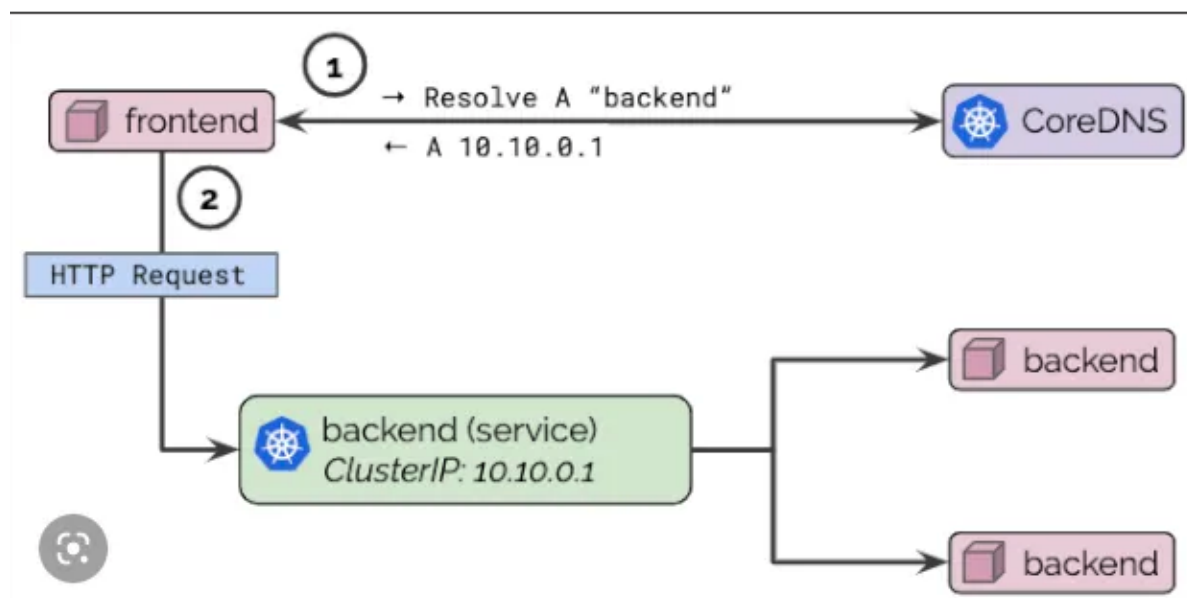
Ingress 控制器使用 Ingress 资源中定义的一组规则来确定应将流量路由到何处。为了让入口工作，您需要使用入口控制器，即 Nginx、AWS。

一旦 Ingress 控制器确定了将流量路由到的适当服务，它就会将请求转发到服务的 ClusterIP，这是由 Kubernetes 网络插件分配给服务的内部 IP 地址。然后该服务使用自己的规则来确定将流量转发到哪个 Pod。

然后流量到达目标 pod，由在 pod 内运行的应用程序处理。然后应用程序的响应通过服务和 Ingress 控制器通过相同的路径发送回原始请求发送者。

请务必注意，Ingress 资源仅负责将流量路由到集群内的服务，不处理任何身份验证或安全性。因此，它经常与 TLS 终止或 Web 应用程序防火墙 (WAF) 等其他安全措施结合使用，以提供安全可靠的服务。

2 DNS——域名系统



域名系统 (DNS) 用于为服务和 Pod 提供一种相互发现和通信的方式。它是负责将服务名称转换或解析为其 IP 地址的服务。

Kubernetes 集群中的每个 pod 都被分配了一个唯一的主机名，该主机名源自 pod 的名称及其运行所在的命名空间。默认情况下，每个主机名都可以在集群的 DNS 命名空间中解析。

在 Kubernetes 中创建服务时，会为其分配一个稳定的 DNS 名称，用于从集群中的其他 pod 和服务访问该服务。

DNS 名称的格式通常为 `servicename.namespace.svc.cluster.local`，其中 `servicename` 是服务名称，

`namespace` 是运行服务的 Kubernetes 命名空间，`cluster.local` 是集群的默认 DNS 域。

当 pod 想要与服务通信时，它可以简单地使用服务的 DNS 名称来连接它。

Kubernetes DNS 服务会将 DNS 名称解析为分配给该服务的相应 ClusterIP，并将流量路由到适当的 pod。

除了为服务解析 DNS 名称外，Kubernetes 还支持自定义 DNS 配置，允许您指定在解析 DNS 查询时应使用的其他 DNS 服务器或搜索域。如果您需要在 Kubernetes 集群之外解析 DNS 名称，例如访问外部服务或 API，这会很有用。

总的来说，DNS 是 Kubernetes 中网络的重要组成部分，因为它允许服务和 pod 相互发现和通信，并且是在 Kubernetes 集群上部署可靠和可扩展的应用程序的基础设施的重要组成部分。

3 Core DNS

CoreDNS 是 Kubernetes 中用于服务发现和 DNS 解析的流行 DNS 服务器实现。它是 Kubernetes 的默认 DNS 服务器，可确保 pod 和服务具有完全限定的域名 (FQDN)。CoreDNS 是一种灵活且可扩展的 DNS 服务器，旨在轻松集成到 Kubernetes 集群中，并且可以进行定制以支持广泛的用例。如果没有 CoreDNS，集群的通信将停止工作。

在 Kubernetes 中，CoreDNS 通常作为一个 pod 部署在集群中，负责解析服务和 pod 的 DNS 查询。CoreDNS 使用 Kubernetes API 检索有关服务和 pod 的信息，并自动为它们中的每一个生成 DNS 记录。

在 Kubernetes 中使用 CoreDNS 的好处之一是它具有高度可配置性，并且可以扩展以支持自定义插件和 DNS 提供程序。例如，您可以使用 CoreDNS 插件来添加对自定义 DNS 区域的支持或与外部 DNS 提供商集成。

CoreDNS 的另一个好处是它提供了比 Kubernetes 中以前的默认 DNS 服务器 kube-dns 更好的性能和可扩展性。CoreDNS 使用 Go 编写，旨在实现轻量级和高效，这使其非常适合在高流量 Kubernetes 环境中处理大量 DNS 查询。

要在 Kubernetes 集群中使用 CoreDNS，您可以使用 Kubernetes 清单文件或 Helm 图表将其部署为 pod。部署后，您可以配置 CoreDNS 服务器以满足您的特定需求，例如通过添加自定义 DNS 提供程序、定义自定义 DNS 区域或其他 Kubernetes 组件（如 Ingress 或 ExternalDNS）集成。

总的来说，CoreDNS 是一个强大而灵活的 DNS 服务器实现，非常适合在 Kubernetes 集群中使用，并为现代云原生应用程序中的服务发现和 DNS 解析提供了坚实的基础。

FQDN — 完全限定域名是指定其在树层次结构中的确切位置的域名，也称为核心 DNS 的绝对域功能。

4 Probes-探针

在 Kubernetes 中，探测器用于确定在 pod 中运行的容器的健康状况。探测器是 Kubernetes 自我修复和自动扩展功能的关键部分，因为它们为集群提供了一种自动检测不健康容器并从中恢复的方法。探针用于检测容器的状态。

Kubernetes 中有三种类型的探针：

Liveness Probe-活性探针

这种类型的探测用于确定容器是否仍在运行并且是否健康。liveness probe 会周期性的向容器发送请求，如果容器没有响应或者响应错误，liveness probe 会将容器标记为不健康并触发重启。

Liveness 探测可以捕获应用程序正在运行但无法取得任何进展的死锁，而是重新启动容器，这可以使应用程序更可用，尽管存在错误。

Readiness Probe-就绪探测

这种类型的探测用于确定容器是否准备好开始接收流量。就绪探测器向容器发送定期请求，如果容器以成功代码响应，它将被标记为准备好接收流量。如果容器未能响应或响应错误，它将被标记为未就绪，并且在再次准备就绪之前不会接收任何流量。

Startup Probe-启动探针

这种类型的探测用于确定容器是否处于启动过程中。启动探测器向容器发送定期请求，如果容器以成功代码响应，它将被标记为准备好接收流量。如果容器无法响应或响应错误，启动探测器将继续发送请求，直到容器准备就绪，或者直到达到可配置的超时时间。

Kubernetes 中的探测器使用 YAML 配置在 pod 的规范部分中定义。每个探测都定义了一组参数，例如探测类型、要探测的端点、探测超时、探测周期以及成功和失败阈值。

总的来说，探针是 Kubernetes 中一个强大的特性，它可以在发生故障或无响应时自动监控并重启容器，这有助于提高运行在 Kubernetes 集群上的应用程序的可靠性和可用性。

5 Netfilter-网络过滤器

在 Kubernetes 中，Netfilter 用于实现网络策略，用于控制集群中 pod 之间网络流量的流动。网络策略是 Kubernetes 对象，它们定义了如何允许流量在 pod 之间流动的规则，并且它们使用 Netfilter 规则来执行这些策略。

当网络策略应用于 Kubernetes 集群时，Kubernetes API 服务器与网络插件通信，网络插件负责在底层网络基础设施中配置网络规则。反过来，网络插件生成 Netfilter 规则来执行网络策略。

网络插件生成的 Netfilter 规则基于网络策略中指定的选择器。选择器用于识别哪些 pod 应该受到策略的影响，它们可以基于范围广泛的标准，例如 pod 的标签、名称空间或 IP 地址。网络插件生成匹配指定选择器的 Netfilter 规则，然后将策略中指定的操作应用于匹配的数据包。

例如，可以定义网络策略以允许流量仅在具有特定标签的 pod 之间流动。网络插件将生成 Netfilter 规则以匹配具有该标签的 pod 之间的数据包，然后允许这些数据包流经网络。同样，网络策略可能被定义为拒绝两个特定 pod 之间的流量，在这种情况下，网络插件将生成 Netfilter 规则以丢弃这些 pod 之间的数据包。

总的来说，Netfilter 是 Kubernetes 中网络策略实施的关键组件，因为它允许对集群中 pod 之间的网络流量进行精细控制，并提供强大的机制来执行安全和访问控制策略。

6 IPTables

IPTables 是一个基于 Linux 的防火墙工具，允许用户配置和管理网络过滤规则。在 Kubernetes 中，IPTables 用于实现网络策略，控制 pod 和服务之间的流量。

在 Kubernetes 中创建网络策略时，kube-proxy 组件会创建 IPTables 规则来执行该策略。当网络流量通过 pod 或服务所在的节点时，这些规则将应用于网络流量。

Kubernetes 生成的 IPTables 规则基于网络策略的选择器和规则。选择器确定策略适用于哪些 pod，而规则定义应允许或拒绝哪些流量。例如，可以创建一个网络策略，只允许流量流向具有特定标签的 pod 上的特定端口。

Kubernetes 生成的 IPTables 规则被插入到内核的 IPTables 链中，它决定了网络流量的处理方式。这些链按特定顺序进行评估，第一个匹配规则确定对数据包采取的操作。

Kubernetes 还使用 IPTables 来实现 Kubernetes Services，它为访问一组 pod 提供稳定的 IP 地址和 DNS 名称。在 Kubernetes 中创建服务时，kube-proxy 会创建一个 IPTables 规则，以根据服务的选择器将流量转发到适当的 pod。

总体而言，IPTables 是在 Kubernetes 中实施网络策略和服务的重要工具，因为它允许对网络流量进行细粒度控制，并为负载平衡和服务发现提供可靠且可扩展的机制。

7 IPVS

IPVS（IP 虚拟服务器）是一个提供网络负载平衡功能的 Linux 内核模块。在 Kubernetes 中，IPVS 被用来替代 kube-proxy 和 IPTables 来实现服务。

当在 Kubernetes 中创建服务并将服务类型设置为“LoadBalancer”时，IPVS 用于为服务创建虚拟 IP 地址（VIP）。VIP 用作客户端流量的目标地址，并与一组提供实际服务的 pod 相关联。

IPVS 的工作原理是拦截传入 VIP 的流量，并使用负载平衡算法将其分配到可用的 Pod 中。IPVS 中有几种可用的负载均衡算法，包括循环法、最少连接数和加权最少连接数。

IPVS 还提供健康检查以确保流量只发送到健康的 Pod。当 Pod 未通过健康检查时，IPVS 将其从可用 Pod 列表中删除，并在其余健康 Pod 之间重新分配流量。

IPVS 与 kube-proxy 和 IPTables 相比有几个优势，包括更好的可扩展性和性能，以及更灵活的负载平衡算法。IPVS 可以处理大量连接，并针对高吞吐量和低延迟进行了优化。它还支持更高级的负载平衡功能，例如会话持久性和连接耗尽。

但是，与 kube-proxy 和 IPTables 相比，IPVS 需要额外的配置和设置，并且可能无法与所有网络环境兼容。IPVS 还需要内核支持，可能并非在所有 Linux 发行版上都可用。

8 Proxy-代理

代理是一种服务器应用程序，充当请求资源的客户端和提供该资源的服务器之间的中介

Kubectrl proxy

Kubectrl Proxy 是一个命令行工具，允许用户在他们的本地机器和 Kubernetes API 服务器之间创建一个安全隧道。这允许用户访问 Kubernetes API 服务器，而无需直接网络访问或复杂的身份验证配置。Kubectrl Proxy 用于各种目的，例如访问 Kubernetes 仪表板或对远程集群使用 kubectl 命令。

例如，假设用户想要访问在远程集群上运行的 Kubernetes Dashboard。他们可以使用 Kubectrl Proxy 创建安全隧道，然后通过本地 Web 浏览器访问 Dashboard。

Kube-proxy

另一方面，Kube-Proxy 是运行在 Kubernetes 集群中各个节点上的组件，负责实现 Kubernetes Services。Kube-Proxy 监听服务的变化，然后相应地更新本地 IPTables 或 IPVS 规则。这确保流量被正确路由到集群中适当的 pod。

例如，假设在 Kubernetes 中创建了一个服务，它映射到一组标签为“app=myapp”的 pod。Kube-Proxy 将创建 IPTables 或 IPVS 规则，根据服务的选择器将流量定向到适当的 pod。

Kubectrl Proxy 和 Kube-Proxy 都有优点和局限性。Kubectrl Proxy 设置简单并提供对 Kubernetes API 服务器的安全访问，但它可能很慢并且可能不适合生产环境。

Kube-Proxy 可靠且可扩展，但配置起来可能很复杂，可能并不适合所有网络环境。

Envoy

除了 Kube-Proxy 之外，Kubernetes 中使用的另一个流行的代理是 Envoy。Envoy 是一种高性能代理，可提供高级流量管理和负载均衡功能。Envoy 可以作为 Kube-Proxy 的替代品来实现 Kubernetes 服务，也可以作为一个独立的组件来提供高级的流量管理功能。

Envoy 用于许多生产环境，可以提供高级负载均衡算法、熔断和分布式跟踪等优势。

但是，与 Kube-Proxy 相比，Envoy 需要额外的设置和配置，并且可能无法与所有网络环境兼容。此外，Envoy 通常用于更复杂的场景，例如多集群或多云环境，对于更简单的用例来说可能有点过头了。

9 CNI: Container Networking Interface

容器网络接口 (CNI) 是用于在容器化环境中配置网络的规范和工具集，例如 Kubernetes 提供的那些。

CNI 的目标是为网络插件提供一个通用标准，以便容器运行时和编排系统可以与任何支持 CNI API 的网络解决方案一起工作。

CNI 为容器运行时（如 Docker 或 CRI-O）定义了一种标准方式来调用网络插件来配置容器的网络接口。这些插件负责为容器创建和配置网络接口，以及配置网络命名空间和路由表。

在 Kubernetes 中，kubelet 使用 CNI 来配置 pod 的网络接口。

创建 pod 时，kubelet 调用 CNI 插件来配置 pod 的网络。然后 CNI 插件为 pod 创建和配置网络接口，设置任何必要的路由规则，并将 pod 的 IP 地址添加到适当的网络命名空间。

CNI 插件可以内置到容器运行时中，也可以作为独立的二进制文件提供。有许多可用的 CNI 插件，每个都有自己的优点和缺点。一些流行的 CNI 插件包括 Calico、Flannel 和 Weave Net。

CNI 的使用在容器化环境中提供了几个好处。

- 首先，它允许一个可以被多个容器运行时和编排系统使用的通用标准。这意味着网络插件可以独立于容器运行时或编排系统开发，从而提高了灵活性和兼容性。
- 其次，CNI 提供了一个模块化和可扩展的架构，可以轻松与其他网络解决方案集成。这使用户能够为其特定用例选择最佳网络解决方案，并避免供应商锁定。
- 最后，CNI 提供了一个简单灵活的 API 来配置容器网络，这使得开发人员可以轻松地创建和部署适合他们需求的自定义网络解决方案。

简述Kubernetes和Docker的关系？

答：

Docker开源的容器引擎，一种更加轻量级的虚拟化技术；

K8s，容器管理工具，用来管理容器pod的集合，它可以实现容器集群的自动化部署、自动扩缩容、维护等功能；

简述Kubernetes如何实现集群管理？

答：在集群管理方面，Kubernetes将集群中的机器划分为一个Master节点和一群工作节点Node。

其中，在Master节点运行着集群管理相关的一组进程kube-apiserver、kube-controller-manager和kube-scheduler，这些进程实现了整个集群的资源管理、Pod调度、弹性伸缩、安全控制、系统监控和纠错等管理能力，并且都是全自动完成的；

简述Kubernetes的优势、适应场景及其特点？

答：

优势：容器编排、轻量级、开源、弹性伸缩、负载均衡；

场景：快速部署应用、快速扩展应用、无缝对接新的应用功能、节省资源，优化硬件资源的使用；

特点：

可移植：支持公有云、私有云、混合云、多重云（multi-cloud）、

可扩展：模块化、插件化、可挂载、可组合、

自动化：自动部署、自动重启、自动复制、自动伸缩/扩展；

简述Kubernetes的缺点或当前的不足之处？

答：

安装过程和配置相对困难复杂、管理服务相对繁琐、运行和编译需要很多时间、它比其他替代品更昂贵、对于简单的应用程序来说，可能不需要涉及Kubernetes即可满足；

简述Kubernetes中什么是Minikube、Kubectl、Kubelet？

答：

Minikube 是一种可以在本地轻松运行一个单节点 Kubernetes 群集的工具；

Kubectl 是一个命令行工具，可以使用该工具控制Kubernetes集群管理器，如检查群集资源，创建、删除和更新组件，查看应用程序；

Kubelet 是一个代理服务，它在每个节点上运行，并使从服务器与主服务器通信；

kubelet的功能、作用是什么？（重点，经常会问）

答：kubelet部署在每个node节点上的，它主要有4个功能：

1、节点管理。

kubelet启动时会向api-server进行注册，然后会定时的向api-server汇报本节点信息状态，资源使用状态等，这样master就能够知道node节点的资源剩余，节点是否失联等等相关的信息了。master知道了整个集群所有节点的资源情况，这对于 pod 的调度和正常运行至关重要。

2、pod管理。

kubelet负责维护node节点上pod的生命周期，当kubelet监听到master的下发到自己节点的任务时，比如要创建、更新、删除一个pod，kubelet就会通过CRI（容器运行时接口）插件来调用不同的容器运行时来创建、更新、删除容器；常见的容器运行时有docker、containerd、rkt等等这些容器运行时，我们最熟悉的就docker了，但在新版本的k8s已经弃用docker了，k8s1.24版本中已经使用containerd作为容器运行时了。

3、容器健康检查。

pod中可以定义启动探针、存活探针、就绪探针等3种，我们最常用的就是存活探针、就绪探针，kubelet会定期调用容器中的探针来检测容器是否存活，是否就绪，如果是存活探针，则会根据探测结果对检查失败的容器进行相应的重启策略；

4、Metrics Server资源监控。

在node节点上部署Metrics Server用于监控node节点、pod的CPU、内存、文件系统、网络使用等资源使用情况，而kubelet则通过Metrics Server获取所在节点及容器的上的数据。

kube-api-server的端口是多少？各个pod是如何访问kube-api-server的？

kube-api-server的端口是8080和6443，前者是http的端口，后者是https的端口，以我本机使用kubeadm安装的k8s为例：

在命名空间的kube-system命名空间里，有一个名称为kube-api-master的pod，

这个pod就是运行着kube-api-server进程，它绑定了master主机的ip地址和6443端口，但是在default命名空间下，存在一个叫kubernetes的服务，该服务对外暴露端口为443，目标端口6443，

这个服务的ip地址是clusterip地址池里面的第一个地址，同时这个服务的yaml定义里面并没有指定标签选择器，

也就是说这个kubernetes服务所对应的endpoint是手动创建的，该endpoint也是名称叫做kubernetes，该endpoint的yaml定义里面代理到master节点的6443端口，也就是kube-api-server的IP和端口。

这样一来，其他pod访问kube-api-server的整个流程就是：pod创建后嵌入了环境变量，pod获取到了kubernetes这个服务的ip和443端口，请求到kubernetes这个服务其实就是转发到了master节点上的6443端口的kube-api-server这个pod里面。

k8s中命名空间的作用是什么？

namespace是kubernetes系统中的一种非常重要的资源，namespace的主要作用是用来实现多套环境的资源隔离，或者说是多租户的资源隔离。

k8s通过将集群内部的资源分配到不同的namespace中，可以形成逻辑上的隔离，以方便不同的资源进行隔离使用和管理。

不同的命名空间可以存在同名的资源，命名空间为资源提供了一个作用域。

可以通过k8s的授权机制，将不同的namespace交给不同的租户进行管理，这样就实现了多租户的资源隔离，还可以结合k8s的资源配额机制，限定不同的租户能占用的资源，例如CPU使用量、内存使用量等等来实现租户可用资源的管理。

k8s提供了大量的REST接口，其中有一个是Kubernetes Proxy API接口，简述一下这个Proxy接口的作用，已经怎么使用。

kubernetes proxy api接口，从名称中可以得知，proxy是代理的意思，其作用就是代理rest请求；

Kubernets API server 将接收到的rest请求转发到某个node上的kubelet守护进程的rest接口，由该kubelet进程负责响应。

我们可以使用这种Proxy接口来直接访问某个pod，这对于逐一排查pod异常问题很有帮助。

下面是一些简单的例子：

```
1 http://<kube-api-server>:<api-sever-port>/api/v1/nodes/node名称/proxy/pods  
  #查看指定node的所有pod信息  
2 http://<kube-api-server>:<api-sever-port>/api/v1/nodes/node名称/proxy/stats  
  #查看指定node的物理资源统计信息  
3 http://<kube-api-server>:<api-sever-port>/api/v1/nodes/node名称/proxy/spec  
  #查看指定node的概要信息  
4  
5 http://<kube-api-server>:<api-sever-port>/api/v1/namespace/命名名称/pods/pod名称/pod服务的url/ #访问指定pod的程序页面  
6 http://<kube-api-server>:<api-sever-port>/api/v1/namespace/命名名称/servers/svc名称/url/ #访问指定server的url程序页面  
7
```

pod是什么？

在kubernetes的世界中，k8s并不直接处理容器，而是使用多个容器共存的理念，这组容器就叫做pod。

pod是k8s中可以创建和管理的最小单元，是资源对象模型中由用户创建或部署的最小资源对象模型，其他的资源对象都是用来支撑pod对象功能的，

比如，pod控制器就是用来管理pod对象的，service或者ingress资源对象是用来暴露pod引用对象的，persistentvolume资源是用来为pod提供存储等等，

简而言之，k8s不会直接处理容器，而是pod，pod才是k8s中可以创建和管理的最小单元，也是基本单元。

pod的原理是什么？

在微服务的概念里，一般的，一个容器会被设计为运行一个进程，除非进程本身产生子进程，

这样，由于不能将多个进程聚集在同一个单独的容器中，所以需要一种更高级的结构将容器绑定在一起，并将它们作为一个单元进行管理，这就是k8s中pod的背后原理。

聊聊：（支付宝真题）什么是pod，说说pod的创建流程？

Tips:

这个问题几乎是每一家公司必须要问的问题，作为k8s的一个基础，从这个问题可以考察出基础能力如何。

尼恩提示，分为两块作答：先说什么是pod（如上），再说 pod 的创建流程。

什么是pod？参考上面的两题？

再说pod的创建流程

1、客户端提交创建请求，

可以通过API Server的Restful API，也可以使用如何命令行工具。支持的数据类型包括JSON和YAML。

2、API Server处理用户请求，存储Pod数据到etcd。

3、调度器通过API Server查看未绑定的Pod。尝试为Pod分配主机。

4、过滤主机 (调度预选):

调度器用一组规则过滤掉不符合要求的主机。比如Pod指定了所需要的资源量，那么可用资源比Pod需要的资源量少的主机会被过滤掉。

5、主机打分(调度优选):

对第一步筛选出的符合要求的主机进行打分，在主机打分阶段，调度器会考虑一些整体优化策略，比如把容一个Replication Controller的副本分布到不同的主机上，使用最低负载的主机等。

6、选择主机:

选择打分最高的主机，进行binding操作，结果存储到etcd中。

7、kubelet根据调度结果执行Pod创建操作:

绑定成功后，scheduler会调用API Server的API在etcd中创建一个boundpod对象，描述在一个工作节点上绑定运行的所有pod信息。

运行在每个工作节点上的kubelet也会定期与etcd同步boundpod信息，一旦发现应该在该工作节点上运行的boundpod对象没有更新，则调用Docker API创建并启动pod内的容器。

简单讲一下 pod 创建过程

情况一、使用kubectl run命令创建的pod:

- 1 注意:
- 2 `kubectl run` 在旧版本中创建的是deployment,
- 3 但在新的版本中创建的是pod则其创建过程不涉及deployment

如果是单独的创建一个pod，则其创建过程是这样的:

- 1、首先，用户通过kubectl或其他api客户端工具提交需要创建的pod信息给apiserver;
- 2、apiserver验证客户端的用户权限信息，验证通过开始处理创建请求生成pod对象信息，并将信息存入etcd，然后返回确认信息给客户端;
- 3、apiserver开始反馈etcd中pod对象的变化，其他组件使用watch机制跟踪apiserver上的变动;
- 4、scheduler发现有新的pod对象要创建，开始调用内部算法机制为pod分配最佳的主机，并将结果信息更新至apiserver;
- 5、node节点上的kubelet通过watch机制跟踪apiserver发现有pod调度到本节点，尝试调用docker启动容器，并将结果反馈apiserver;

6、apiserver将收到的pod状态信息存入etcd中。
至此，整个pod创建完毕。

情况二、使用deployment来创建pod：

- 1、首先，用户使用kubect create命令或者kubect apply命令提交了要创建一个deployment资源请求；
- 2、api-server收到创建资源的请求后，会对客户端操作进行身份认证，在客户端的~/.kube文件夹下，已经设置好了相关的用户认证信息，这样api-server会知道我是哪个用户，并对此用户进行鉴权，当api-server确定客户端的请求合法后，就会接受本次操作，并把相关的信息保存到etcd中，然后返回确认信息给客户端。
- 3、apiserver开始反馈etcd中过程创建的对象的变化，其他组件使用watch机制跟踪apiserver上的变动。
- 4、controller-manager组件会监听api-server的信息，controller-manager是有多个类型的，比如Deployment Controller，它的作用就是负责监听Deployment，此时Deployment Controller发现有新的deployment要创建，那么它就会去创建一个ReplicaSet，一个ReplicaSet的产生，又被另一个叫做ReplicaSet Controller监听到了，紧接着它就会去分析ReplicaSet的语义，它了解到是要依照ReplicaSet的template去创建Pod，它一看这个Pod并不存在，那么就新建此Pod，当Pod刚被创建时，它的nodeName属性值为空，代表着此Pod未被调度。
- 5、调度器Scheduler组件开始介入工作，Scheduler也是通过watch机制跟踪apiserver上的变动，发现有未调度的Pod，则根据内部算法、节点资源情况，pod定义的亲和性反亲和性等等，调度器会综合的选出一批候选节点，在候选节点中选择一个最优的节点，然后将pod绑定该节点，将信息反馈给api-server。
- 6、kubelet组件部署于Node之上，它也是通过watch机制跟踪apiserver上的变动，监听到有一个Pod应该要被调度到自身所在Node上来，kubelet首先判断本地是否在此Pod，如果不存在，则会进入创建Pod流程，创建Pod有分为几种情况，第一种是容器不需要挂载外部存储，则相当于直接docker run把容器启动，但不会直接挂载docker网络，而是通过CNI调用网络插件配置容器网络，如果需要挂载外部存储，则还要调用CSI来挂载存储。kubelet创建完pod，将信息反馈给api-server，api-server将pod信息写入etcd。
- 7、Pod建立成功后，ReplicaSet Controller会对其持续进行关注，如果Pod因意外或被我们手动退出，ReplicaSet Controller会知道，并创建新的Pod，以保持replicas数量期望值。

pod有什么特点？

- 1、每个pod就像一个独立的逻辑机器，k8s会为每个pod分配一个集群内部唯一的IP地址，所以每个pod都拥有自己的IP地址、主机名、进程等；
- 2、一个pod可以包含1个或多个容器，1个容器一般被设计成只运行1个进程，1个pod只可能运行在单个节点上，即不可能1个pod跨节点运行，pod的生命周期是短暂，也就是说pod可能随时被消亡（如节点异常，pod异常等情况）；
- 2、每一个pod都有一个特殊的被称为"根容器"的pause容器，也称info容器，pause容器对应的镜像属于k8s平台的一部分，除了pause容器，每个pod还包含一个或多个跑业务相关组件的应用容器；
- 3、一个pod中的容器共享network命名空间；
- 4、一个pod里的多个容器共享pod IP，这就意味着1个pod里面的多个容器的进程所占用的端口不能相同，否则在这个pod里面就会产生端口冲突；既然每个pod都有自己的IP和端口空间，那么对不同的两个pod来说就不可能存在端口冲突；
- 5、应该将应用程序组织到多个pod中，而每个pod只包含紧密相关的组件或进程；
- 6、pod是k8s中扩容、缩容的基本单位，也就是说k8s中扩容缩容是针对pod而言而非容器。

pod的重启策略有哪些？

pod重启容器策略是指针对pod内所有容器的重启策略，不是重启pod，其可以通过restartPolicy字段配置pod重启容器的策略，如下：

- Always: 当容器终止退出后，总是重启容器，默认策略就是Always。
- OnFailure: 当容器异常退出，退出状态码非0时，才重启容器。
- Never: 当容器终止退出，不管退出状态码是什么，从不重启容器。

pod的镜像拉取策略有哪几种？

pod镜像拉取策略可以通过imagePullPolicy字段配置镜像拉取策略，

主要有3中镜像拉取策略，如下：

- IfNotPresent: 默认值，镜像在node节点宿主机上不存在时才拉取。
- Always: 总是重新拉取，即每次创建pod都会重新从镜像仓库拉取一次镜像。
- Never: 永远不会主动拉取镜像，仅使用本地镜像，需要你手动拉取镜像到node节点，如果node节点不存在镜像则pod启动失败。

kubernetes针对pod资源对象的健康监测机制？（必须记住3重探测方式，重点，经常问）

提供了三类probe（探针）来执行对pod的健康监测：

- livenessProbe探针（存活探针）：

可以根据用户自定义规则来判定pod是否健康，用于判断容器是否处于Running状态，

如果不是，kubelet就会杀掉该容器，并根据重启策略做相应的处理。如果容器不包含该探针，那么kubelet就会默认返回值都是success；

- ReadinessProbe探针:

同样是可以根据用户自定义规则来判断pod是否健康，容器服务是否可用（Ready），如果探测失败，控制器会将此pod从对应service的endpoint列表中移除，从此不再将任何请求调度到此Pod上，直到下次探测成功；

- startupProbe探针:

启动检查机制，应用一些启动缓慢的业务，避免业务长时间启动而被上面两类探针kill掉，

这个问题也可以换另一种方式解决，就是定义上面两类探针机制时，初始化时间定义的长一些即可；

备注：每种探测方法能支持以下几个相同的检查参数，用于设置控制检查时间：

- initialDelaySeconds：初始第一次探测间隔，用于应用启动的时间，防止应用还没启动而健康检查失败；
- periodSeconds：检查间隔，多久执行probe检查，默认为10s；
- timeoutSeconds：检查超时时长，探测应用timeout后为失败；
- successThreshold：成功探测阈值，表示探测多少次为健康正常，默认探测1次。

就绪探针（ReadinessProbe探针）与存活探针（livenessProbe探针）区别是什么？

两者作用不一样，

存活探针是将检查失败的容器杀死，创建新的启动容器来保持pod正常工作；

就绪探针是，当就绪探针检查失败，并不重启容器，而是将pod移出endpoint，就绪探针确保了service中的pod都是可用的，确保客户端只与正常的pod交互并且客户端永远不会知道系统存在问题。

存活探针的属性参数有哪几个？

存活探针的附加属性参数有以下几个：

- initialDelaySeconds：表示在容器启动后延时多久秒才开始探测；
- periodSeconds：表示执行探测的频率，即间隔多少秒探测一次，默认间隔周期是10秒，最小1秒；
- timeoutSeconds：表示探测超时时间，默认1秒，最小1秒，表示容器必须在超时时间范围内做出响应，否则视为本次探测失败；
- successThreshold：表示最少连续探测成功多少次才被认定为成功，默认是1，对于liveness必须是1，最小值是1；
- failureThreshold：表示连续探测失败多少次才被认定为失败，默认是3，连续3次失败，k8s将根据pod重启策略对容器做出决定；

注意：定义存活探针时，一定要设置initialDelaySeconds属性，该属性为初始延时，如果不设置，默认容器启动时探针就开始探测了，这样可能会存在应用程序还未启动就绪，就会导致探针检测失败，k8s就会根据pod重启策略杀掉容器然后再重新创建容器的莫名其妙的问题。

在生产环境中，一定要定义一个存活探针。

pod的就绪探针有哪几种？

我们知道，当一个pod启动后，就会立即加入service的endpoint ip列表中，并开始接收到客户端的链接请求，

假若此时pod中的容器的业务进程还没有初始化完毕，那么这些客户端链接请求就会失败，为了解决这个问题，kubernetes提供了就绪探针来解决这个问题的。

在pod中的容器定义一个就绪探针，就绪探针周期性检查容器，

如果就绪探针检查失败了，说明该pod还未准备就绪，不能接受客户端链接，则该pod将从endpoint列表中移除，

pod被剔除了，service就不会把请求分发给该pod，

然后就绪探针继续检查，如果随后容器就绪，则再重新把pod加回endpoint列表。

kubernetes提供了3种探测容器的存活探针，如下：

- httpGet：通过容器的IP、端口、路径发送http 请求，返回200-400范围内的状态码表示成功。
- exec：在容器内执行shell命令，根据命令退出状态码是否为0进行判断，0表示健康，非0表示不健康。

- TCPSocket：与容器的IP、端口建立TCP Socket链接，能建立则说明探测成功，不能建立则说明探测失败

pod的就绪探针的属性参数有哪些

就绪探针的附加属性参数有以下几个：

- initialDelaySeconds：延时秒数，即容器启动多少秒后才开始探测，不写默认容器启动就探测；
- periodSeconds：执行探测的频率（秒），默认为10秒，最低值为1；
- timeoutSeconds：超时时间，表示探测时在超时时间内必须得到响应，负责视为本次探测失败，默认为1秒，最小值为1；
- failureThreshold：连续探测失败的次数，视为本次探测失败，默认为3次，最小值为1次；
- successThreshold：连续探测成功的次数，视为本次探测成功，默认为1次，最小值为1次；

pod的重启策略是什么？

答：通过命令“[kubectl explain pod.spec](#)”查看pod的重启策略；

- Always：但凡pod对象终止就重启，此为默认策略；
- OnFailure：仅在pod对象出现错误时才重启；

k8s 创建一个pod的详细流程，涉及的组件怎么通信的？

答：

- 1) 客户端提交创建请求，可以通过 api-server 提供的 restful 接口，或者通过 kubectl 命令行工具，支持的数据类型包括 JSON 和 YAML；
- 2) api-server 处理用户请求，将 pod 信息存储至 etcd 中；
- 3) kube-scheduler 通过 api-server 提供的接口监控到未绑定的 pod，尝试为 pod 分配 node 节点，主要分为两个阶段，预选阶段和优选阶段，其中预选阶段是遍历所有的 node 节点，根据策略筛选出候选节点，而优选阶段是在第一步的基础上，为每一个候选节点进行打分，分数最高者胜出；
- 4) 选择分数最高的节点，进行 pod binding 操作，并将结果存储至 etcd 中；
- 5) 随后目标节点的 kubelet 进程通过 api-server 提供的接口监测到 kube-scheduler 产生的 pod 绑定事件，然后从 etcd 获取 pod 清单，下载镜像并启动容器；

简单描述一下pod的终止过程

- 1、用户向apiserver发送删除pod对象的命令；

- 2、apiserver中的pod对象信息会随着时间的推移而更新，在宽限期内（默认30s），pod被视为dead；
- 3、将pod标记为terminating状态；
- 4、kubectl在监控到pod对象为terminating状态了就会启动pod关闭过程；
- 5、endpoint控制器监控到pod对象的关闭行为时将其从所有匹配到此endpoint的server资源endpoint列表中删除；
- 6、如果当前pod对象定义了preStop钩子处理器，则在其被标记为terminating后会异步的方式启动执行；
- 7、pod对象中的容器进程收到停止信息；
- 8、宽限期结束后，若pod中还存在运行的进程，那么pod对象会收到立即终止的信息；
- 9、kubelet请求apiserver将此pod资源的宽限期设置为0从而完成删除操作，此时pod对用户已不可见。

pod的生命周期有哪几种？

pod生命周期有的5种状态（也称5种相位），如下：

- Pending（挂起）：API server已经创建pod，但是该pod还有一个或多个容器的镜像没有创建，包括正在下载镜像的过程；
- Running（运行中）：Pod内所有的容器已经创建，且至少有一个容器处于运行状态、正在启动或正在重启状态；
- Succeeded（成功）：Pod内所有容器均已退出，且不会再重启；
- Failed（失败）：Pod内所有容器均已退出，且至少有一个容器为退出失败状态
- Unknown（未知）：某于某种原因apiserver无法获取该pod的状态，可能由于网络通行问题导致；

pod一直处于pending状态一般有哪些情况，怎么排查？ (重点，持续更新)

（这个问题被问到的概率非常大）

一个pod一开始创建的时候，它本身就是会处于pending状态，这时可能是正在拉取镜像，正在创建容器的过程。

如果等了一会发现pod一直处于pending状态，

那么我们可以使用kubectl describe命令查看一下pod的Events详细信息。一般可能会有这么几种情况导致pod一直处于pending状态：

- 1、调度器调度失败。

Scheduler调度器无法为pod分配一个合适的node节点。

而这又会有很多种情况，比如，node节点处在cpu、内存压力，导致无节点可调度；pod定义了资源请求，没有node节点满足资源请求；node节点上有污点而pod没有定义容忍；pod中定义了亲和性或反亲和性而没有节点满足这些亲和性或反亲和性；以上是调度器调度失败的几种情况。

- 2、pvc、pv无法动态创建。

如果因为pvc或pv无法动态创建，那么pod也会一直处于pending状态，比如要使用StatefulSet 创建redis集群，因为粗心大意，定义的storageClassName名称写错了，那么会造成无法创建pvc，这种情况pod也会一直处于pending状态，或者，即使pvc是正常创建了，但是由于某些异常原因导致动态供应存储无法正常创建pv，那么这种情况pod也会一直处于pending状态。

DaemonSet资源对象的特性？

答：

DemonSet这种资源对象会在每个k8s集群中的节点上运行，并且每个节点只能运行一个pod，这是它和deployment资源对象的最大也是唯一的区别。

所以，在其yaml文件中，不支持定义replicas，

除此之外，与Deployment、RS等资源对象的写法相同，

DemonSet一般使用的场景有

- 在去做每个节点的日志收集工作；
- 监控每个节点的运行状态；

删除一个Pod会发生什么事情？

答：

Kube-apiserver会接受到用户的删除指令，默认有30秒时间等待优雅退出，超过30秒会被标记为死亡状态，

此时Pod的状态Terminating，kubelet看到pod标记为Terminating就开始了关闭Pod的工作；

关闭流程如下：

- 1) pod从service的endpoint列表中被移除；
- 2)如果该pod定义了一个停止前的钩子，其会在pod内部被调用，停止钩子一般定义了如何优雅地结束进程；
- 3)进程被发送TERM信号（kill -14）；
- 4)当超过优雅退出的时间后，Pod中的所有进程都会被发送SIGKILL信号（kill -9）；

pod的共享资源？

答：

- 1) PID 命名空间：Pod 中的不同应用程序可以看到其他应用程序的进程 ID；
- 2) 网络命名空间：Pod 中的多个容器能够访问同一个IP和端口范围；
- 3) IPC 命名空间：Pod 中的多个容器能够使用 SystemV IPC 或 POSIX 消息队列进行通信；
- 4) UTS 命名空间：Pod 中的多个容器共享一个主机名；
- 5) Volumes（共享存储卷）：Pod 中的各个容器可以访问在 Pod 级别定义的 Volumes；

pod的初始化容器是干什么的？

init container，初始化容器用于在启动应用容器之前完成应用容器所需要的前置条件，

初始化容器本质上和应用容器是一样的，但是初始化容器是仅允许一次就结束的任务，初始化容器具有两大特征：

- 1、初始化容器必须运行完成直至结束，若某初始化容器运行失败，那么kubernetes需要重启它直到成功完成；
- 2、初始化容器必须按照定义的顺序执行，当且仅当前一个初始化容器成功之后，后面的一个初始化容器才能运行；

pod的资源请求、限制如何定义？

pod的资源请求、资源限制可以直接在pod中定义

主要包括两块内容，

- limits，限制pod能使用的最大cpu和内存，
- requests，pod启动时申请的cpu和内存。

```
1 resources:                                #资源配额
2     limits:                                #限制最大资源，上限
3         cpu: 2                             #CPU限制，单位是code数
4         memory: 2G                         #内存最大限制
5     requests:                              #请求资源（最小，下限）
6         cpu: 1                             #CPU请求，单位是code数
7         memory: 500G                      #内存最小请求
8
```

pod的定义中有个command和args参数，这两个参数不会和docker镜像的entrypoint冲突吗？

不会。

在pod中定义的command参数用于指定容器的启动命令列表，如果不指定，则默认使用Dockerfile打包时的启动命令，args参数用于容器的启动命令需要的参数列表；

特别说明：

kubernetes中的command、args其实是实现覆盖dockerfile中的ENTRYPOINT的功能的。

- 1、如果command和args均没有写，那么使用Dockerfile的配置；
- 2、如果command写了但args没写，那么Dockerfile默认的配置会被忽略，执行指定的command；
- 3、如果command没写但args写了，那么Dockerfile中的ENTRYPOINT的会被执行，使用当前args的参数；
- 4、如果command和args都写了，那么Dockerfile会被忽略，执行输入的command和args。

pause容器作用是什么？

每个pod里运行着一个特殊的被称之为pause的容器，也称根容器，而其他容器则称为业务容器；

创建pause容器主要是为了为业务容器提供 Linux命名空间，共享基础：包括 pid、icp、net 等，以及启动 init 进程，并收割僵尸进程；

这些业务容器共享pause容器的网络命名空间和volume挂载卷，

当pod被创建时，pod首先会创建pause容器，从而把其他业务容器加入pause容器，从而让所有业务容器都在同一个命名空间中，这样就可以实现网络共享。

pod还可以共享存储，在pod级别引入数据卷volume，业务容器都可以挂载这个数据卷从而实现持久化存储。

标签及标签选择器是什么，如何使用？

标签是键值对类型，标签可以附加到任何资源对象上，主要用于管理对象，查询和筛选。

标签常被用于标签选择器的匹配度检查，从而完成资源筛选；一个资源可以定义一个或多个标签在其上面。

标签选择器，标签要与标签选择器结合在一起，标签选择器允许我们选择标记有特定标签的资源对象子集，如pod，并对这些特定标签的pod进行查询，删除等操作。

标签和标签选择器最重要的使用之一在于，在deployment中，在pod模板中定义pod的标签，然后在deployment定义标签选择器，这样就通过标签选择器来选择哪些pod是受其控制的，service也是通过标签选择器来关联哪些pod最后其服务后端pod。

service是如何与pod关联的？

答案是通过标签选择器，每一个由deployment创建的pod都带有标签，

这样，service就可以定义标签选择器来关联哪些pod是作为其后端了，就是这样，service就与pod管联在一起了。

service的域名解析格式、pod的域名解析格式

service的DNS域名表示格式为 `<servicename>.<namespace>.svc.<clusterdomain>`，

servicename是service的名称，namespace是service所处的命名空间，clusterdomain是k8s集群设置的域名后缀，一般默认为 cluster.local

pod的DNS域名格式为： `<pod-ip>.<namespace>.pod.<clusterdomain>`，

其中，pod-ip需要使用-将ip直接的点替换掉，namespace为pod所在的命名空间，clusterdomain是k8s集群设置的域名后缀，一般默认为 cluster.local，

演示如下： `10-244-1-223.default.pod.cluster.local`

对于deployment、daemonsets等创建的pod，还还可以通过 `<pod-ip>.<deployment-name>.<namespace>.svc.<clusterdomain>` 这样的域名访问。

service的类型有哪几种

service的类型一般有4中，分别是：

- ClusterIP：表示service仅供集群内部使用，默认值就是ClusterIP类型
- NodePort：表示service可以对外访问应用，会在每个节点上暴露一个端口，这样外部浏览器访问地址为：任意节点的IP：NodePort就能连上service了
- LoadBalancer：表示service对外访问应用，这种类型的service是公有云环境下的service，此模式需要外部云厂商的支持，需要有一个公网IP地址
- ExternalName：这种类型的service会把集群外部的服务引入集群内部，这样集群内直接访问service就可以间接的使用集群外部服务了

一般情况下，service都是ClusterIP类型的，通过ingress接入的外部流量。

Pod到Service的通信？

- 1) k8s在创建服务时为服务分配一个虚拟IP，客户端通过该IP访问服务，服务则负责将请求转发到后端Pod上；
- 2) Service是通过kube-proxy服务进程实现，该进程在每个Node上均运行可以看作一个透明代理兼负载均衡器；
- 3) 对每个TCP类型Service，kube-proxy都会在本节点上建立一个SocketServer来负责接受请求，然后均匀发送到后端Pod默认采用Round Robin负载均衡算法；
- 4) Service的Cluster IP与NodePort等概念是kube-proxy通过Iptables的NAT转换实现，kube-proxy进程动态创建与服务相关的Iptables规则；
- 5) kube-proxy通过查询和监听API Server中Service与Endpoints的变化来实现其主要功能，包括为新创建的Service打开一个本地代理对象，接收请求针对针对发生变化的Service列表，kube-proxy会逐个处理；

一个应用pod是如何发现service的，或者说，pod里面的容器用于是如何连接service的？

答：有两种方式，一种是通过环境变量，另一种是通过service的dns域名方式。

1、环境变量：

当pod被创建之后，k8s系统会自动为容器注入集群内有效的service名称和端口号等信息为环境变量的形式，

这样容器应用直接通过取环境变量值就能访问service了，

如 `curl http://${WEBAPP_SERVICE_HOST}:${WEBAPP_SERVICE_PORT}`

2、DNS方式：

使用dns域名解析的前提是k8s集群内有DNS域名解析服务器，

默认k8s中会有一个CoreDNS作为k8s集群的默认DNS服务器提供域名解析服务器；

service的DNS域名表示格式为 `<servicename>.<namespace>.svc.<clusterdomain>`，

servicename是service的名称，namespace是service所处的命名空间，clusterdomain是k8s集群设置的域名后缀，一般默认为 cluster.local，

这样容器应用直接通过service域名就能访问service了，

如 `wget http://svc-deployment-nginx.default.svc.cluster.local:80`，

另外，service的port端口如果定义了名称，那么port也可以通过DNS进行解析，

格式为： `__<portname>._<protocol>.<servicename>.<namespace>.svc.<clusterdomain>`

如何创建一个service代理外部的服务，或者换句话说来说，在k8s集群内的应用如何访问外部的服务，如数据库服务，缓存服务等？

答：可以通过创建一个没有标签选择器的service来代理集群外部的服务。

1、创建service时不指定selector标签选择器，但需要指定service的port端口、端口的name、端口协议等，这样创建出来的service因为没有指定标签选择器就不会自动创建endpoint；

2、手动创建一个与service同名的endpoint，endpoint中定义外部服务的IP和端口，endpoint的名称一定要与service的名称一样，端口协议也要一样，端口的name也要与service的端口的name一样，不然endpoint不能与service进行关联。

完成以上两步，k8s会自动将service和同名的endpoint进行关联，

这样，k8s集群内的应用服务直接访问这个service就可以相当于访问外部的服务了。

聊聊：service、endpoint、kube-proxys三种的关系是什么？

service：

在kubernetes中，service是一种为一组功能相同的pod提供单一不变的接入点的资源。

当service被建立时，service的IP和端口不会改变，这样外部的客户端（也可以是集群内部的客户端）通过service的IP和端口来建立链接，这些链接会被路由到提供该服务的任意一个pod上。

通过这样的方式，客户端不需要知道每个单独提供服务的pod地址，这样pod就可以在集群中随时被创建或销毁。

endpoint：

service维护一个叫endpoint的资源列表，endpoint资源对象保存着service关联的pod的ip和端口。

从表面上看，当pod消失，service会在endpoint列表中剔除pod，当有新的pod加入，service就会将pod ip加入endpoint列表；

但是正在底层的逻辑是，endpoint的这种自动剔除、添加、更新pod的地址其实底层是由 `endpoint controller` 控制的，`endpoint controller` 负责监听service和对应的pod副本的变化，如果监听到service被删除，则删除和该service同名的endpoint对象，如果监听到新的service被创建或者修改，则根据该service信息获取相关pod列表，然后创建或更新service对应的endpoint对象，如果监听到pod事件，则更新它所对应的service的endpoint对象。

kube-proxy:

kube-proxy运行在node节点上，在Node节点上实现Pod网络代理，维护网络规则和四层负载均衡工作，

`kube-proxy` 会监听 `api-server` 中从而获取service和endpoint的变化情况，创建并维护路由规则以提供服务IP和负载均衡功能。

简单理解此进程是Service的透明代理兼负载均衡器，其核心功能是将到某个Service的访问请求转发到后端的多个Pod实例上。

聊聊：k8s中pod之间是怎么通信的？

tips: 金山云问了这个问题，主要考察一些基础的网络模式。

- 1.同一个Pod内的多个容器之间：lo（通过localhost回环地址通信）。
- 2.各Pod之间的通讯：overlay Network（覆盖网络）。
- 3.Pod与 Service之间的通讯：（使用各节点的 Iptables规则）。

聊聊：无头service和普通的service有什么区别，无头service使用场景是什么？

答：

无头service没有cluster ip，在定义service时将 `service.spec.clusterIP: None`，就表示创建的是无头service。

普通的service是用于为一组后端pod提供请求连接的负载均衡，让客户端能通过固定的service ip地址来访问pod，这类的pod是没有状态的，同时service还具有负载均衡和服务发现的功能。普通service跟我们平时使用的nginx反向代理很相识。

试想这样一种情况，有6个redis pod，它们相互之间要通信并要组成一个redis集群，

不需要所谓的service负载均衡，这时无头service就是派上用场了，

无头service由于没有cluster ip，kube-proxy就不会处理它也就不会对它生成规则负载均衡，无头service直接绑定的是pod的ip。无头service仍会有标签选择器，有标签选择器就会有endpoint资源。

无头service使用场景：

无头service一般用于有状态的应用场景，如Kafka集群、Redis集群等，这类pod之间需要相互通信相互组成集群，不在需要所谓的service负载均衡。

deployment怎么扩容或缩容?

答：直接修改pod副本数即可，可以通过下面的方式来修改pod副本数：

- 1、直接修改yaml文件的replicas字段数值，然后 `kubectl apply -f xxx.yaml` 来实现更新；
- 2、使用 `kubectl edit deployment xxx` 修改replicas来实现在线更新；
- 3、使用 `kubectl scale --replicas=5 deployment/deployment-nginx` 命令来扩容缩容。

deployment的更新升级策略有哪些?

答：deployment的升级策略主要有两种。

- 1、Recreate 重建更新：这种更新策略会杀掉所有正在运行的pod，然后再重新创建的pod；
- 2、rollingUpdate 滚动更新：这种更新策略，deployment会以滚动更新的方式来逐个更新pod，同时通过设置滚动更新的两个参数 `maxUnavailable`、`maxSurge` 来控制更新的过程。

deployment的滚动更新策略有两个特别主要的参数，解释一下它们是什么意思?

答：deployment的滚动更新策略，rollingUpdate 策略，主要有两个参数，`maxUnavailable`、`maxSurge`。

- `maxUnavailable`：最大不可用数，`maxUnavailable`用于指定deployment在更新的过程中不可用状态的pod的最大数量，`maxUnavailable`的值可以是一个整数值，也可以是pod期望副本的百分比，如25%，计算时向下取整。
- `maxSurge`：最大激增数，`maxSurge`指定deployment在更新的过程中pod的总数量最大能超过pod副本数多少个，`maxUnavailable`的值可以是一个整数值，也可以是pod期望副本的百分比，如25%，计算时向上取整。

聊聊：deployment更新的命令有哪些?

答：可以通过三种方式来实现更新deployment。

- 1、直接修改yaml文件的镜像版本，然后 `kubectl apply -f xxx.yaml` 来实现更新；
- 2、使用 `kubectl edit deployment xxx` 实现在线更新；
- 3、使用 `kubectl set image deployment/nginx busybox=busybox nginx=nginx:1.9.1` 命令来更新。

简述一下deployment的更新过程?

deployment是通过控制replicaset来实现，由replicaset真正创建pod副本，每更新一次deployment，都会创建新的replicaset，下面来举例deployment的更新过程：

假设要升级一个nginx-deployment的版本镜像为nginx:1.9，deployment的定义滚动更新参数如下：

```
1 replicas: 3
2 deployment.spec.strategy.type: RollingUpdate
3 maxUnavailable: 25%
4 maxSurge: 25%
5
```

通过计算我们得出， $3 * 25\% = 0.75$ ，maxUnavailable是向下取整，则maxUnavailable=0，maxSurge是向上取整，则maxSurge=1，所以我们得出在整个deployment升级镜像过程中，不管旧的pod和新的pod是如何创建消亡的，pod总数最大不能超过 $3 + \text{maxSurge} = 4$ 个，最大pod不可用数 $3 - \text{maxUnavailable} = 3$ 个。

现在具体讲一下deployment的更新升级过程：

使用 `kubectl set image deployment/nginx nginx=nginx:1.9 --record` 命令来更新；

- 1、deployment创建一个新的replaceset，先新增1个新版本pod，此时pod总数为4个，不能再新增了，再新增就超过pod总数4个了；旧=3，新=1，总=4；
- 2、减少一个旧版本的pod，此时pod总数为3个，这时不能再减少了，再减少就不满足最大pod不可用数3个了；旧=2，新=1，总=3；
- 3、再新增一个新版本的pod，此时pod总数为4个，不能再新增了；旧=2，新=2，总=4；
- 4、减少一个旧版本的pod，此时pod总数为3个，这时不能再减少了；旧=1，新=2，总=3；
- 5、再新增一个新版本的pod，此时pod总数为4个，不能再新增了；旧=1，新=3，总=4；
- 6、减少一个旧版本的pod，此时pod总数为3个，更新完成，pod都是新版本了；旧=0，新=3，总=3；

聊聊：deployment的回滚使用什么命令

在升级deployment时kubectl set image 命令加上 --record 参数可以记录具体的升级历史信息，

使用 `kubectl rollout history deployment/deployment-nginx` 命令来查看指定的deployment升级历史记录，

如果需要回滚到某个指定的版本，可以使用 `kubectl rollout undo deployment/deployment-nginx --to-revision=2` 命令来实现。

讲一下都有哪些存储卷，作用分别是什么？

卷	作用	常用场景
emptyDir	用于存储临时数据的简单空目录	一个pod中的多个容器需要共享彼此的数据，emptyDir的数据随着容器的消亡也会销毁
hostPath	用于将目录从工作节点的文件系统挂载到pod中	不常用，缺点是，pod的调度不是固定的，也就是当pod消失后 deployment重新创建一个pod，而这pod如果不是被调度到之前pod的节点，那么该pod就不能访问之前的数据
configMap	用于将非敏感的数据保存到键值对中，使用时可以使用作为环境变量、命令行参数arg，存储卷被pods挂载使用	将应用程序的不敏感配置文件创建为 configmap卷，在pod中挂载 configmap卷，可是实现热更新
secret	主要用于存储和管理一些敏感数据，然后通过 Pod 的容器里挂载 Volume 的方式或者环境变量的方式访问到这些 Secret 里保存的信息了，pod会自动解密Secret 的信息	将应用程序的账号密码等敏感信息通过secret卷的形式挂载到pod中使用
downwardApi	主要用于暴露pod元数据，如pod的名字	pod中的应用程序需要指定pod的 name等元数据，就可以通过 downwardApi 卷的形式挂载给pod使用
projected	这是一种特殊的卷，用于将上面这些卷一次性的挂载给pod使用	将上面这些卷一次性的挂载给pod使用
pvc	pvc是存储卷声明	通常会创建pvc表示对存储的申请，然后在pod中使用pvc
网络存储卷	pod挂载网络存储卷，这样就能将数据持久化到后端的存储里	常见的网络存储卷有nfs存储、glusterfs 卷、ceph rbd存储卷

聊聊：pv的访问模式有哪几种

pv的访问模式有3种，如下：

- ReadWriteOnce，简写：RWO 表示，只允许单个节点以读写方式挂载；
- ReadOnlyMany，简写：ROX 表示，可以被许多节点以只读方式挂载；
- ReadWriteMany，简写：RWX 表示，可以被多个节点以读写方式挂载；

聊聊：pv的回收策略有哪几种

主要有3中回收策略：retain 保留、delete 删除、Recycle回收。

- Retain：保留，该策略允许手动回收资源，当删除PVC时，PV仍然存在，PV被视为已释放，管理员可以手动回收卷。

- Delete：删除，如果Volume插件支持，删除PVC时会同时删除PV，动态卷默认为Delete，目前支持Delete的存储后端包括AWS EBS，GCE PD，Azure Disk，OpenStack Cinder等。
- Recycle：回收，如果Volume插件支持，Recycle策略会对卷执行rm -rf清理该PV，并使其可用于下一个新的PVC，但是本策略将来会被弃用，目前只有NFS和HostPath支持该策略。（这种策略已经被废弃，不用记）

聊聊：在pv的生命周期中，一般有几中状态

pv一共有4中状态，分别是：

创建pv后，pv的状态有以下4种：Available（可用）、Bound（已绑定）、Released（已释放）、Failed（失败）

- 1 Available，表示pv已经创建正常，处于可用状态；
- 2 Bound，表示pv已经被某个pvc绑定，注意，一个pv一旦被某个pvc绑定，那么该pvc就独占该pv，其他pvc不能再与该pv绑定；
- 3 Released，表示pvc被删除了，pv状态就会变成已释放；
- 4 Failed，表示pv的自动回收失败；
- 5

聊聊：pv存储空间不足怎么扩容？

一般的，我们会使用动态分配存储资源，

在创建storageclass时指定参数 allowVolumeExpansion：true，表示允许用户通过修改pvc申请的存储空间自动完成pv的扩容，

当增大pvc的存储空间时，不会重新创建一个pv，而是扩容其绑定的后端pv。

这样就能完成扩容了。但是allowVolumeExpansion这个特性只支持扩容空间不支持减少空间。

聊聊：存储类的资源回收策略：

主要有2中回收策略，delete 删除，默认就是delete策略、retain 保留。

Retain：保留，该策略允许手动回收资源，当删除PVC时，PV仍然存在，PV被视为已释放，管理员可以手动回收卷。

Delete：删除，如果Volume插件支持，删除PVC时会同时删除PV，动态卷默认为Delete，目前支持Delete的存储后端包括AWS EBS，GCE PD，Azure Disk，OpenStack Cinder等。

注意：使用存储类动态创建的pv默认继承存储类的回收策略，当然当pv创建后你也可以手动修改pv的回收策略。

聊聊：怎么使一个node脱离集群调度，比如要停机维护单又不能影响业务应用

使用kubectl drain 命令

聊聊：k8s生产中遇到什么特别映像深刻的问题吗，问题排查解决思路是怎么样的？（重点）

（此问题被问到的概率高达90%，所以可以自己准备几个自己在生产环境中遇到的问题进行讲解）

答：前端的lb负载均衡服务器上的keepalived出现过脑裂现象。

- 1、当时问题现象是这样的，vip同时出现在主服务器和备服务器上，但业务上又没受到影响；
- 2、这时首先去查看备服务器上的keepalived日志，发现有日志信息显示凌晨的时候备服务器出现了vrrp协议超时，所以才导致了备服务器接管了vip；查看主服务器上的keepalived日志，没有发现明显的报错信息，继续查看主服务器和备服务器上的keepalived进程状态，都是running状态的；查看主服务器上检测脚本所检测的进程，其进程也是正常的，也就是说主服务器根本没有成功执行检测脚本（成功执行检查脚本是会kill掉keepalived进程，脚本里面其实就是配置了检查nginx进程是否存活，如果检查到nginx不存活则kill掉keepalived，这样来实现备服务器接管vip）；
- 3、排查服务器上的防火墙、selinux，防火墙状态和selinux状态都是关闭着的；
- 4、使用tcpdump工具在备服务器上进行抓取数据包分析，分析发现，现在确实是备接管的vip，也确实是备服务器也在对外发送vrrp心跳包，所以现在外部流量应该都是流入备服务器上的vip；
- 5、怀疑：主服务器上设置的vrrp心跳包时间间隔太长，以及检测脚本设置的检测时间设置不合理导致该问题；
- 6、修改vrrp协议的心跳包时间间隔，由原来的2秒改成1秒就发送一次心跳包；检测脚本的检测时间也修改短一点，同时还修改检测脚本的检测失败的次数，比如连续检测2次失败才认定为检测失败；
- 7、重启主备上的keepalived，现在keepalived是正常的，主服务器上有vip，备服务器上没有vip；
- 8、持续观察：第二天又发现keepalived出现过脑裂现象，vip又同时出现在主服务器和备服务器上，又是凌晨的时候备服务器显示vrrp心跳包超时，所以才导致备服务器接管了vip；
- 9、同样的时间，都是凌晨，vrrp协议超时；很奇怪，很有理由怀疑是网络问题，询问第三方厂家上层路由器是否禁止了vrrp协议，第三方厂家回复，没有禁止vrrp协议；
- 10、百度、看官方文档求解；
- 11、百度、看官网文档得知，keepalived有2种传播模式，一种是组播模式，一种是单播模式，keepalived默认在组播模式下工作，主服务器会往主地址224.0.0.18发送心跳包，当局域网内有多多个keepalived实例的时候，如果都用主播模式，会存在冲突干扰的情况，所以官方建议使用单播模式通信，单播模式就是点对点通行，即主向备服务器一对一的发送心跳包；
- 12、将keepalived模式改为单播模式，继续观察，无再发生脑裂现象。问题得以解决。

聊聊：k8s生产中遇到什么特别映像深刻的问题吗，问题排查解决思路是怎么样的？（重点）

参考答案二：测试环境二进制搭建etcd集群，etcd集群出现2个leader的现象。

- 1、问题现象就是：刚搭建的k8s集群，是测试环境的，搭建完成之后发现，使用kubectl get nodes 显示没有资源，kubectl get namespace 一会能正常显示全部的命名空间，一会又显示不了命名空间，这种奇怪情况。
- 2、当时经验不是很足，第一点想到的是不是因为网络插件calico没装导致的，但是想想，即使没有安装网络插件，最多是node节点状态是notready，也不可能是没有资源发现呀；
- 3、然后想到etcd数据库，k8s的资源都是存储在etcd数据库中的；

- 4、查看etcd进程服务的启动状态，发现etcd服务状态是处于running状态，但是日志有大量的报错信息，日志大概报错信息就是集群节点的id不匹配，存在冲突等等报错信息；
- 5、使用etcdctl命令查看etcd集群的健康状态，发现集群是health状态，但是居然显示有2个leader，这很奇怪（当初安装etcd的时候其实也只是简单看到了集群是健康状态，然后没注意到有2个leader，也没太关注etcd服务进程的日志报错信息，以为etcd集群状态是health状态就可以了）
- 6、现在etcd出现了2个leader，肯定是存在问题的；
- 7、全部检测一遍etcd的各个节点的配置文件，确认配置文件里面各个参数配置都没有问题，重启etcd集群，报错信息仍未解决，仍然存在2个leader；
- 8、尝试把其中一个leader节点踢出集群，然后再重新添加它进入集群，仍然是报错，仍然显示有2个leader；
- 9、尝试重新生成etcd的证书，重新颁发etcd的证书，问题仍然存在，仍然显示有2个leader；日志仍是报错集群节点的id不匹配，存在冲突；
- 10、计算etcd命令的MD5值，确保各个节点的etcd命令是相同的，确保在scp传输的时候没有损耗等等，问题仍未解决；
- 11、无解，请求同事，架构师介入帮忙排查问题，仍未解决；
- 12、删除全部etcd相关的文件，重新部署etcd集群，etcd集群正常了，现在只有一个leader，使用命令kubectl get nodes 查看节点，也能正常显示了；
- 13、最终问题的原因也没有定位出来，只能怀疑是环境问题，由于是刚部署的k8s测试环境，etcd里面没有数据，所以可以删除重新创建etcd集群，如果是线上环境的etcd集群出现这种问题，就不能随便删除etcd集群了，必须要先进行数据备份才能进行其他方法的处理。

聊聊：etcd集群节点可以设置为偶数个吗，为什么要设置为基数个呢？

不能，也不建议这么设置。

底层的原理，涉及到集群的脑裂，

具体的答案，请参考《尼恩java面试宝典 专题14》

聊聊：什么是Redis的脑裂问题？该如何解决？

什么是脑裂问题？脑裂问题，通常分布式、高可用集群，是因为网络问题导致的共性问题。

所以，脑裂不是redis 集群的专利，zookeeper 集群/elasticSearch 集群 等等，都存在脑裂问题。

step1: 回答一下，什么是脑裂问题？

在一个高可用集群中，当多个服务器在指定的时间内，由于网络的原因无法互相检测到对方，而各自形成一个新的小规模集群，并且各小集群当中，会选举新的master节点，都对外提供独立的服务，由于网络断裂的原因，一个高可用集群中，实际上分裂为多个小的集群，这种情况就称为脑裂。

也有的人称之为分区集群，或者大脑垂直分隔，互相接管对方的资源，出现多个Master的情况，都可以称为脑裂。

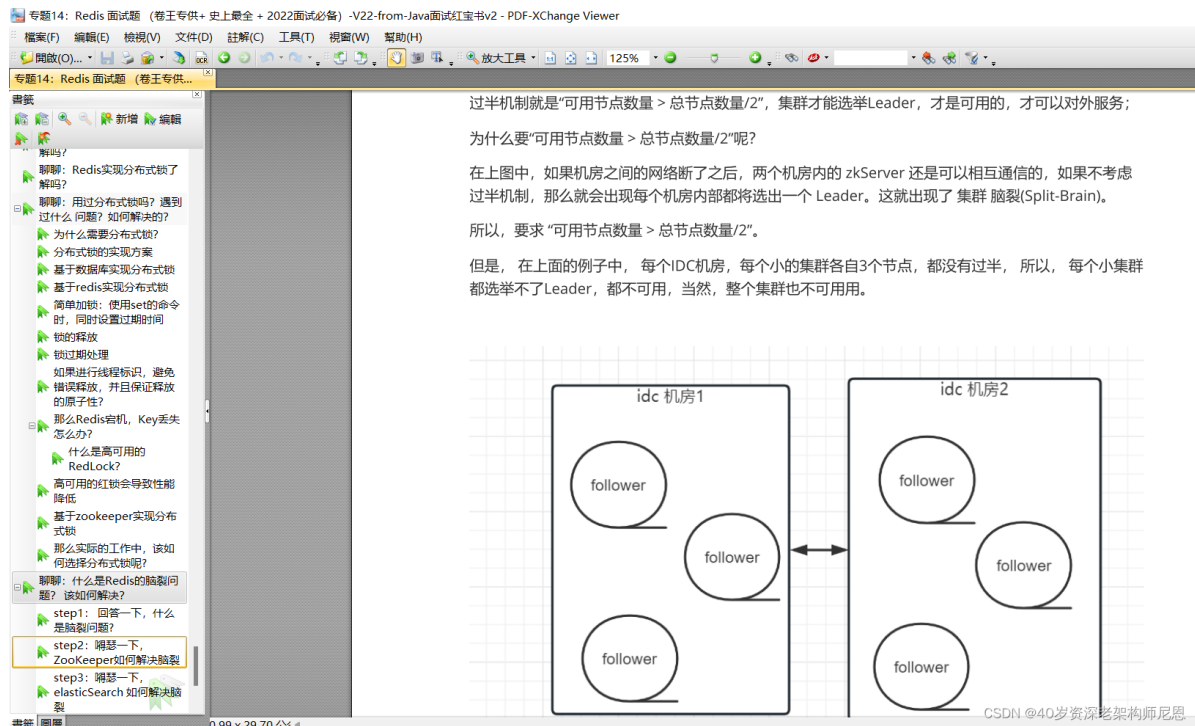
脑裂(Split-Brain)是一个形象的比喻，好比“大脑分裂”，也就是本来一个“大脑”被拆分了两个或多个“大脑”。

脑裂之后的后果是，可能会导致服务器端的数据不一致，或造成数据的丢失

step2: 嗨瑟一下，ZooKeeper如何解决脑裂

安装ZooKeeper之前，需要规划一下节点，ZooKeeper节点数有一个重要的要求：**ZooKeeper集群节**

安装ZooKeeper之前，需要规划一下节点，ZooKeeper节点数有一个重要的要求：**ZooKeeper集群节**



聊聊：etcd集群节点之间是怎么同步数据的？

总体而言，是通过Raft协议进行节点之间数据同步，保证节点之间的数据一致性

在正式开始介绍 Raft 协议之前，我们有必要简单介绍一下其相关概念。

在现实的场景中，节点之间的一致性也就很难保证，这样就需要 Paxos、Raft 等一致性协议。

一致性协议可以保证在集群中大部分节点可用的情况下，集群依然可以工作并给出一个正确的结果，从而保证依赖于该集群的其他服务不受影响。

这里的“大部分节点可用”指的是集群中超过半数以上的节点可用，例如，集群中共有 5 个节点，此时其中有 2 个节点出现故障宕机，剩余的可用节点数为 3，此时，集群中大多数节点处于可用的状态，从外部来看集群依然是可用的。

常见的一致性算法有 Paxos、Raft 等，

Paxos 协议是 Leslie Lamport 于 1990 年提出的一种基于消息传递的、具有高度容错特性的一致性算法，Paxos 算法解决的主要问题是分布式系统内如何就某个值达成一致。在相当长的一段时间内，Paxos 算法几乎成为一致性算法的代名词，

但是 Paxos 有两个明显的缺点：第一个也是最明显的缺点就是 Paxos 算法难以理解，Paxos 算法的论文本身就比较晦涩难懂，要完全理解 Paxos 协议需要付出较大的努力，很多经验丰富的开发者在看完 Paxos 论文之后，无法将其有效地应用到具体工程实践中，这明显增加了工程化的门槛，也正因如此，才出现了几次用更简单的术语来解释 Paxos 的尝试。

Paxos 算法的第二个缺点就是它没有提供构建现实系统的良好基础，也有很多工程化 Paxos 算法的尝试，但是它们对 Paxos 算法本身做了比较大的改动，彼此之间的实现差距都比较大，实现的功能和目的都有所不同，同时与 Paxos 算法的描述有很多出入。例如，著名 Chubby，它实现了一个类 Paxos 的算法，但其中很多细节并未被明确。本章并不打算详细介绍 Paxos 协议的相关内容，如果读者对 Paxos 感兴趣，则可以参考 Lamport 发表的三篇论文：《The Part-Time Parliament》、《Paxos made simple》、《Fast Paxos》。

Raft算法是一种用于管理复制日志的一致性算法，其功能与Paxos算法相同类似，但其算法结构和Paxos算法不同，在设计Raft算法时设计者就将易于理解作为其目标之一，这使得Raft算法更易于构建实际的系统，大幅度减少了工程化的工作量，也方便开发者此基础上进行扩展。

Raft协议中，核心就是用于：

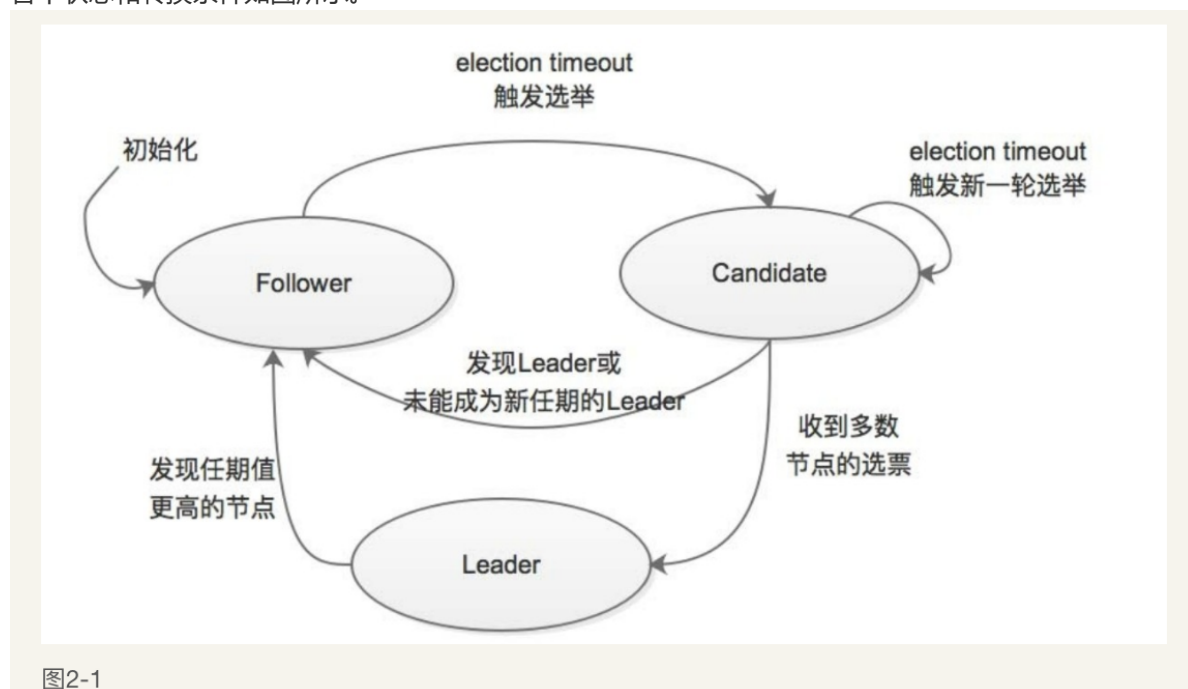
- Leader选举
- 日志复制。

Leader选举

Raft 协议的工作模式是一个 Leader 节点和多个 Follower 节点的模式，也就是常说的Leader-Follower 模式。

在 Raft 协议中，每个节点都维护了一个状态机，该状态机有三种状态，分别是Leader状态、Follower状态和Candidate状态，在任意时刻，集群中的任意一个节点都处于这三个状态之一。

各个状态和转换条件如图所示。



在多数情况下，集群中有一个Leader节点，其他节点都处于Follower状态，下面简单介绍一下每个状态的节点负责的主要工作。

- Leader节点负责处理所有客户端的请求，当接收到客户端的写入请求时，Leader节点会在本地追加一条相应的日志，然后将其封装成消息发送到集群中其他的Follower节点。当Follower节点收到该消息时会对其进行响应。如果集群中多数（超过半数）节点都已收到该请求对应的日志记录时，则Leader节点认为该条日志记录已提交（committed），可以向客户端返回响应。Leader还会处理客户端的只读请求，其中涉及一个简单的优化，后面介绍具体实现时，再进行详细介绍。Leader节点的另一项工作是定期向集群中的Follower节点发送心跳消息，这主要是为了防止集群中的其他Follower节点的选举计时器超时而触发新一轮选举。
- Follower节点不会发送任何请求，它们只是简单地响应来自Leader或者Candidate的请求；Follower节点也不处理Client的请求，而是将请求重定向给集群的Leader节点进行处理。
- Candidate节点是由Follower节点转换而来的，当Follower节点长时间没有收到Leader节点发送的心跳消息时，则该节点的选举计时器就会过期，同时会将自身状态转换成Candidate，发起新一轮选举。选举的具体过程在下面详细描述。

了解了Raft协议中节点的三种状态及各个状态下节点的主要行为之后，我们通过一个示例介绍Raft协议中Leader选举的大致流程。为了方便描述，我们假设当前集群中有三个节点（A、B、C），如图所示。

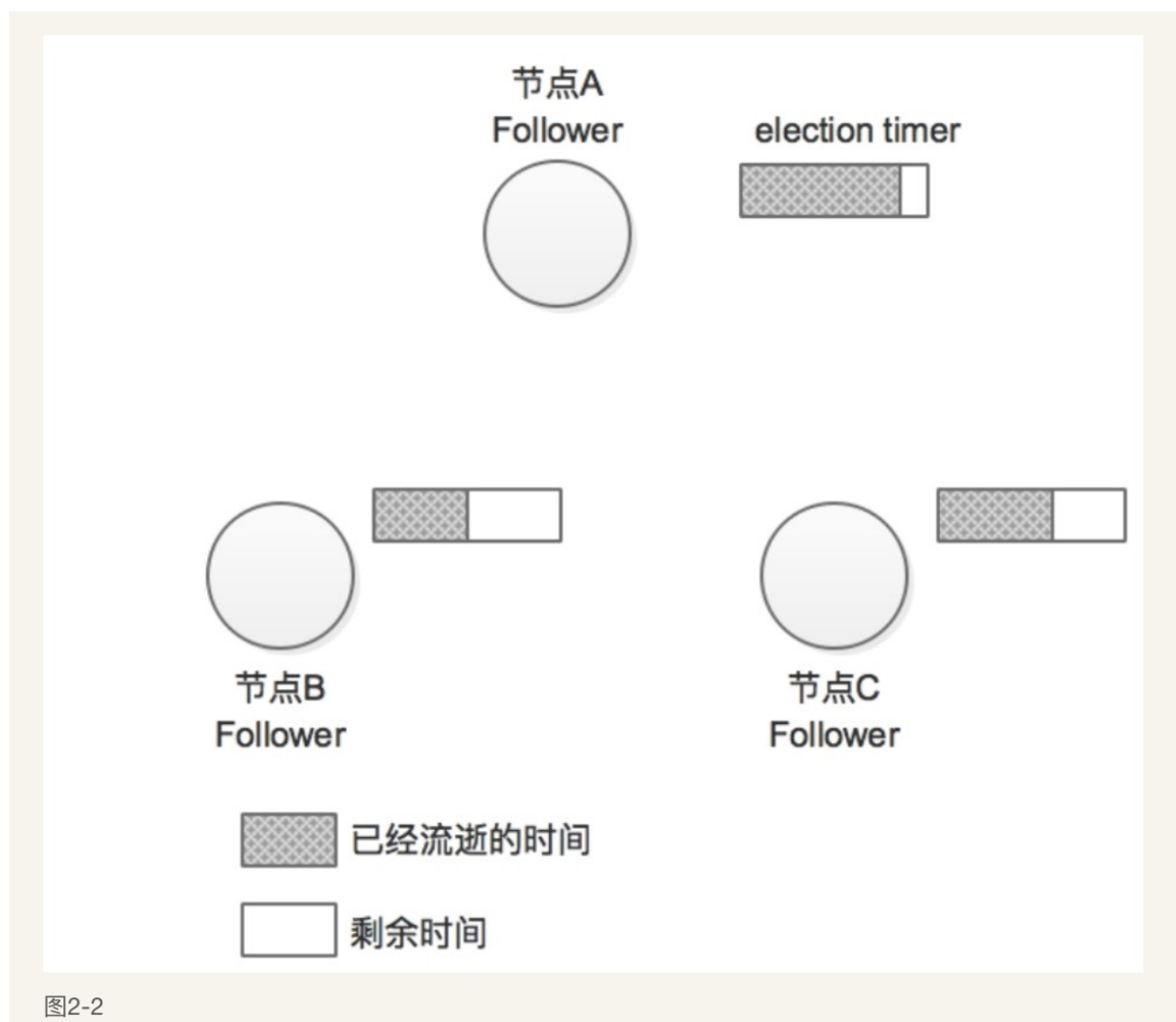


图2-2

在Raft协议中有两个时间控制Leader选举发生，其中一个选举超时时间（election timeout），每个Follower节点在接收不到Leader节点的心跳消息之后，并不会立即发起新一轮选举，而是需要等待一段时间之后才切换成Candidate状态发起新一轮选举。这段等待时长就是这里所说的election timeout（后面介绍etcd的具体实现时会提到，Follower节点等待的时长并不完全等于该配置）。之所以这样设计，主要是Leader节点发送的心跳消息可能因为瞬间的网络延迟或程序瞬间的卡顿而迟到（或是丢失），因此就触发新一轮选举是没有必要的。election timeout一般设置为150ms~300ms之间的随机数。另一个超时时间是心跳超时时间（heartbeat timeout），也就是Leader节点向集群中其他Follower节点发送心跳消息的时间间隔。

当集群初始化时，所有节点都处于Follower的状态，此时的集群中没有Leader节点。当Follower节点一段时间（选举计时器超时）内收不到Leader节点的心跳消息，则认为Leader节点出现故障导致其任期（Term）过期，Follower节点会转换成Candidate状态，发起新一轮的选举。所谓“任期（Term）”，实际上就是一个全局的、连续递增的整数，在Raft协议中每进行一次选举，任期（Term）加一，在每个节点中都会记录当前的任期值（currentTerm）。每一个任期都是从一次选举开始的，在选举时，会出现一个或者多个Candidate节点尝试成为Leader节点，如果其中一个Candidate节点赢得选举，则该节点就会切换为Leader状态并成为该任期的Leader节点，直到该任期结束。

回到前面的示例中，此时节点A由于长时间未收到Leader的心跳消息，就会切换成为Candidate状态并发起选举（节点A的选举计时器（election timer）已被重置）。

在选举过程中，节点A首先会将自己的选票投给自己，并向集群中其他节点发送选举请求（Request Vote）以获取其选票，如图2-3（1）所示；此时的节点B和节点C还都是处于Term=0的任期之中，且都是Follower状态，均未投出Term=1任期中的选票，所以节点B和节点C在接收到节点A的选举请求后会将选票投给节点A，另外，节点B、C在收到节点A的选举请求的同时会将选举定时器重置，这是为了防止一个任期中同时出现多个Candidate节点，导致选举失败，如图2-3（2）所示。

注意，节点B和节点C也会递增自身记录的Term值。

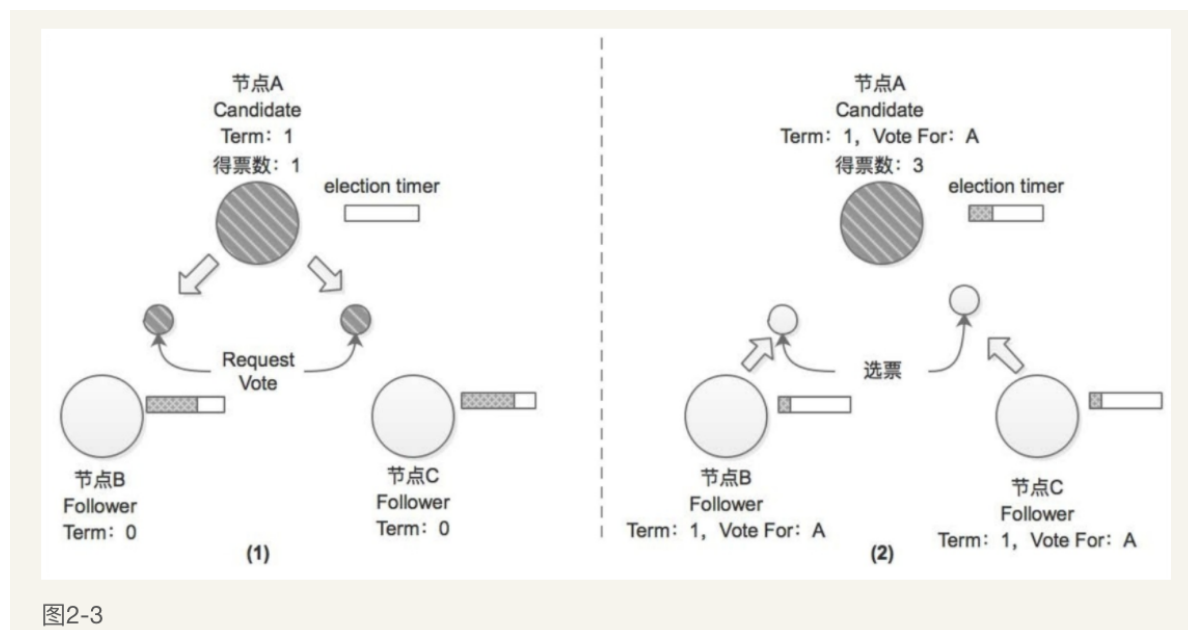


图2-3

在节点 A 收到节点 B、C 的投票之后，其收到了集群中超过半数的选票，所以在 Term=1 这个任期中，该集群的Leader节点就是节点A，其他节点将切换成Follower状态，如图2-4所示。

另外需要读者了解的是，集群中的节点除了记录当期任期号（currentTerm），还会记录在该任期中当前节点的投票结果（VoteFor）。

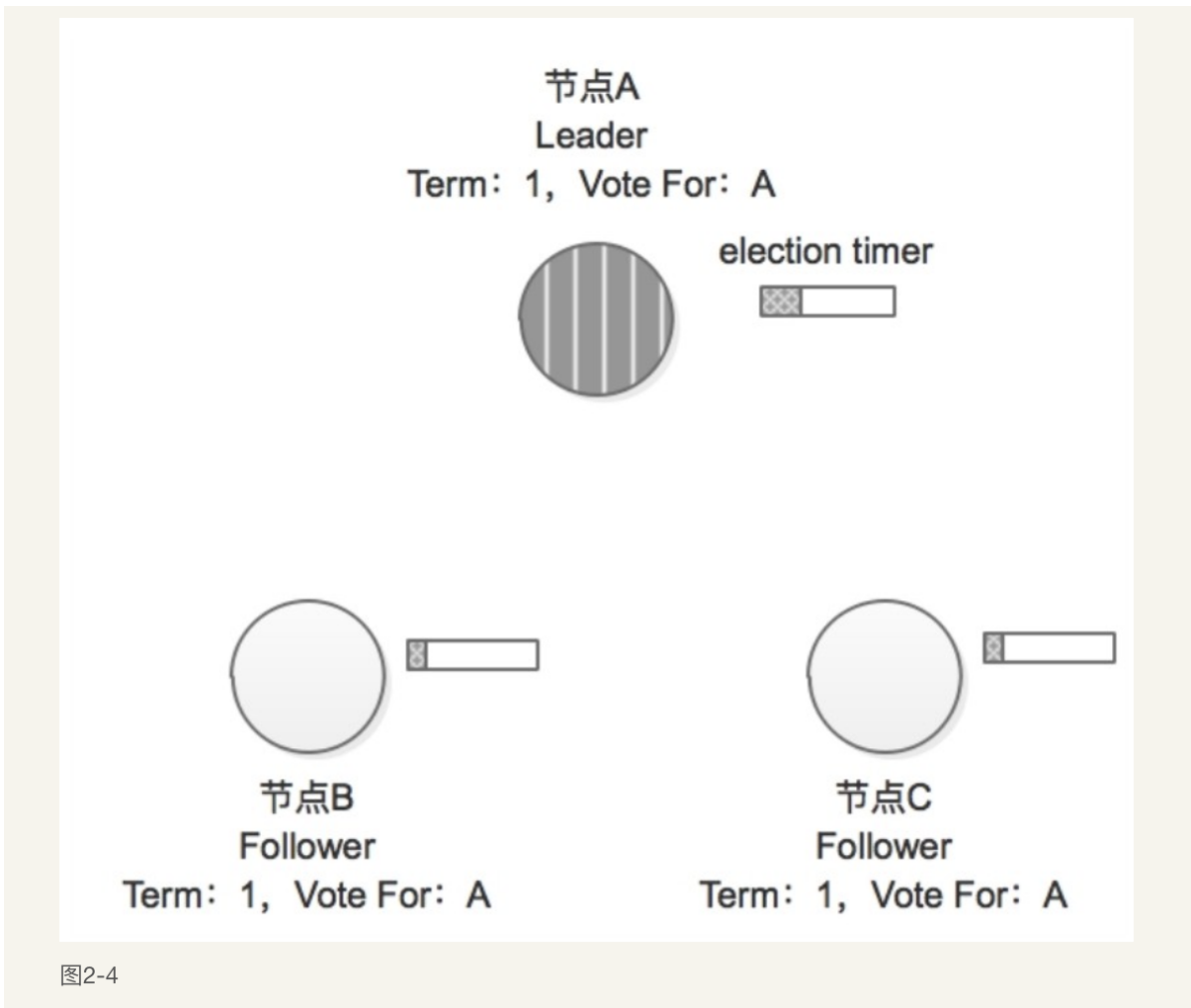


图2-4

继续前面的示例，成为Term=1任期的Leader节点之后，节点A会定期向集群中的其他节点发送心跳消息，如图2-5（1）所示，

这样就可以防止节点B和节点C中的选举计时器（election timer）超时而触发新一轮的选举；当节点B和节点C（Follower）收到节点A的心跳消息之后会重置选举计时器，如图2-5（2）所示，由此可见，心跳超时时间（heartbeat timeout）需要远远小于选举超时时间（election timeout）

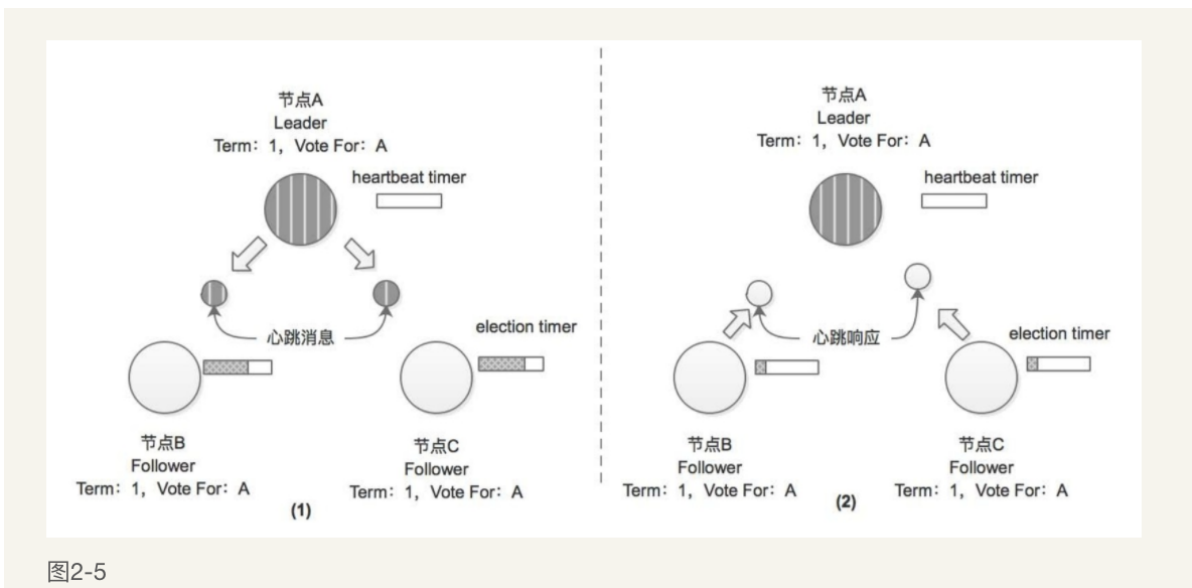
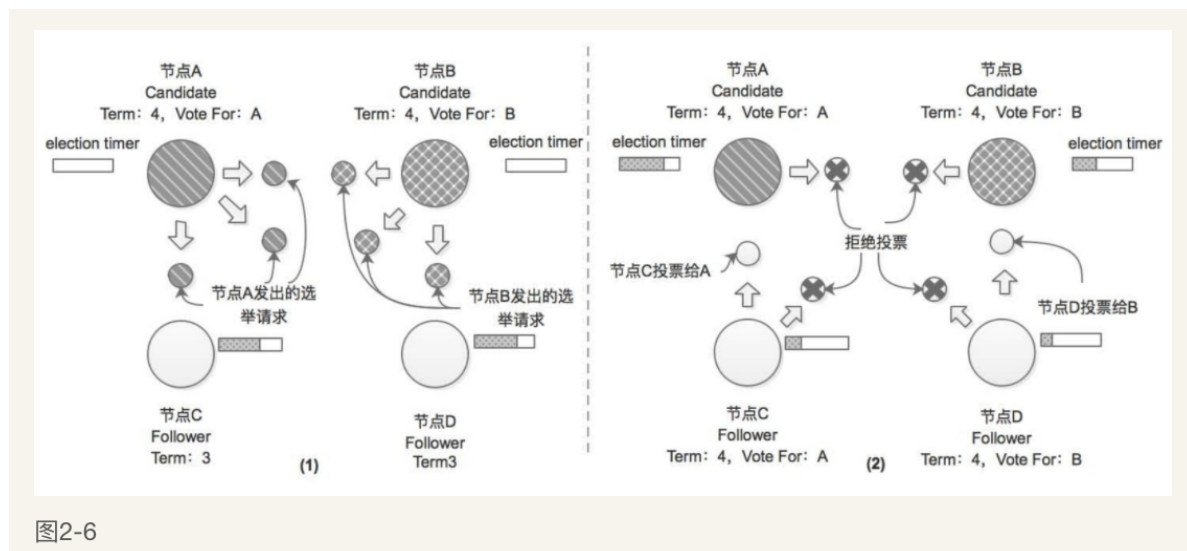


图2-5

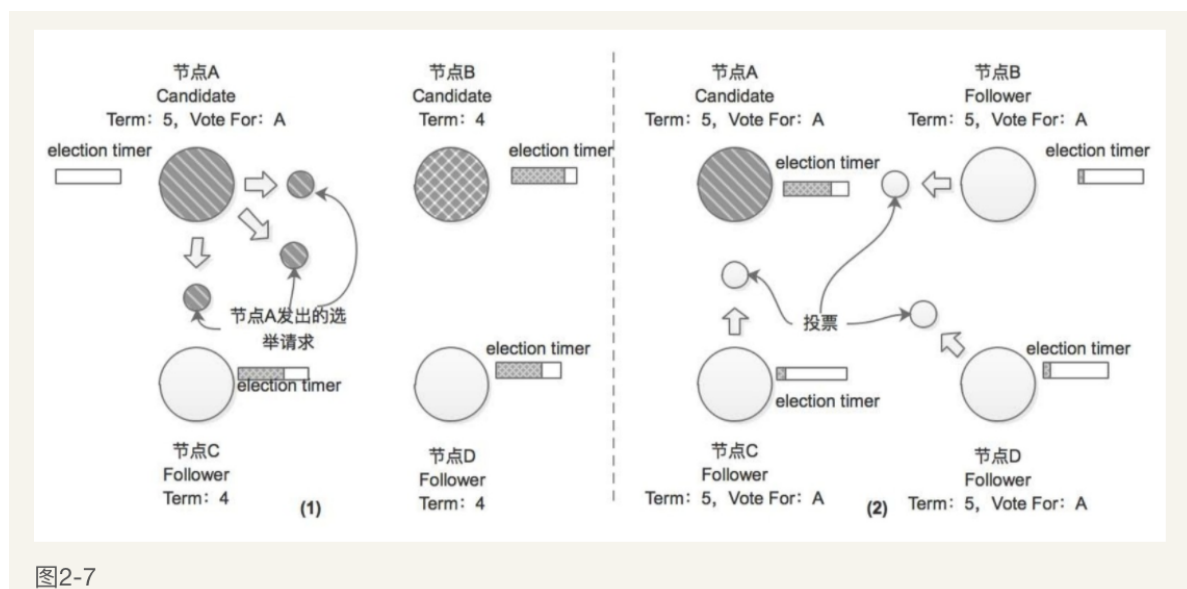
到这里读者可能会问，如果有两个或两个以上节点的选举计时器同时过期，则这些节点会同时由 Follower 状态切换到 Candidate 状态，然后同时触发新一轮选举，在该轮选举中，每个Candidate节点获取的选票都不到半数，无法选举出Leader节点，那么Raft协议会如何处理呢？这种情况确实存在，假设集群中有4个节点，其中节点A和节点B的选举计时器同时到期，切换到Candidate状态并向集群中其他

节点发出选举请求，如图2-6（1）所示。

这里假设节点A发出的选举请求先抵达节点C，节点B发出的选举请求先抵达节点D，如图2-6（2）所示，节点A和节点B除了得到自身的选票之外，还分别得到了节点C和节点D投出的选票，得票数都是2，都没有超过半数。在这种情况下，Term=4这个任期会以选举失败结束，随着时间的流逝，当任意节点的选举计时器到期之后，会再次发起新一轮的选举。前面提到过election timeout是在一个时间区间内取的随机数，所以在配置合理的时候，像上述情况多次出现的概率并不大。



继续上面的示例，这里假设节点A的选举计时器再次到期（此次节点B、C、D的选举计时器并未到期），它会切换到Candidate状态并发起新一轮选举（Term=5），如图2-7（1）所示，其中节点B虽然处于Candidate状态，但是接收到Term值比自身记录的Term值大的请求时，节点会切换到Follower状态并更新自身记录的Term值，所以该示例中的节点B也会将选票投给节点A，如图2-7（2）所示。



在获取集群中半数以上的选票并成为新任期（Term=5）的Leader之后，节点A会定期向集群中其他节点发送心跳消息；当集群中其他节点收到Leader节点的心跳消息的时候，会重置选举定时器，如图2-8所示。

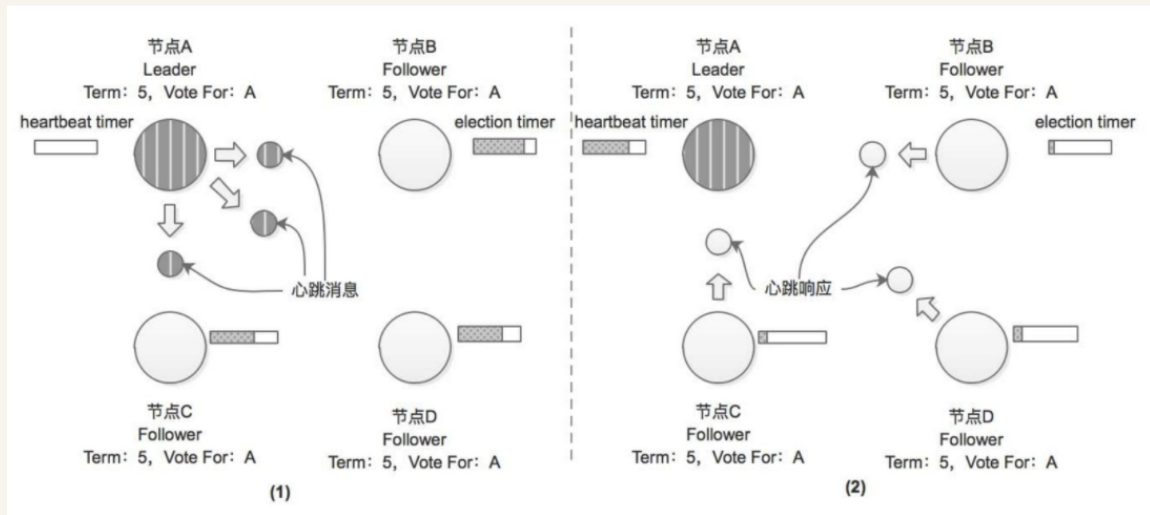


图2-8

CSDN @yznI !

介绍完集群启动时的Leader选举流程之后，下面分析Leader节点宕机之后重新选举的场景。继续上述4节点集群的示例，在系统运行一段时间后，集群当前的Leader节点（A）因为故障而宕机，此时将不再有心跳消息发送到集群的其他Follower节点（节点B、C、D），一段时间后，会有一个Follower节点的选举计时器最先超时，这里假设节点D的选举计时器最先超时，然后它将切换为Candidate状态并发起新一轮选举，如图2-9（1）所示。

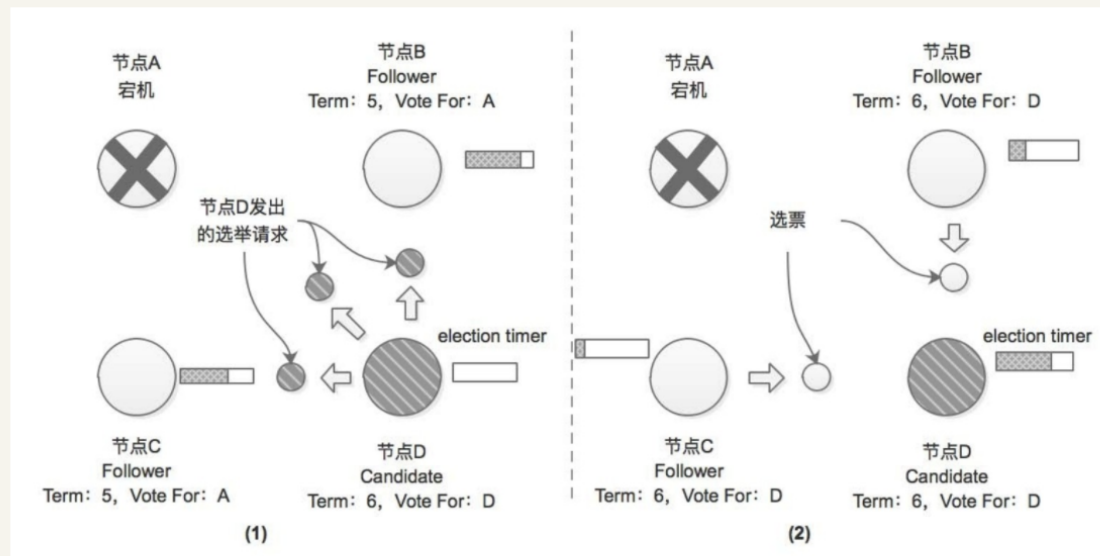


图2-9

CSDN @yznI !

当节点B和节点C收到节点D的选举请求后，会将其选票投给节点D，由于节点A已经宕机，没有参加此次选举，也就无法进行投票，但是在此轮选举中，节点D依然获得了半数以上的选票，故成为新任期（Term=6）的Leader节点，并开始向其他Follower节点发送心跳消息，如图2-10所示。

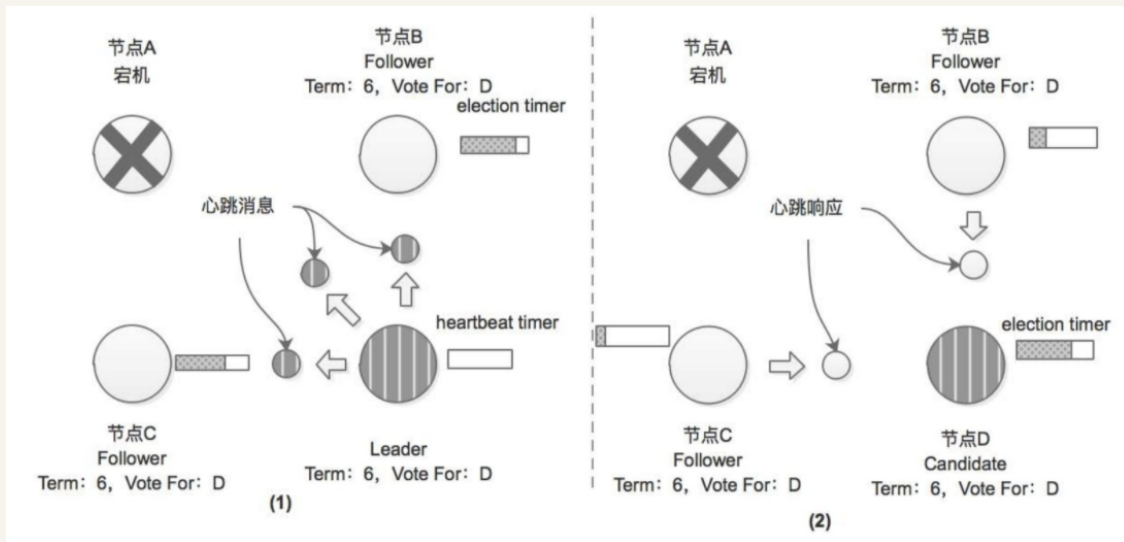


图2-10

CSDN @yznl !

当节点A恢复之后，会收到节点D发来的心跳消息，该消息中携带的任期号（Term=6）大于节点A当前记录的任期号（Term=5），所以节点A会切换成Follower状态。在Raft协议中，当某个节点接收到的消息所携带的任期号大于当前节点本身记录的任期号，那么该节点会更新自身记录的任期号，同时会切换为Follower状态并重置选举计时器，这是Raft算法中所有节点最后请读者考虑一个场景：如果集群中选出的Leader节点频繁崩溃或是其他原因导致选举频繁发生，这会使整个集群中没有一个稳定的Leader节点，这样客户端无法与集群中的Leader节点正常交互，也就导致整个集群无法正常工作。

Leader选举是Raft算法中对时间要求较为严格的一个点，一般要求整个集群中的时间满足如下不等式：
广播时间 << 选举超时时间 << 平均故障间隔时间

在上述不等式中，广播时间指的是从一个节点发送心跳消息到集群中的其他节点并接收响应的平均时间；平均故障间隔时间就是对于一个节点而言，两次故障之间的平均时间。为了保证整个Raft集群可用，广播时间必须比选举超时时间小一个数量级，这样Leader节点才能够发送稳定的心跳消息来重置其他Follower节点的选举计时器，从而防止它们切换成Candidate状态，触发新一轮选举。在前面的描述中也提到过，选举超时时间是一个随机数，通过这种随机的方式，会使得多个Candidate节点瓜分选票的情况明显减少，也就减少了选举耗时。

另外，选举超时时间应该比平均故障间隔时间小几个数量级，这样Leader节点才能稳定存在，整个集群才能稳定运行。当Leader节点崩溃之后，整个集群会有大约相当于选举超时的时间不可用，这种情况占比整个集群稳定运行的时间还是非常小的。

广播时间和平均故障间隔时间是由网络和服务本身决定的，但是选举超时时间是可以由我们自己调节的。

一般情况下，广播时间可以做到0.5ms~50ms，选举超时时间设置为200ms~1s之间，而大多数服务器的平均故障间隔时间都在几个月甚至更长，很容易满足上述不等式的时间需求。

日志复制

通过上一节介绍的Leader选举过程，集群中最终会选举出一个Leader节点，而集群中剩余的其他节点将会成为Follower节点。

Leader节点除了向Follower节点发送心跳消息，**还会处理客户端的请求**，并将客户端的更新操作以消息（Append Entries消息）的形式发送到集群中所有的Follower节点。

当Follower节点记录收到的这些消息之后，会向Leader节点返回相应的响应消息。当Leader节点在收到半数以上的Follower节点的响应消息之后，会对客户端的请求进行应答。

最后，Leader会提交客户端的更新操作，该过程会发送Append Entries消息到Follower节点，通知Follower节点该操作已经提交，同时Leader节点和Follower节点也就可以将该操作应用到自己的状态机中。

上面这段描述仅仅是Raft协议中日志复制部分的大致流程，下面我们依然通过一个示例描述该过程，为了方便描述，我们依然假设当前集群中有三个节点（A、B、C），其中A是Leader节点，B、C是Follower节点，此时有一个客户端发送了一个更新操作到集群，如图2-11（1）所示。前面提到过，集群中只有Leader节点才能处理客户端的更新操作，这里假设客户端直接将请求发给了节点A。当收到客户端的请求时，节点A会将该更新操作记录到本地的Log中，如图2-11（2）所示。

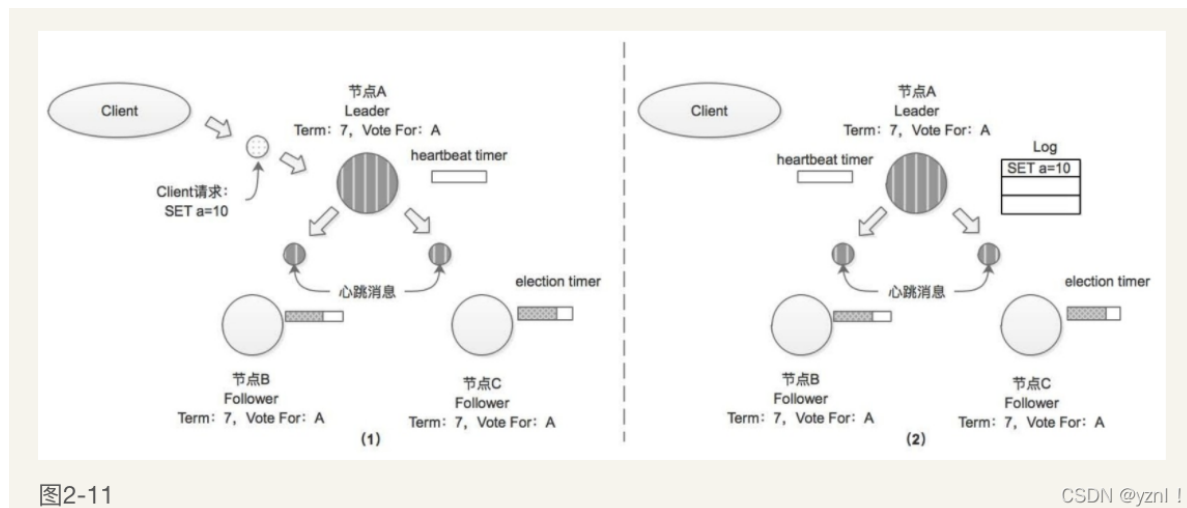


图2-11

CSDN @yznl !

之后，节点A会向其他节点发送Append Entries消息，其中记录了Leader节点最近接收到的请求日志，如图2-12（1）所示。集群中其他Follower节点收到该Append Entries消息之后，会将该操作记录到本地的Log中，并返回相应的响应消息，如图2-12（2）所示。

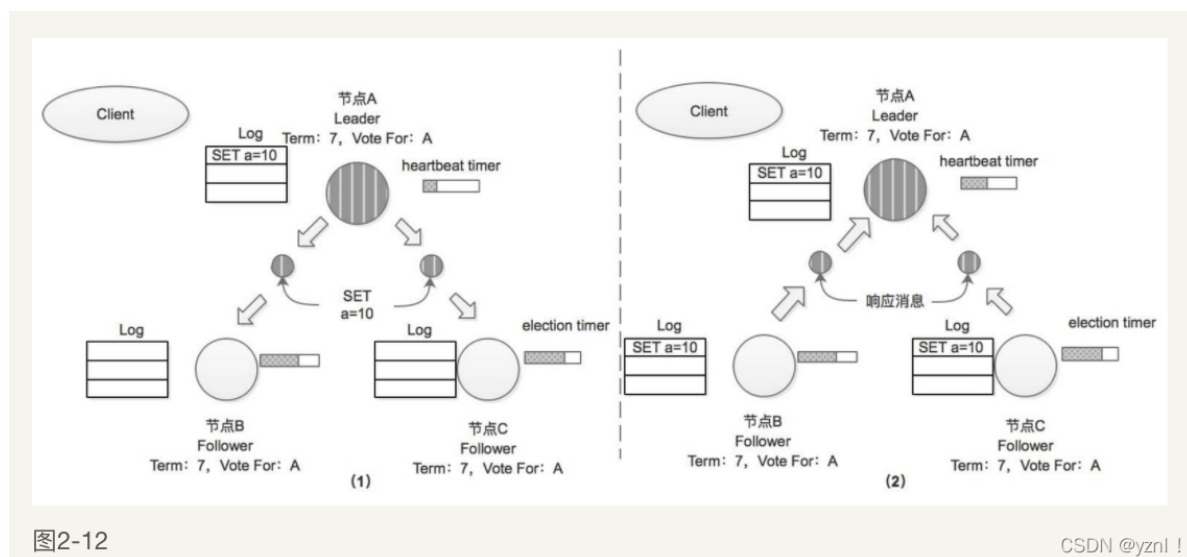


图2-12

CSDN @yznl !

当Leader节点收到半数以上的响应消息之后，会认为集群中有半数以上的节点已经记录了该更新操作，Leader节点会将该更新操作对应的日志记录设置为已提交（committed），并应用到自身的状态机中。同时Leader节点还会对客户端的请求做出响应，如图2-13（1）所示。同时，Leader节点也会向集群中的其他Follower节点发送消息，通知它们该更新操作已经被提交，Follower节点收到该消息之后，才会将该更新操作应用到自己的状态机中，如图2-13（2）所示。

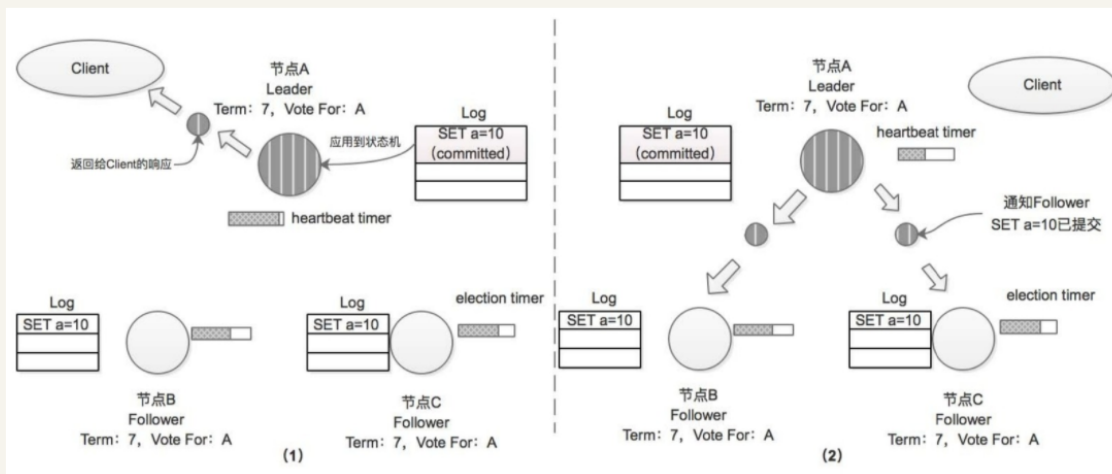


图2-13

CSDN @yznl !

在上述示例的描述中我们可以看到，集群中各个节点都会维护一个本地Log用于记录更新操作，除此之外，每个节点还会维护commitIndex和lastApplied两个值，它们是本地Log的索引值，其中commitIndex表示的是当前节点已知的、最大的、已提交的日志索引值，lastApplied表示的是当前节点最后一条被应用到状态机中的日志索引值。当节点中的 commitIndex 值大于lastApplied值时，会将lastApplied 加1，并将lastApplied对应的日志应用到其状态机中。

在Leader节点中不仅需要知道自己的上述信息，还需要了解集群中其他Follower节点的这些信息，例如，Leader节点需要了解每个Follower节点的日志复制到哪个位置，从而决定下次发送 Append Entries 消息中包含哪些日志记录。为此，Leader 节点会维护 nextIndex[]和matchIndex[]两个数组，这两个数组中记录的都是日志索引值，其中nextIndex[]数组记录了需要发送给每个 Follower 节点的下一条日志的索引值，matchIndex[]表示记录了已经复制给每个Follower节点的最大的日志索引值。

这里简单看一下 Leader 节点与某一个 Follower 节点复制日志时，对应 nextIndex 和matchIndex值的变化：Follower节点中最后一条日志的索引值大于等于该Follower节点对应的nextIndex 值，那么通过 Append Entries 消息发送从 nextIndex 开始的所有日志。之后，Leader节点会检测该 Follower 节点返回的相应响应，如果成功则更新相应该 Follower 节点对应的nextIndex值和matchIndex值；如果因为日志不一致而失败，则减少nextIndex值重试。

下面我们依然通过一个示例来说明nextIndex[]和matchIndex[]在日志复制过程中的作用，假设集群现在有三个节点，其中节点A是Leader节点（Term=1），而Follower节点C因为宕机导致有一段时间未与Leader节点同步日志。此时，节点C的Log中并不包含全部的已提交日志，而只是节点A的Log的子集，节点C故障排除后重新启动，当前集群的状态如图2-14所示（这里只关心Log、nextIndex[]、matchIndex[]，其他的细节省略，另外需要注意的是，图中的Term=1表示的是日志发送时的任期号，而非当前的任期号）。

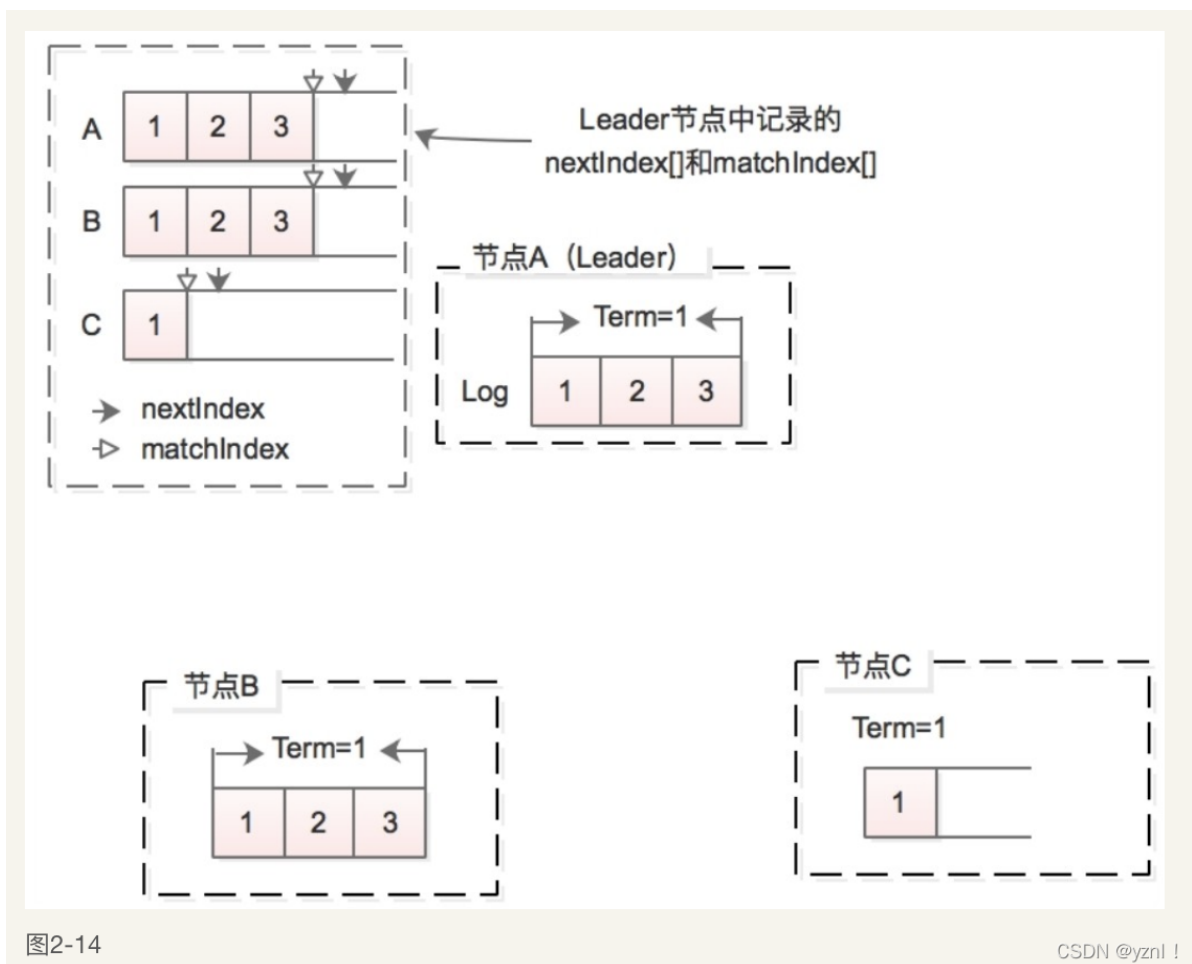


图2-14

CSDN @yznl !

A作为Leader节点，记录了nextIndex[]和matchIndex[]，所以知道应该向节点C发送哪些日志，在本例中，Leader节点在下次发送Append Entries消息时会携带Index=2的消息（这里为了描述简单，每条消息只携带单条日志，Raft协议采用批量发送的方式，这样效率更高），如图2-15（1）所示。当节点C收到Append Entries消息后，会将日志记录到本地Log中，然后向Leader节点返回追加日志成功的响应，当Leader节点收到响应之后，会递增节点C对应的nextIndex和matchIndex，这样Leader节点就知道下次发送日志的位置了，该过程如图2-15（2）所示。

在上例中，当Leader节点并未发生过切换，所以Leader节点始终准确地知道节点C对应nextIndex值和matchIndex值。

如果在上述示例中，在节点C故障恢复后，节点A宕机后重启，并且导致节点B成为新任期（Term=2）的Leader节点，则此时节点B并不知道旧Leader节点中记录的nextIndex[]和matchIndex[]信息，所以新Leader节点会重置nextIndex[]和matchIndex[]，其中会将nextIndex[]全部重置为其自身Log的最后一条已提交日志的Index值，而matchIndex[]全部重置为0，如图2-16所示。

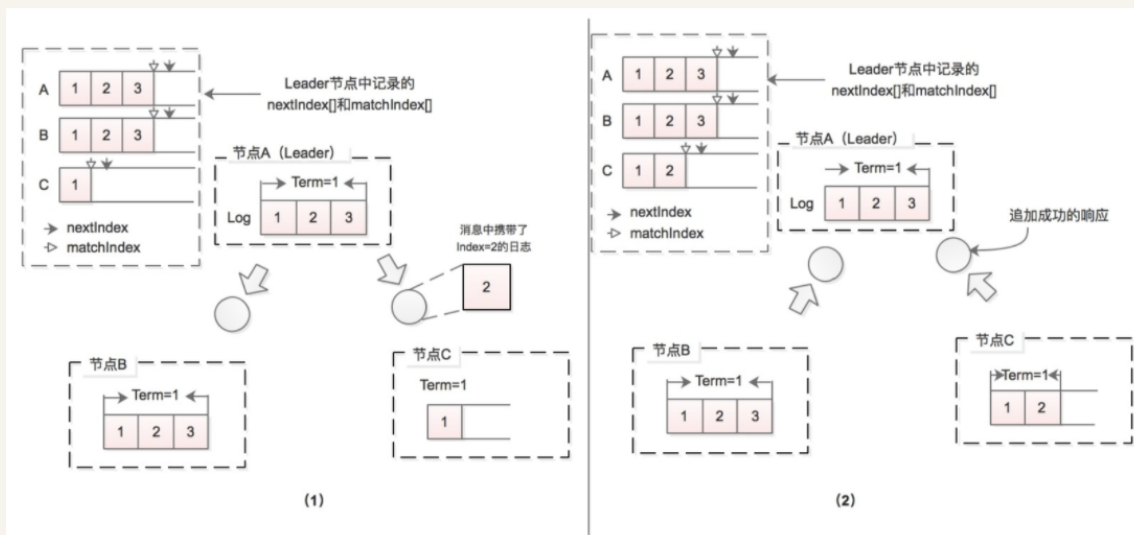


图2-15

CSDN @yznl !

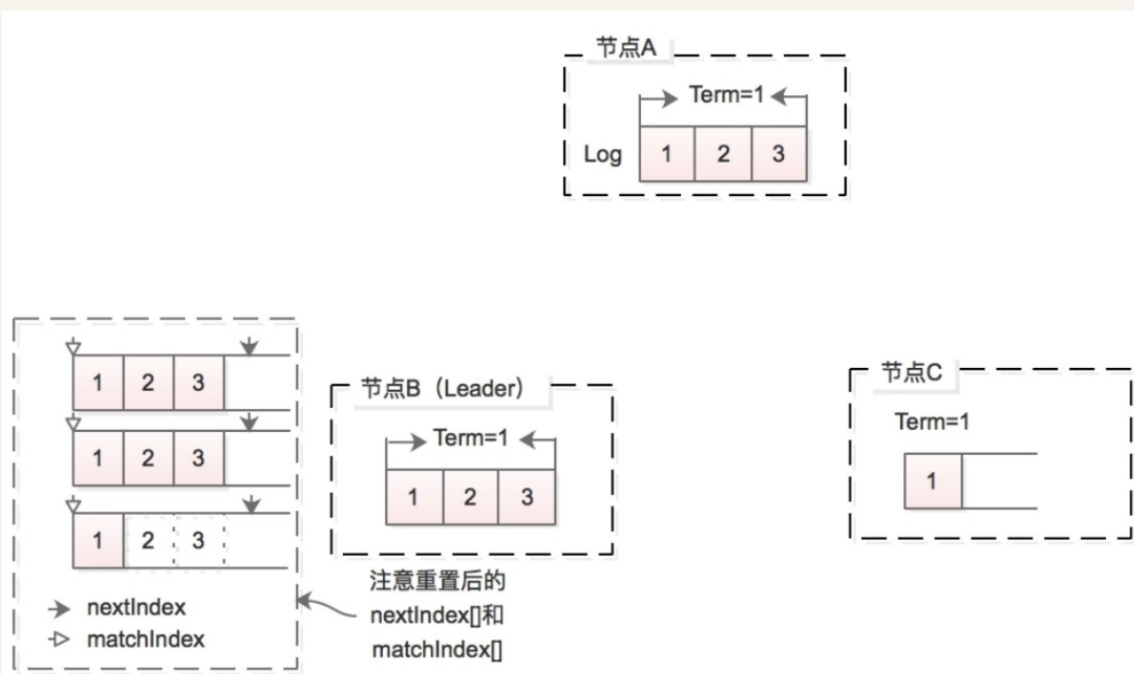


图2-16

CSDN @yznl !

随后，新任期中的Leader节点会向其他节点发送Append Entries消息，如图2-17 (1) 所示，节点A已经拥有了当前Leader的全部日志记录，所以会返回追加成功的响应并等待后续的日志，而节点C并没有Index=2和Index=3两条日志，所以返回追加日志失败的响应，在收到该响应后，Leader节点会将nextIndex前移，如图2-17 (2) 所示。

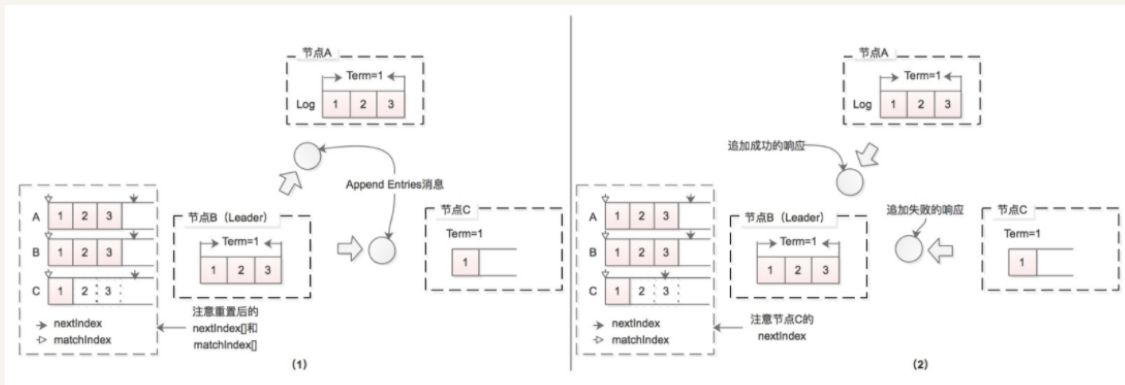


图2-17

CSDN @yzn!

然后新 Leader 节点会再次尝试发送 Append Entries 消息，循环往复，不断减小 nextIndex 值，直至节点 C 返回追加成功的响应，之后就进入了正常追加消息记录的流程，不再赘述。

了解了 Log 日志及节点中基本的数据结构之后，请读者回顾前面描述的选举过程，

其中 Follower 节点的投票过程并不像前面描述的那样简单（先收到哪个 Candidate 节点的选举请求，就将选票投给哪个 Candidate 节点），Follower 节点还需要比较该 Candidate 节点的日志记录与自身的日志记录，拒绝那些日志没有自己新的 Candidate 节点发来的投票请求，确保将选票投给包含了全部已提交（committed）日志记录的 Candidate 节点。

这也就保证了已提交的日志记录不会丢失：Candidate 节点为了成为 Leader 节点，必然会在选举过程中向集群中半数以上的节点发送选举请求，因为已提交的日志记录必须存在集群中半数以上的节点中，这也就意味着每一条已提交的日志记录肯定在这些接收到节点中的至少存在一份。也就是说，记录全部已提交日志的节点和接收到 Candidate 节点的选举请求的节点必然存在交集，如图 2-18 所示。

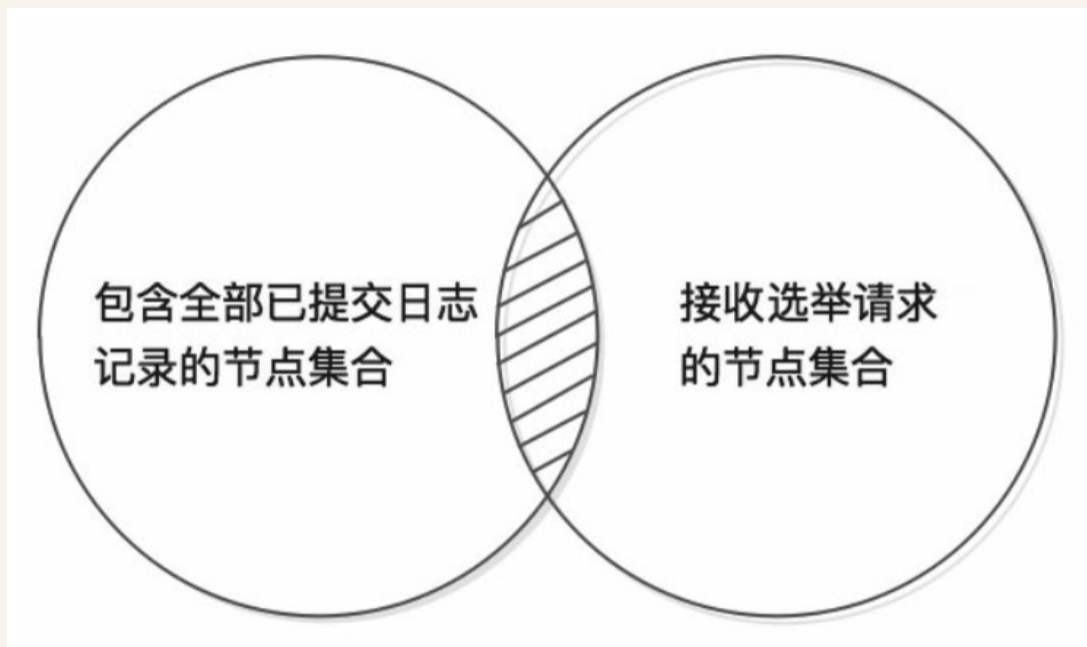


图2-18

CSDN @yzn!

如果 Candidate 节点上的日志记录与集群中大多数节点上的日志记录一样新，那么其日志一定包含所有已经提交的日志记录，也就可以获得这些节点的投票并成为 Leader。

在比较两个节点的日志新旧时，Raft 协议通过比较两节点日志中的最后一条日志记录的索引值和任期号，以决定谁的日志比较新：首先会比较最后一条日志记录的任期号，如果最后的日志记录的任期号不同，那么任期号大的日志记录比较新；如果最后一条日志记录的任期号相同，那么日志索引较大的比较新。

这里只是大概介绍一下 Raft 协议的流程和节点使用的各种数据结构，读者需要了解的是 Raft 协议的工作原理，如果对上述数据结构描述感到困惑，在后面介绍 etcd-raft 模块时，还会再次涉及这些数据结构，到时候读者可以结合代码及这里的描述进一步进行分析。

请详述 kube-proxy 原理？

答：集群中每个 Node 上都会运行一个 kube-proxy 服务进程，他是 Service 的透明代理兼均衡负载器，其核心功能是将某个 Service 的访问转发到后端的多个 Pod 上。

kube-proxy 通过监听集群状态变更，并对本机 iptables 做修改，从而实现网络路由。

而其中的负载均衡，也是通过 iptables 的特性实现的。

从 V1.8 版本开始，用 IPVS (IP Virtual Server) 模式，用于路由规则的配置，主要优势是：

- 1) 为大型集群提供了更好的扩展性和性能。采用哈希表的数据结构，更高效；
- 2) 支持更复杂的负载均衡算法；
- 3) 支持服务器健康检查和连接重试；
- 4) 可以动态修改 ipset 的集合；

聊聊：flannel 和 ovs 网络的区别？

答：

1) 配置是否自动化：OpenvSwitch (ovs) 作为开源的交换机软件，相对比较成熟和稳定，支持各种网络隧道和协议，经历了大型项目 OpenStack 的考验，而 flannel 除了支持建立覆盖网络来实现 Pod 到 Pod 之间的无缝通信之外，还跟 docker、k8s 的架构体系紧密结合，flannel 能感知 k8s 中的 service 对象，然后动态维护自己的路由表，并通过 etcd 来协助 docker 对整个 k8s 集群的 docker0 网段进行规范，而 ovs，这些操作则需要手动完成，假如集群中有 N 个节点，则需要建立 $N(N-1)/2$ 个 Vxlan 或者 gre 连接，这取决于集群的规模，如果集群的规模很大，则必须通过自动化脚本来初始化，避免出错。

2) 是否支持隔离：flannel 虽然很方便实现 Pod 到 Pod 之间的通信，但不能实现多租户隔离，也不能很好地限制 Pod 的网络流量，而 ovs 网络有两种模式：单租户模式和多租户模式，单租户模式直接使用 openvswitch + vxlan 将 k8s 的 pod 网络组成一个大二层，所有的 pod 可以互相通信访问，多租户模式以 Namespace 为维度分配虚拟网络，从而形成一个网络独立用户，一个 Namespace 中的 pod 无法访问其他 Namespace 中的 pod 和 svc 对象；

聊聊：k8s 集群外流量怎么访问 Pod？

答：

可以通过Service的NodePort方式访问，会在所有节点监听同一个端口，比如：30000，访问节点的流量会被重定向到对应的Service上面；

聊聊：K8S 资源限制 QoS?

答：Quality of Service (QoS)

主要有三种类别：

- 1) BestEffort：什么都不设置（CPU or Memory），佛系申请资源；
- 2) Burstable：Pod 中的容器至少一个设置了CPU 或者 Memory 的请求；
- 3) Guaranteed：Pod 中的所有容器必须设置 CPU 和 Memory，并且 request 和 limit 值相等；

聊聊：k8s数据持久化的方式有哪些？

答：

- 1) EmptyDir（空目录）：没有指定要挂载宿主机上的某个目录，直接由Pod内保部映射到宿主机上。类似于docker中的manager volume；场景有：a.只需要临时将数据保存在磁盘上，比如在合并/排序算法中；b.作为两个容器的共享存储，使得第一个内容管理的容器可以将生成的数据存入其中，同时由同一个webserver容器对外提供这些页面；emptyDir的特性：同个pod里面的不同容器，共享同一个持久化目录，当pod节点删除时，volume的数据也会被删除。如果仅仅是容器被销毁，pod还在，则不会影响volume中的数据。总结来说：emptyDir的数据持久化的生命周期和使用的pod一致。一般是作为临时存储使用。
- 2) Hostpath：将宿主机上已存在的目录或文件挂载到容器内部。类似于docker中的bind mount挂载方式；
- 3) PersistentVolume（简称PV）：基于NFS服务的PV，也可以基于GFS的PV。它的作用是统一数据持久化目录，方便管理，PVC是向PV申请应用所需的容量大小，K8s集群中可能会有多个PV，PVC和PV若要关联，其定义的访问模式必须一致。定义的storageClassName也必须一致，若群集中存在相同的（名字、访问模式都一致）两个PV，那么PVC会选择向它所需容量接近的PV去申请，或者随机申请；

聊聊：K8S的基本组成部分？

答：

Master节点主要有五个组件，分别是kubectI、api-server、controller-manager、kube-scheduler 和 etcd；

node节点主要有三个组件，分别是 kubelet、kube-proxy 和 容器运行时 docker 或者 rkt；

kubectI：客户端命令行工具，作为整个系统的操作入口。

apiserver：以REST API服务形式提供接口，作为整个系统的控制入口。

controller-manager：执行整个系统的后台任务，包括节点状态状况、Pod个数、Pods和服务的关联等。

kube-scheduler：负责节点资源管理，接收来自kube-apiserver创建Pods任务，并分配到某个节点。

etcd：负责节点间的服务发现和配置共享。

kube-proxy：运行在每个计算节点上，负责Pod网络代理。定时从etcd获取到service信息来做相应的策

略。

kubelet：运行在每个计算节点上，作为agent，接收分配该节点的Pods任务及管理容器，周期性获取容器状态，反馈给kube-apiserver。

DNS：一个可选的DNS服务，用于为每个Service对象创建DNS记录，这样所有的Pod就可以通过DNS访问服务了。

聊聊：K8s中镜像的下载策略是什么？

答：可通过命令“`kubectl explain pod.spec.containers`”来查看imagePullPolicy这行的解释，

K8s的镜像下载策略有三种：

Always：镜像标签为latest时，总是从指定的仓库中获取镜像；

Never：禁止从仓库中下载镜像，也就是说只能使用本地镜像；

IfNotPresent：仅当本地没有对应镜像时，才从目标仓库中下载；

聊聊：标签与标签选择器的作用是什么？

答：标签：是当相同类型的资源对象越来越多的时候，为了更好的管理，可以按照标签将其分为一个组，为的是提升资源对象的管理效率；标签选择器：就是标签的查询过滤条件。

聊聊：K8s的负载均衡器？

答：负载均衡器是暴露服务的最常见和标准方式之一。

根据工作环境使用两种类型的负载均衡器，即内部负载均衡器或外部负载均衡器。内部负载均衡器自动平衡负载并使用所需配置分配容器，而外部负载均衡器将流量从外部负载引导至后端容器；

聊聊：kubelet 监控 Node 节点资源使用是通过什么组件来实现的？

答：用Metrics Server提供核心指标，包括Node、Pod的CPU和内存的使用。而Metrics Server需要采集node上的cAdvisor提供的资源数据，

当 kubelet 服务启动时，它会自动启动 cAdvisor 服务，然后 cAdvisor 会实时采集所在节点的性能指标及在节点上运行的容器的性能指标。

kubelet 的启动参数 `--cadvisor-port` 可自定义 cAdvisor 对外提供服务的端口号，默认是 4194；

聊聊：Pod的状态？

答：

- 1) Pending：已经创建了Pod，但是其内部还有容器没有创建；
- 2) Running：Pod内部的所有容器都已经创建，只有由一个容器还处于运行状态或者重启状态；
- 3) Succeeded：Pod内所有容器均已经成功执行并且退出，不会再重启；
- 4) Failed：Pod内所有容器都退出，但至少有一个为退出失败状态；
- 5) Unknown：由于某种原因不能获取该Pod的状态，可能是网络问题；

聊聊：deployment/rs的区别？

答：

deployment是rs的超集，提供更多的部署功能，如：回滚、暂停和重启、版本记录、事件和状态查看、滚动升级和替换升级。

如果能使用deployment，则不应再使用rc和rs；

聊聊：rc/rs实现原理？

答：

Replication Controller 可以保证Pod始终处于规定的副本数，

而当前推荐的做法是使用Deployment+ReplicaSet，

ReplicaSet 号称下一代的 Replication Controller，当前唯一区别是RS支持set-based selector，

RC是通过ReplicationManager监控RC和RC内Pod的状态，从而增删Pod，以实现维持特定副本数的功能，RS也是大致相同；

聊聊：kubernetes服务发现？

答：

1) 环境变量：当你创建一个Pod的时候，kubelet会在该Pod中注入集群内所有Service的相关环境变量。**需要注意：**要想一个Pod中注入某个Service的环境变量，则必须Service要先比该Pod创建；

2) DNS：可以通过cluster add-on方式轻松的创建KubeDNS来对集群内的Service进行服务发现；

聊聊：k8s发布(暴露)服务，service的类型有那些？

答：

kubernetes原生的，一个Service的ServiceType决定了其发布服务的方式。

1) ClusterIP：这是k8s默认的ServiceType。通过集群内的ClusterIP在内部发布服务。

2) NodePort：这种方式是常用的，用来对集群外暴露Service，你可以通过访问集群内的每个NodeIP:NodePort的方式，访问到对应Service后端的Endpoint。

3) LoadBalancer：这也是用来对集群外暴露服务的，不同的是这需要Cloud Provider的支持，比如AWS等。

4) ExternalName：这个也是在集群内发布服务用的，需要借助KubeDNS(version >= 1.7)的支持，就是用KubeDNS将该service和ExternalName做一个Map，KubeDNS返回一个CNAME记录；

聊聊：简述ETCD及其特点？

答：etcd是一个分布式的、高可用的、一致的key-value存储数据库，基于Go语言实现，主要用于共享配置和服务发现。特点：

- 1) 完全复制：集群中的每个节点都可以使用完整的存档；
- 2) 高可用性：Etcd可用于避免硬件的单点故障或网络问题；
- 3) 一致性：每次读取都会返回跨多主机的最新写入；
- 4) 简单：包括一个定义良好、面向用户的API（gRPC）；
- 5) 安全：实现了带有可选的客户端证书身份验证的自动化TLS；
- 6) 快速：每秒10000次写入的基准速度；
- 7) 可靠：使用Raft算法实现了强一致、高可用的服务存储目录；

简述ETCD适应的场景？

答：

- 1) 服务发现：服务发现要解决的也是分布式系统中最常见的问题之一，即在同一个分布式集群中的进程或服务，要如何才能找到对方并建立连接。本质上来说，服务发现就是想要了解集群中是否有进程在监听udp或tcp端口，并且通过名字就可以查找和连接。
- 2) 消息发布与订阅：在分布式系统中，最实用对的一种组件间的通信方式：消息发布与订阅。构建一个配置共享中心，数据提供者在这个配置中心发布消息，而消息使用者订阅他们关心的主题，一旦主题有消息发布，就会实时通知订阅者。达成集中式管理与动态更新。应用中用到的一些配置信息放到etcd上进行集中管理。
- 3) 负载均衡：分布式系统中，为了保证服务的高可用以及数据的一致性，通常都会把数据和服务部署多份，以此达到对等服务，即使其中的某一个服务失效了，也不影响使用。etcd本身分布式架构存储的信息访问支持负载均衡。
- 4) 分布式通知与协调：通过注册与异步通知机制，实现分布式环境下不同系统之间的通知与协调，从而对数据变更做到实时处理。
- 5) 分布式锁：因为etcd使用Raft算法保持了数据的强一致性，某次操作存储到集群中的值必然是全局一致的，所以很容易实现分布式锁。锁服务有两种使用方式，一是保持独占，二是控制时序。
- 6) 分布式队列：分布式队列的常规用法与场景五中所描述的分布式锁的控制时序用法类似，即创建一个先进先出的队列，保证顺序。
- 7) 集群监控与Leader精选：通过etcd来进行监控实现起来非常简单并且实时性强；

简述Kubernetes RC的机制？

答：Replication Controller用来管理Pod的副本，保证集群中存在指定数量的Pod副本。当定义了RC并提交至Kubernetes集群中之后，Master节点上的Controller Manager组件获悉，并同时巡检系统中当前存活的目标Pod，并确保目标Pod实例的数量刚好等于此RC的期望值，若存在过多的Pod副本在运行，系统会停止一些Pod，反之则自动创建一些Pod；

简述kube-proxy作用？

答：kube-proxy 运行在所有节点上，它监听 apiserver 中 service 和 endpoint 的变化情况，创建路由规则以提供服务 IP 和负载均衡功能。

简单理解此进程是Service的透明代理兼负载均衡器，其核心功能是将到某个Service的访问请求转发到后端的多个Pod实例上；

简述kube-proxy iptables原理？

答：Kubernetes从1.2版本开始，将iptables作为kube-proxy的默认模式。iptables模式下的kube-proxy不再起到Proxy的作用，其核心功能：通过API Server的Watch接口实时跟踪Service与Endpoint的变更信息，并更新对应的iptables规则，Client的请求流量则通过iptables的NAT机制“直接路由”到目标Pod；

简述kube-proxy ipvs原理？

答：IPVS在Kubernetes1.11中升级为GA稳定版。

IPVS则专门用于高性能负载均衡，并使用更高效的数据结构（Hash表），允许几乎无限的规模扩张，因此被kube-proxy采纳为最新模式；

在IPVS模式下，使用iptables的扩展ipset，而不是直接调用iptables来生成规则链。

iptables规则链是一个线性的数据结构，ipset则引入了带索引的数据结构，因此当规则很多时，也可以很高效地查找和匹配；

可以将ipset简单理解为一个IP（段）的集合，这个集合的内容可以是IP地址、IP网段、端口等，iptables可以直接添加规则对这个“可变的集合”进行操作，这样做的好处在于可以大大减少iptables规则的数量，从而减少性能损耗；

简述kube-proxy ipvs和iptables的异同？

答：iptables与IPVS都是基于Netfilter实现的，但因为定位不同，二者有着本质的差别：

iptables是为防火墙而设计的；IPVS则专门用于高性能负载均衡，并使用更高效的数据结构（Hash表），允许几乎无限的规模扩张。

与iptables相比，IPVS拥有以下明显优势：为大型集群提供了更好的可扩展性和性能；支持比iptables更复杂的复制均衡算法（最小负载、最少连接、加权等）；支持服务器健康检查和连接重试等功能；可以动态修改ipset的集合，即使iptables的规则正在使用这个集合；

简述Kubernetes中什么是静态Pod？

答：静态pod是由kubelet进行管理的仅存在于特定Node的Pod上，他们不能通过API Server进行管理，无法与ReplicationController、Deployment或者DaemonSet进行关联，并且kubelet无法对他们进行健康检查。

静态Pod总是由kubelet进行创建，并且总是在kubelet所在的Node上运行；

简述Kubernetes Pod的常见调度方式？

答：

- 1) Deployment 或RC：该调度策略主要功能就是自动部署一个容器应用的多份副本，以及持续监控副本的数量，在集群内始终维持用户指定的副本数量；
- 2) NodeSelector：定向调度，当需要手动指定将Pod调度到特定Node上，可以通过Node的标签（Label）和Pod的nodeSelector属性相匹配；
- 3) NodeAffinity亲和性调度：亲和性调度机制极大的扩展了Pod的调度能力，目前有两种节点亲和力表达：硬规则，必须满足指定的规则，调度器才可以调度Pod至Node上（类似nodeSelector，语法不同）；软规则，优先调度至满足的Node的节点，但不强求，多个优先级规则还可以设置权重值；
- 4) Taints和Tolerations（污点和容忍）：Taint：使Node拒绝特定Pod运行；Toleration：为Pod的属性，表示Pod能容忍（运行）标注了Taint的Node；

简述Kubernetes初始化容器（init container）？

答：init container的运行方式与应用容器不同，它们必须先于应用容器执行完成，当设置了多个init container时，将按顺序逐个运行，并且只有前一个init container运行成功后才能运行后一个init container。

当所有init container都成功运行后，Kubernetes才会初始化Pod的各种信息，并开始创建和运行应用容器；

聊聊：（支付宝真题）deployment的创建流程？（重点）

Tips:这个问题几乎是每一家公司必须要问的问题，作为k8s的一个基础，从这个问题可以考察出基础能力如何。

（tips：这个要回答出创建rs的过程）

- 1、准备好一个包含应用程序的Deployment的yaml文件，然后通过kubectl客户端工具发送给ApiServer。
- 2、ApiServer接收到客户端的请求，并将资源内容存储到数据库(etcd)中。
- 3、Controller组件(包括scheduler、replication、endpoint)监控资源变化，并作出反应。
- 4、ReplicaSet检查数据库变化，创建期望数量的pod实例。
- 5、Scheduler再次检查数据库变化，发现尚未被分配到具体执行节点(node)的Pod，然后根据一组相关规则将pod分配到可以运行它们的节点上，并更新数据库，记录pod分配情况。
- 6、Kubelete监控数据库变化**，管理后续pod的生命周期，发现被分配到它所在的节点上运行的那些pod。

如果找到新pod，则会在该节点上运行这个新pod。

- 7、kuberproxy运行在集群各个主机上，管理网络通信，如服务发现、负载均衡。

例如当有数据发送到主机时，将其路由到正确的pod或容器。

对于从主机上发出的数据，它可以基于请求地址发现远程服务器，并将数据正确路由，在某些情况下会使用轮训调度算法(Round-robin)将请求发送到集群中的多个实例。

8、kubectl提交一个请求，来创建RC，此时Controller Manager通过API server里的接口监听到这个RC事件，分析之后，发现当前集群中还没有它对应的Pod实例，于是根据RC里的Pod模板定义Pod对象；

接下来，**此事件被Scheduler发现**，它立即执行一个复杂的调度流程，**为这个新Pod选定一个落户的Node**，这个过程可称为绑定；

随后模板Node上运行的Kubelet进程通过API Server监测到这个“新生的”Pod并按照它的定义，启动Pod并负责后期的管理；

随后我们通过Kubectl提交一个映射到该Pod的Server的创建请求，Controller Manager会通过Label标签查询到相关联的Pod实例，然后生成Service的Endpoints信息；

接下来，所有Node上运行的kubeproxy 进程通过API Server查询并监听Service对象，及其对应的Endpoints信息，建立一个负载均衡器来实现Service访问到后端Pod的流量转发功能；

简述Kubernetes deployment升级过程？

答：

初始创建Deployment时，系统创建了一个ReplicaSet，并按用户的需求创建了对应数量的Pod副本；

当更新Deployment时，系统创建了一个新的ReplicaSet，并将其副本数量扩展到1，然后将旧ReplicaSet缩减为2；

之后，系统继续按照相同的更新策略对新旧两个ReplicaSet进行逐个调整；

最后，新的ReplicaSet运行了对应个新版本Pod副本，旧的ReplicaSet副本数量则缩减为0；

简述Kubernetes deployment升级策略？

答：

在Deployment的定义中，可以通过spec.strategy指定Pod更新的策略，

目前支持两种策略：Recreate（重建）和RollingUpdate（滚动更新），

默认值为RollingUpdate；

Recreate：设置spec.strategy.type=Recreate，表示Deployment在更新Pod时，会先杀掉所有正在运行的Pod，然后创建新的Pod；

RollingUpdate：设置spec.strategy.type=RollingUpdate，表示Deployment会以滚动更新的方式来逐个更新Pod。同时，可以通过设置spec.strategy.rollingUpdate下的两个参数（maxUnavailable和maxSurge）来控制滚动更新的过程；

简述Kubernetes DaemonSet类型的资源特性？

答：

DaemonSet资源对象会在每个Kubernetes集群中的节点上运行，并且每个节点只能运行一个pod，这是它和deployment资源对象的最大也是唯一的区别。

因此，在定义yaml文件中，不支持定义replicas。

它的一般使用场景如下：在去做每个节点的日志收集工作。监控每个节点的运行状态。

简述Kubernetes自动扩容机制？

答：

Kubernetes使用Horizontal Pod Autoscaler（HPA）的控制器实现基于CPU使用率进行自动Pod扩缩容的功能。

HPA控制器周期性地监测目标Pod的资源性能指标，并与HPA资源对象中的扩缩容条件进行对比，在满足条件时对Pod副本数量进行调整；

简述Kubernetes Service分发后端的策略？

答：

- 1) RoundRobin：默认为轮询模式，即轮询将请求转发到后端的各个Pod上；
- 2) SessionAffinity：基于客户端IP地址进行会话保持的模式，即第1次将某个客户端发起的请求转发到后端的某个Pod上，之后从相同的客户端发起的请求都将被转发到后端相同的Pod上；

简述Kubernetes Headless Service？

答：在某些应用场景中，若需要人为指定负载均衡器，不使用Service提供的默认负载均衡的功能，或者应用程序希望知道属于同组服务的其他实例。

Kubernetes提供了Headless Service来实现这种功能，即不为Service设置ClusterIP（入口IP地址），仅通过Label Selector将后端的Pod列表返回给调用的客户端；

简述Kubernetes外部如何访问集群内的服务？

答：

映射Pod到物理机：将Pod端口号映射到宿主机，即在Pod中采用hostPort方式，以使客户端应用能够通过物理机访问容器应用；

映射Service到物理机：将Service端口号映射到宿主机，即在Service中采用nodePort方式，以使客户端应用能够通过物理机访问容器应用；

映射Service到LoadBalancer：通过设置LoadBalancer映射到云服务商提供的LoadBalancer地址。这种用法仅用于在公有云服务提供商的云平台上设置Service的场景；

简述Service的四种类型？ 四者之间有何不同？

(1) ClusterIP：

默认类型，自动分配一个仅cluster内部可以访问的虚拟IP

(2) NodePort：

在 ClusterIP 基础上为Service在每台机器上绑定一个端口，这样就可以通过 NodeIP:NodePort 来访问该服务。

如果kube-proxy设置了--nodeport-addresses=10.240.0.0/16 （v1.10 支持），那么仅该NodePort 仅对设置在范围内的IP有效。

(3) LoadBalancer：

在 NodePort 的基础上，借助cloud provider创建一个外部的负载均衡器，并将请求转发到 NodeIP:NodePort

(4) ExternalName：

将服务通过 DNS CNAME记录方式转发到指定的域名（通过 spec.externalName设定）。需要kube-dns版本在1.7以上。

Nodeport、ExternalName和LoadBalancer，都可以进行外部访问。

NodePort

当Service工作在NodePort类型时，默认每个Node会在全部网络接口开启一个端口来转发对集群内服务的请求。

当请求到来时，Node会转发请求到集群中的服务。

这样就带来2个问题：

- 请求的目标Node可能会Down掉或其他别的原因导致网络不能访问，
- NodePort还有一个问题就是对外暴露的端口有限制，默认端口范围是30,000-32,767。这就限制了可以对外暴露服务的个数，使用这些不易记录的端口访问服务也是让人头疼的问题，这就引入了下一个Service类型：LoadBalancer。

LoadBalancer

大多数公有云平台都支持创建这种类型的服务，每个服务可以支持多种协议和多个端口，使用单个IP来访问。因为需要在集群外访问内部服务，所以这个IP地址是公有的，这会产生额外的费用。

如果暴露的服务很多，使用时需要慎重。在私有云环境中，不能创建此类的服务，可以创建NodePort类型的服务然后使用HAproxy来充当Load Balancer，这样和公有云平台的LoadBalancer差别不大。

Service的三种类型：ClusterIP、NodePort、LoadBalancer，后一种是前一种的增强，NodePort类型Service会创建ClusterIP类型Service，LoadBalancer类型Service会创建NodePort和ClusterIP类型Service。

Service组件依赖操作系统中的iptables或ipvs，这是Service的灵魂。

简述Kubernetes ingress?

答：

K8s的Ingress资源对象，用于将不同URL的访问请求转发到后端不同的Service，以实现HTTP层的业务路由机制。

K8s使用了Ingress策略和Ingress Controller，两者结合并实现了一个完整的Ingress负载均衡器。

使用Ingress进行负载分发时，Ingress Controller基于Ingress规则将客户端请求直接转发到Service对应的后端Endpoint（Pod）上，从而跳过kube-proxy的转发功能，kube-proxy不再起作用，

全过程为：ingress controller + ingress 规则 ----> services；

简述Kubernetes镜像的下载策略?

答：

- 1) Always：镜像标签为latest时，总是从指定的仓库中获取镜像；
- 2) Never：禁止从仓库中下载镜像，也就是说只能使用本地镜像；
- 3) IfNotPresent：仅当本地没有对应镜像时，才从目标仓库中下载；默认的镜像下载策略是：当镜像标签是latest时，默认策略是Always；当镜像标签是自定义时（也就是标签不是latest），那么默认策略是IfNotPresent；

简述Kubernetes的负载均衡器?

答：

根据工作环境使用两种类型的负载均衡器，即内部负载均衡器或外部负载均衡器。

内部负载均衡器自动平衡负载并使用所需配置分配容器，而外部负载均衡器将流量从外部负载引导至后端容器；

简述Kubernetes各模块如何与API Server通信?

答：K8s API Server作为集群的核心，负责集群各功能模块之间的通信。

集群内的各个功能模块通过API Server将信息存入etcd，当需要获取和操作这些数据时，则通过API Server提供的REST接口（用GET、LIST或WATCH方法）来实现，从而实现各模块之间的信息交互。

- 1) kubelet进程与API Server的交互：每个Node上的kubelet每隔一个时间周期，就会调用一次API Server的REST接口报告自身状态，API Server在接收到这些信息后，会将节点状态信息更新到etcd中；
- 2) kube-controller-manager进程与API Server的交互：kube-controller-manager中的Node Controller模块通过API Server提供的Watch接口实时监控Node的信息，并做相应处理

； 3) kube-scheduler进程与API Server的交互：Scheduler通过API Server的Watch接口监听到新建Pod副本的信息后，会检索所有符合该Pod要求的Node列表，开始执行Pod调度逻辑，在调度成功后将Pod绑定到目标节点上；

简述Kubernetes Scheduler作用及实现原理？

答：

Scheduler是负责Pod调度的重要功能模块，负责接收Controller Manager创建的新Pod，为其调度至目标Node，调度完成后，目标Node上的kubelet服务进程接管后继工作，负责Pod接下来生命周期；

Scheduler的作用是将待调度的Pod，按照特定的调度算法和调度策略绑定（Binding）到集群中某个合适的Node上，并将绑定信息写入etcd中；

Scheduler通过调度算法调度为待调度Pod列表中的每个Pod从Node列表中选择一个最适合的Node来实现Pod的调度。随后，目标节点上的kubelet通过API Server监听到Kubernetes Scheduler产生的Pod绑定事件，然后获取对应的Pod清单，下载Image镜像并启动容器；

简述Kubernetes Scheduler使用哪两种算法将Pod绑定到worker节点？

答：

1) 预选（Predicates）：输入是所有节点，输出是满足预选条件的节点。kube-scheduler根据预选策略过滤掉不满足策略的Nodes。如果某节点的资源不足或者不满足预选策略的条件则无法通过预选；

2) 优选（Priorities）：输入是预选阶段筛选出的节点，优选会根据优先策略为通过预选的Nodes进行打分排名，选择得分最高的Node。例如，资源越富裕、负载越小的Node可能具有越高的排名；

简述Kubernetes kubelet的作用？

答：

在Kubernetes集群中，在每个Node（又称Worker）上都会启动一个kubelet服务进程。

该进程用于处理Master下发到本节点的任务，管理Pod及Pod中的容器。

每个kubelet进程都会在API Server上注册节点自身的信息，定期向Master汇报节点资源的使用情况，并通过cAdvisor监控容器和节点资源；

简述Kubernetes kubelet监控Worker节点资源是使用什么组件来实现的？

答：

kubelet使用cAdvisor对worker节点资源进行监控。

在Kubernetes系统中，cAdvisor已被默认集成到kubelet组件内，当kubelet服务启动时，它会自动启动cAdvisor服务，然后cAdvisor会实时采集所在节点的性能指标及在节点上运行的容器的性能指标；

简述Kubernetes如何保证集群的安全性？

答：

- 1) 基础设施方面：保证容器与其所在宿主机的隔离；
- 2) 用户权限：划分普通用户和管理员的角色；
- 3) API Server的认证授权：Kubernetes集群中所有资源的访问和变更都是通过Kubernetes API Server来实现的，因此需要建议采用更安全的HTTPS或Token来识别和认证客户端身份（Authentication），以及随后访问权限的授权（Authorization）环节；
- 4) API Server的授权管理：通过授权策略来决定一个API调用是否合法。对合法用户进行授权并且随后在用户访问时进行鉴权，建议采用更安全的RBAC方式来提升集群安全授权；
- 5) AdmissionControl（准入机制）：对kubernetes api的请求过程中，顺序为：先经过认证 & 授权，然后执行准入操作，最后对目标对象进行操作；

简述Kubernetes准入机制？

答：

在对集群进行请求时，每个准入控制代码都按照一定顺序执行。

如果有一个准入控制拒绝了此次请求，那么整个请求的结果将会立即返回，并提示用户相应的error信息，准入控制（AdmissionControl）准入控制本质上为一段准入代码，在对kubernetes api的请求过程中，顺序为：先经过认证 & 授权，然后执行准入操作，最后对目标对象进行操作。

常用组件（控制代码）如下：

AlwaysAdmit：允许所有请求；

AlwaysDeny：禁止所有请求，多用于测试环境；

ServiceAccount：它将serviceAccounts实现了自动化，它会辅助serviceAccount做一些事情，比如如果pod没有serviceAccount属性，它会自动添加一个default，并确保pod的serviceAccount始终存在；

LimitRanger：观察所有的请求，确保没有违反已经定义好的约束条件，这些条件定义在namespace中LimitRange对象中；

NamespaceExists：观察所有的请求，如果请求尝试创建一个不存在的namespace，则这个请求被拒绝；

简述Kubernetes RBAC及其特点（优势）？

答：

RBAC是基于角色的访问控制，是一种基于个人用户的角色来管理对计算机或网络资源的访问的方法，

优势：

- 1) 对集群中的资源和非资源权限均有完整的覆盖；
- 2) 整个RBAC完全由几个API对象完成，同其他API对象一样，可以用kubectl或API进行操作；
- 3) 可以在运行时进行调整，无须重新启动API Server；

简述Kubernetes Secret作用？

答：

Secret对象，主要作用是保管私密数据，比如密码、OAuth Tokens、SSH Keys等信息。

将这些私密信息放在Secret对象中比直接放在Pod或Docker Image中更安全，也更便于使用和分发；

简述Kubernetes Secret有哪些使用方式？

答：

- 1) 在创建Pod时，通过为Pod指定Service Account来自动使用该Secret；
- 2) 通过挂载该Secret到Pod来使用它；
- 3) 在Docker镜像下载时使用，通过指定Pod的spec.ImagePullSecrets来引用它；

简述Kubernetes PodSecurityPolicy机制？

答：

Kubernetes PodSecurityPolicy是为了更精细地控制Pod对资源的使用方式以及提升安全策略。

在开启PodSecurityPolicy准入控制器后，Kubernetes默认不允许创建任何Pod，需要创建PodSecurityPolicy策略和相应的RBAC授权策略（Authorizing Policies），Pod才能创建成功；

简述Kubernetes PodSecurityPolicy机制能实现哪些安全策略？

- 1) 特权模式：privileged是否允许Pod以特权模式运行；
- 2) 宿主机资源：控制Pod对宿主机资源的控制，如hostPID：是否允许Pod共享宿主机的进程空间；
- 3) 用户和组：设置运行容器的用户ID（范围）或组（范围）；
- 4) 提升权限：AllowPrivilegeEscalation：设置容器内的子进程是否可以提升权限，通常在设置非root用户（MustRunAsNonRoot）时进行设置；
- 5) SELinux：进行SELinux的相关配置；

简述Kubernetes网络模型？

答：Kubernetes网络模型中每个Pod都拥有一个独立的IP地址，不管它们是否运行在同一个Node（宿主机）中，都要求它们可以直接通过对方的IP进行访问；

同时为每个Pod都设置一个IP地址的模型使得同一个Pod内的不同容器会共享同一个网络命名空间，也就是同一个Linux网络协议栈。

这就意味着同一个Pod内的容器可以通过localhost来连接对方的端口；在Kubernetes的集群里，IP是以Pod为单位进行分配的。一个Pod内部的所有容器共享一个网络堆栈；

简述Kubernetes CNI模型？

答：

Kubernetes CNI模型是对容器网络进行操作和配置的规范，通过插件的形式对CNI接口进行实现。

CNI仅关注在创建容器时分配网络资源，和在销毁容器时删除网络资源。

容器（Container）：是拥有独立Linux网络命名空间的环境，例如使用Docker或rkt创建的容器。容器需要拥有自己的Linux网络命名空间，这是加入网络的必要条件；

网络（Network）：表示可以互连的一组实体，这些实体拥有各自独立、唯一的IP地址，可以是容器、物理机或者其他网络设备（比如路由器）等；

简述Kubernetes网络策略？

答：

为实现细粒度的容器间网络访问隔离策略，K8s引入Network Policy主要功能是对Pod间的网络通信进行限制和准入控制，设置允许访问或禁止访问的客户端Pod列表。

Network Policy定义网络策略，配合策略控制器（Policy Controller）进行策略的实现；

简述Kubernetes网络策略原理？

答：

Network Policy的工作原理主要为：policy controller需要实现一个API Listener，监听用户设置的Network Policy定义，并将网络访问规则通过各Node的Agent进行实际设置（Agent则需要通过CNI网络插件实现）；

简述Kubernetes中flannel的作用？

答：

- 1) 它能协助Kubernetes，给每一个Node上的Docker容器都分配互相不冲突的IP地址；
- 2) 它能在这些IP地址之间建立一个覆盖网络（Overlay Network），通过这个覆盖网络，将数据包原封不动地传递到目标容器内；

简述Kubernetes Calico网络组件实现原理？

答：

Calico是一个基于BGP的纯三层的网络方案，与OpenStack、Kubernetes、AWS、GCE等云平台都能够良好地集成，Calico在每个计算节点都利用Linux Kernel实现了一个高效的vRouter来负责数据转发。每个vRouter都通过BGP协议把在本节点上运行的容器的路由信息向整个Calico网络广播，并自动设置到达其他节点的路由转发规则；Calico保证所有容器之间的数据流量都是通过IP路由的方式完成互联互通的。

Calico节点组网时可以直接利用数据中心的网络结构（L2或者L3），不需要额外的NAT、隧道或者Overlay Network，没有额外的封包解包，能够节约CPU运算，提高网络效率；

简述Kubernetes共享存储的作用？

答：

Kubernetes对于有状态的容器应用或者对数据需要持久化的应用，因此需要更加可靠的存储来保存应用产生的重要数据，以便容器应用在重建之后仍然可以使用之前的数据。因此需要使用共享存储；

简述Kubernetes PV和PVC？

答：

PV是对底层网络共享存储的抽象，将共享存储定义为一种“资源”；

PVC则是用户对存储资源的一个“申请”；

简述Kubernetes PV生命周期内的阶段？

答：

- 1) Available：可用状态，还未与某个PVC绑定；
- 2) Bound：已与某个PVC绑定；
- 3) Released：绑定的PVC已经删除，资源已释放，但没有被集群回收；
- 4) Failed：自动资源回收失败；

简述Kubernetes CSI模型？

答：

CSI是Kubernetes推出与容器对接的存储接口标准，存储提供方只需要基于标准接口进行存储插件的实现，就能使用Kubernetes的原生存储机制为容器提供存储服务，CSI使得存储提供方的代码能和Kubernetes代码彻底解耦，部署也与Kubernetes核心组件分离；

CSI包括CSI Controller：的主要功能是提供存储服务视角对存储资源和存储卷进行管理和操作；Node的主要功能是对主机（Node）上的Volume进行管理和操作；

简述Kubernetes Worker节点加入集群的过程？

答：在该Node上安装Docker、kubelet和kube-proxy服务；然后配置kubelet和kubeproxy的启动参数，将Master URL指定为当前Kubernetes集群Master的地址，最后启动这些服务；通过kubelet默认的自动注册机制，新的Worker将会自动加入现有的Kubernetes集群中；Kubernetes Master在接受了新Worker的注册之后，会自动将其纳入当前集群的调度范围；

简述Kubernetes Pod如何实现对节点的资源控制？

答：

Kubernetes集群里的节点提供的资源主要是计算资源，计算资源是可计量的能被申请、分配和使用的基础资源。当前Kubernetes集群中的计算资源主要包括CPU、GPU及Memory。

CPU与Memory是被Pod使用的，因此在配置Pod时可以通过参数CPU Request及Memory Request为其中的每个容器指定所需使用的CPU与Memory量，Kubernetes会根据Request的值去查找有足够资源的Node来调度此Pod；

简述Kubernetes Requests和Limits如何影响Pod的调度？

答：

当一个Pod创建成功时，Kubernetes调度器（Scheduler）会为该Pod选择一个节点来执行。对于每种计算资源（CPU和Memory）而言，每个节点都有一个能用于运行Pod的最大容量值。调度器在调度时，首先要确保调度后该节点上所有Pod的CPU和内存的Requests总和，不超过该节点能提供给Pod使用的CPU和Memory的最大容量值；

简述Kubernetes Metric Service？

答：在Kubernetes从1.10版本后采用Metrics Server作为默认的性能数据采集和监控，主要用于提供核心指标（Core Metrics），包括Node、Pod的CPU和内存使用指标。

对其他自定义指标（Custom Metrics）的监控则由Prometheus等组件来完成；

简述Kubernetes中，如何使用EFK实现日志的统一管理？

答：

在Kubernetes集群环境中，通常一个完整的应用或服务涉及组件过多，建议对日志系统进行集中化管理，EFK是 Elasticsearch、Fluentd 和 Kibana 的组合，

Elasticsearch：是一个搜索引擎，负责存储日志并提供查询接口；

Fluentd：负责从 Kubernetes 搜集日志，每个node节点上面的fluentd监控并收集该节点上面的系统日志，并将处理过后的日志信息发送给Elasticsearch；

Kibana：提供了一个 Web GUI，用户可以浏览和搜索存储在 Elasticsearch 中的日志；

简述Kubernetes如何进行优雅的员工关机维护？

答：由于Kubernetes节点运行大量Pod，因此在员工关机维护之前，建议先使用kubectl drain将该节点的Pod进行驱逐，然后进行关机维护；

简述Kubernetes集群联邦？

答：Kubernetes集群联邦可以将多个Kubernetes集群作为一个集群进行管理。因此，可以在一个数据中心/云中创建多个Kubernetes集群，并使用集群联邦在一个地方控制/管理所有集群；

简述Helm及其优势？

答：

Helm 是 Kubernetes 的软件包管理工具，Helm能够将一组K8S资源打包统一管理, 是查找、共享和使用为Kubernetes构建的软件的最佳方式。

Helm中通常每个包称为一个Chart，一个Chart是一个目录，优势：

- 1) 统一管理、配置和更新这些分散的 k8s 的应用资源文件；
- 2) 分发和复用一套应用模板；
- 3) 将应用的一系列资源当做一个软件包管理；
- 4) 对于应用发布者而言，可以通过 Helm 打包应用、管理应用依赖关系、管理应用版本并发布应用到软件仓库；
- 5) 对于使用者而言，使用 Helm 后不需要编写复杂的应用部署文件，可以以简单的方式在 Kubernetes 上查找、安装、升级、回滚、卸载应用程序；

标签与标签选择器的作用是什么？

答：

- 1) 标签可以附加在kubernetes任何资源对象之上的键值型数据，常用于标签选择器的匹配度检查，从而完成资源筛选；
- 2) 标签选择器用于表达标签的查询条件或选择标准，Kubernetes API目前支持两个选择器：基于等值关系（equality-based）的标签选项器以及基于集合关系（set-based）的标签选择器；

什么是Google容器引擎？

答：Google Container Engine（GKE）是Docker容器和集群的开源管理平台。这个基于Kubernetes的引擎仅支持在Google的公共云服务中运行的群集；

image的状态有那些？

答：

- 1) Running：Pod所需的容器已经被成功调度到某个节点，且已经成功运行；
- 2) Pending：APIServer创建了pod资源对象，并且已经存入etcd中，但它尚未被调度完成或者仍然处于仓库中下载镜像的过程；

3) Unknown: APIserver无法正常获取到pod对象的状态, 通常是其无法与所在工作节点的kubelet通信所致;

Service这种资源对象的作用是什么?

答:

service就是将多个POD划分到同一个逻辑组中, 并统一向外提供服务, POD是通过Label Selector加入到指定的service中。

Service相当于是一个负载均衡器, 用户请求会先到达service, 再由service转发到它内部的某个POD上, 通过 `services.spec.type` 字段来指定:

- 1) ClusterIP: 用于集群内部访问。该类型会为service分配一个IP, 集群内部请求先到达service, 再由service转发到其内部的某个POD上;
- 2) NodePort: 用于集群外部访问。该类型会将Service的Port映射到集群的每个Node节点上, 然后在集群之外, 就能通过Node节点上的映射端口访问到这个Service;
- 3) LoadBalancer: 用于集群外部访问。该类型是在所有Node节点前又挂了一个负载均衡器, 作为集群外部访问的统一入口, 外部流量会先到达LoadBalancer, 再由它转发到集群的node节点上, 通过nodePort再转发给对应的service, 最后由service转发到后端Pod中;
- 4) ExternalName: 创建一个DNS别名 (即CNAME) 并指向到某个Service Name上, 也就是为某个Service Name添加一条CNAME记录, 当有请求访问这个CNAME时会自动解析到这个Service Name上;

常用的标签分类有哪些?

答: release (版本): stable (稳定版)、canary (金丝雀版本)、beta (测试版本)、environment (环境变量): dev (开发)、qa (测试)、production (生产)、application (应用): ui、as (application software应用软件)、pc、sc、tier (架构层级): frontend (前端)、backend (后端)、cache (缓存)、partition (分区): customerA (客户A)、customerB (客户B)、track (品控级别): daily (每天)、weekly (每周);

说说你对Job这种资源对象的了解?

答:

Job控制一组Pod容器, 可以通过Job这种资源对象定义并启动一个批处理任务的Job, 其中Job所控制的Pod副本是短暂运行的, 可以将其视为一组Docker容器, 每个Docker容器都仅仅运行一次, 当Job控制的所有Pod的副本都运行结束时, 对应的Job也就结束。

Job生成的副本是不能自动重启的, 对应的Pod副本的RestartPolicy都被设置为Never。

Job所控制的Pod副本的工作模式能够多实例并行计算。

聊聊: k8s是怎么进行服务注册的?

答:

- 1) Service创建的时候会向 API Server 用 POST 方式提交一个新的 Service 定义，这个请求需要经过认证、鉴权以及其它的准入策略检查过程之后才会放行；
- 2) CoreDns 会为Service创建一个dns记录，Service 得到一个 ClusterIP（虚拟 IP 地址），并保存到集群数据仓库；
- 3) 在集群范围内传播 Service 配置；

聊聊：Kubernetes与Docker Swarm的区别如何？

答：

- 1) 安装和部署：k8s安装很复杂;但是一旦安装完毕，集群就非常强大，Docker Swarm安装非常简单;但是集群不是很强大;2)图形用户界面：k8s有，Docker Swarm无；
- 3) 可伸缩性：k8s支持，Docker Swarm比k8s快5倍；
- 4) 自动伸缩：k8s有，Docker Swarm无；
- 5) 负载均衡：k8s在不同的Pods中的不同容器之间平衡负载流量，需要手动干预，Docker Swarm可以自动平衡集群中容器之间的流量；
- 6) 滚动更新回滚：k8s支持，Docker Swarm可以部署滚动更新，但不能自动回滚；
- 7) 数据量：k8s可以共享存储卷。只能与其他集装箱在同一Pod，Docker Swarm可以与任何其他容器共享存储卷；
- 8) 日志记录和监控：k8s内置的日志和监控工具，Docker Swarm要用第三方工具进行日志记录和监控；

聊聊：什么是Container Orchestration？

答：

- 1) 资源编排 - 负责资源的分配，如限制 namespace 的可用资源，scheduler 针对资源的不同调度策略；
- 2) 工作负载编排 - 负责在资源之间共享工作负载，如 Kubernetes 通过不同的 controller 将 Pod 调度到合适的 node 上，并且负责管理它们的生命周期；
- 3) 服务编排 - 负责服务发现和高可用等，如 Kubernetes 中可用通过 Service 来对内暴露服务，通过 Ingress 来对外暴露服务；容器编排常用的控制器有：Deployment 经常被作为无状态实例控制器使用；StatefulSet 是一个有状态实例控制器；DaemonSet 可以指定在选定的 Node 上跑，每个 Node 上会跑一个副本，它有一个特点是它的 Pod 的调度不经过调度器，在 Pod 创建的时候就直接绑定 NodeName；最后一个定时任务，它是一个上级控制器，和 Deployment 有些类似，当一个定时任务触发的时候，它会去创建一个 Job，具体的任务实际上是由 Job 来负责执行的；

聊聊：什么是Heapster？

答：

Heapster 是 K8s 原生的集群监控方案。

Heapster 以 Pod 的形式运行，它会自动发现集群节点、从节点上的 Kubelet 获取监控数据。Kubelet 则是从节点上的 cAdvisor 收集数据；

聊聊：k8s Architecture的不同组件有哪些？

答：

主要有两个组件 – 主节点和工作节点。

主节点具有kube-controller-manager, kube-apiserver, kube-scheduler等组件。

而工作节点具有kubelet和kube-proxy等组件；

聊聊： 能否介绍一下Kubernetes中主节点的工作情况？

答：

主节点是集群控制节点，负责集群管理和控制，包含：

- 1) apiserver: rest接口，资源增删改查入口；
- 2) controller-manager:所有资源对象的控制中心；
- 3) scheduler:负责资源调度，例如pod调度；
- 4) etcd: 保存资源对象数据；

聊聊： kube-apiserver和kube-scheduler的作用是什么？

答：

kube-apiserver: rest接口，增删改查接口，集群内模块通信；

kube-scheduler: 将待调度的pod按照调度算法绑定到合适的pod，并将绑定信息写入etcd；

你能简要介绍一下Kubernetes控制管理器吗？

Kubernetes控制管理器是集群内部的控制中心，负责node,pod,namespace等管理，

控制管理器负责管理各种控制器，每个控制器通过api server监控资源对象状态，将现有状态修正到期望状态；

聊聊： Kubernetes有哪些不同类型的服务？

- ClusterIP、
- NodePort、
- LoadBalancer、
- ExternalName；

聊聊：你对Kubernetes的负载均衡器有什么了解？

答：

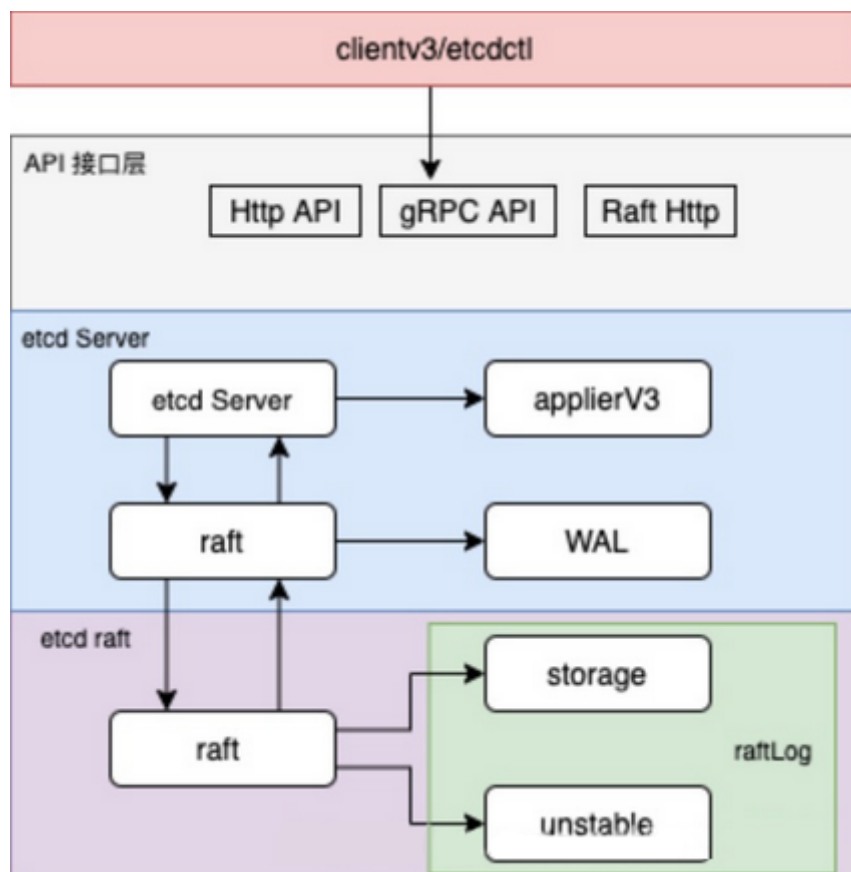
- 1) 内部负载均衡器: 自动平衡负载并使用所需配置分配容器；
- 2) 外部负载均衡器: 将流量从外部负载引导至后端容器；

聊聊：使用Kubernetes时可以采取哪些最佳安全措施？

- 1) 确保容器本身安全；
- 2) 锁定容器的Linux内核；
- 3) 使用基于角色的访问控制（RBAC）；
- 4) 保守秘密的辛勤工作；5) 保持网络安全；

聊聊：（天翼云真题）Etcd的写数据流程是什么？

tips：天翼云问了这个问题，Etcd写数据流程，这个很细节，不准备的话基本就不会了



总体上的请求流程从上至下依次为客户端 → API 接口层 → etcd Server → etcd raft 算法库。)

读请求：客户端通过负载均衡选择一个 etcd 节点发出读请求，API 接口层提供了 Range RPC 方法，etcd 服务端拦截到 gRPC 读请求后，调用相应的处理器处理请求。

写请求：客户端通过负载均衡选择一个 etcd 节点发起写请求，etcd 服务端拦截到 gRPC 写请求，**涉及一些校验和监控，之后 KVServer 向 raft 模块发起提案，内容即为写入数据的命令。经过网络转发，当集群中的多数节点达成一致并持久化数据后，状态变更且 MVCC 模块执行提案内容。**

聊聊：k8s中的pod中OOM机制有了解过吗，原理是什么？

tips：蚂蚁金服的oceanbase数据库产品问了这个问题。这个涉及到pod的QOS的相关知识。阿里面试官首先介绍了系统中进程的OOM机制，会给每个进程进行打分，然后kill掉得分最高的那一个进程，然后问pod的OOM是怎么样的？

答：

当系统 OOM 上时，对于处理不同 OOMScore 的进程表现不同，**OOMScore 是针对 memory 的**，当宿主上 memory 不足时系统会优先 kill 掉**OOMScore 值高的进程**，可以使用如下指令：

```
1 | $ cat /proc/$PID/oom_score
```

查看进程的 OOMScore。OOMScore 的取值范围为 [-1000, 1000]，

Guaranteed pod 的默认值为 -998，

Burstable pod 的值为 2~999，

BestEffort pod 的值为 1000，**也就是说当系统 OOM 时，首先会 kill 掉 BestEffort pod 的进程，若系统依然处于 OOM 状态，然后才会 kill 掉 Burstable pod，最后是 Guaranteed pod；**

参考文献：

https://blog.csdn.net/qg_21222149/article/details/89201744

https://blog.csdn.net/warrior_0319/article/details/80073720

<http://www.sel.zju.edu.cn/?p=840>

<http://alexander.holbreich.org/docker-components-explained/>

<https://www.cnblogs.com/sparkdev/p/9129334.html>

免费领取11个技术圣经PDF

技术自由圈 7个 学习圣经 PDF

- 1 《SpringCloud Alibaba 学习圣经》** 377页 PDF
v1 版，发布日期：2023年03月15日
- 2 《Docker 学习圣经》** 145页 PDF
v2 版，发布日期：2023年03月12日
- 3 《Kubernetes 学习圣经》** 530页 PDF
v1 版，发布日期：2023年04月11日
- 4 《响应式圣经》** 114页 PDF
v2 版，发布日期：2023年02月14日
- 5 《缓存之王 Caffeine 红宝书》** 175页 PDF
v2 版，发布日期：2022年11月30日
- 6 《队列之王 Disruptor 红宝书》** 75页 PDF
v1 版，发布日期：2022年10月05日
- 7 《Prometheus+grafna 红宝书》** 100页 PDF
v1 版，发布日期：2022年10月08日

技术自由圈 3个 高并发圣经 PDF

- 8 《Java高并发核心编程 卷1 加强版：NIO、Netty、Redis、ZooKeeper》** 609页 PDF
加强版，发布日期：2023年01月01日
- 9 《Java高并发核心编程 卷2 加强版：多线程、锁、JMM、JUC、高并发设计模式》** 448页 PDF
加强版，发布日期：2022年11月14日
- 10 《Java高并发核心编程 卷3 加强版：亿级用户Web应用架构与实战》** 481页 PDF
加强版，发布日期：2022年11月14日

技术自由圈 面试圣经 40个 面试题 PDF

- 11 《尼恩 Java 面试宝典》40个专题** 4000页 PDF
v60 版，发布日期：2023年03月11日

领取方式：技术自由圈 公众号



硬核推荐：尼恩Java硬核架构班

详情：<https://www.cnblogs.com/crazymakercircle/p/9904544.html>



尼恩java 硬核架构班



已经发布

- ★ 《高性能RPC的基础实操之：从0到1开始IM撸一个IM》
- ★ 《分布式高性能RPC的基础实操之：千万级用户分布式IM实操- 含简历指导》
- ★ 《亿级用户超高并发秒杀实操- 含简历指导》
亮点：助力小伙伴搞定70W年薪，N个涨薪50%，**2023夏招面试涨薪神器**
- ★ 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- ★ 《第1部曲：超级底层：葵花宝典（高性能秘籍）__架构师视角解读OS操作系统》
亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
2023夏招面试涨薪大神器
- ★ 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
亮点：起底式、绞杀式解读 rocketmq如何保障消息的可靠性？
- ★ 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- ★ 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- ★ 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- ★ 《架构师实操篇：redis cluster 工业级高可用实操》
- ★ 《架构师实操篇：100W级别QPS日志平台实操》
- ★ 《彻底穿透：skywalking 源码(代表链路跟踪)+Java agent+bytebuddy 探针》
- ★ 《超高并发场景100Wqps三级缓存组件原理和实操》
- ★ 《全链路异步超底层原理和实操：手写hystrix熔断+webflux+Lettuce+Dubbo》

规划中



左手大数据 (写入简历, 让简历 蓬荜生辉、金光闪闪)

HBASE + Flink + ElasticSearch 原理、架构、真刀实操



右手云原生 (写入简历, 让简历 蓬荜生辉、金光闪闪)

K8S + Devops + ServiceMesh 原理、架构、真刀实操

架构师实操篇: 基于netty 手写 rpc 框架- 参考 dubbo、seata rpc框架

架构师实操篇: go语言学习, 以及基于 go 手写 rpc 框架

架构师实操篇: 千万级任务调度平台 架构与实操- 基于尼恩17年的亿级搜索项目

架构师实操篇: 工业级 亿级文档搜索 平台 架构与实操- 基于尼恩17年的亿级搜索项目

特色

会员制

提供技术方向指导,
职业生涯指导, 少躺坑, 少弯路

简历指导

这个很重要,
对于挪窝涨薪来说

实操性

以上项目, 都是老架构师
在生产上实操过的项目

非水货

40岁老架构师, 不是水货架构师
《Java高并发三部曲》为证

架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

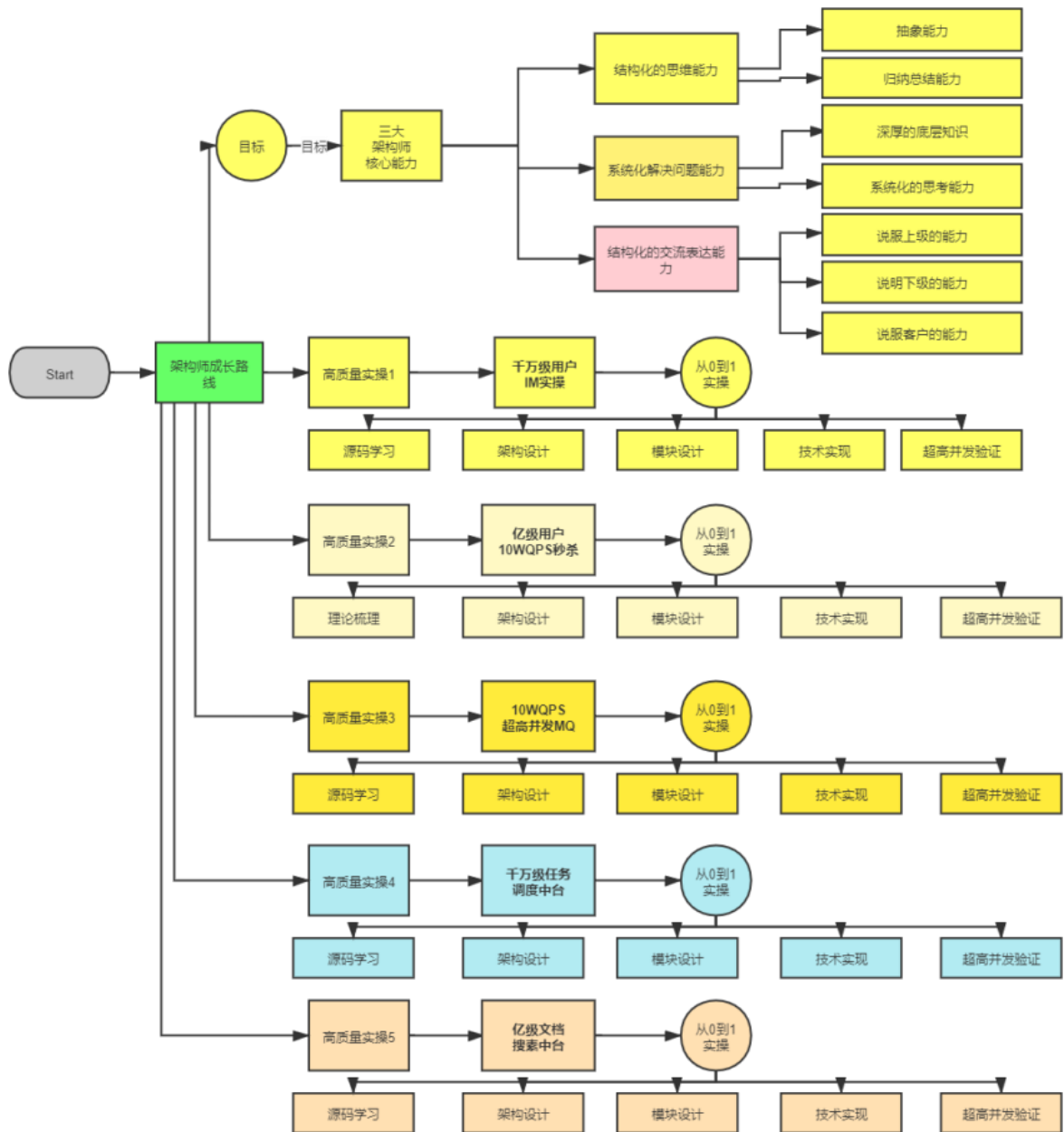
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群 VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷 3 月成高手，狠卷 3 年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩



工业级 rocketmq 高可用底层原理和搭建实操，包含：高可用集群的搭建。

- 1、技术难题：RocketMQ 如何最大限度的保证消息不丢失的呢？RocketMQ 消息如何做到高可靠投递？
- 2、技术难题：基于消息的分布式事务，核心原理不理解
- 3、选型难题：kafka or rocketmq，该娶谁？

成功案例：2 年翻 3 倍，35 岁卷王成功转型为架构师

详情：<http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

最新 最后发表 热门 精华

成功案例：[1057号卷王] 3年小伙拿到外企offer，薪酬涨了200%

卷王1号 超级版主 前天 17:41

成功案例：[645号卷王] 4年经验卷王逆袭，被毕业后，反涨24W

卷王1号 超级版主 2022-9-21

成功案例：[878号卷王] 小伙8年经验，年薪60W

卷王1号 超级版主 2022-8-13

年薪70W案例：通过尼恩的指导，小伙伴年薪从40W涨到70W

卷王1号 超级版主 2022-2-11

成功案例：[493号卷王] 5年小伙拿满意offer，就业寒冬季逆涨30%

卷王1号 超级版主 前天 17:43

成功案例：[250号卷王] 就业极寒时代，收offer 涨25%

卷王1号 超级版主 前天 17:38

成功案例：[612号卷王] 就业极寒时代，从外包到自研

卷王1号 超级版主 前天 17:15

成功案例：[913号卷王] 热烈祝贺6年经验卷王，年薪40W

卷王1号 超级版主 2022-9-21

成功案例：[959号卷王] 4年经验卷王，喜获百度、Boss直聘等N个优质offer，最高涨100%

卷王1号 超级版主 2022-9-21

成功案例：[529号卷王] 5年经验卷王喜收2大offer，最高涨5K

卷王1号 超级版主 2022-9-21

成功案例：[811号卷王] 热烈祝贺7年经验卷王，薪酬涨30%

卷王1号 超级版主 2022-9-21

成功案例：[287号卷王] 不惧大寒潮，卷王逆市收4 offer，涨30%，可喜可贺

卷王1号 超级版主 2022-5-30

成功案例：[1002号卷王] 5月份“被毕业”，改简历后，斩获顶级央企Offer，涨薪7000+

卷王1号 超级版主 2022-7-5

成功案例: [7号卷王] 热烈祝贺小伙伴涨薪120%

1 卷王1号 超级版主 2022-8-13

成功案例: [134号卷王] 大三小伙卷1年, 斩获顶级央企Offer, 成功逆袭

1 卷王1号 超级版主 2022-7-6

成功案例: [1008号卷王] 5年经验卷王收42W offer, 月涨8000, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [453号卷王] 非全日制 6年卷王喜提3 offer, 年薪30W, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [924号卷王] 6年卷王喜提4 offer, 最高涨薪9000, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [15号卷王] 4年卷王入职 微软, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-12

成功案例: [527号卷王] 4年卷王喜提2 offer, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-13

成功案例: [788号卷王] 3年卷王喜提优质Offer, 涨薪60%

1 卷王1号 超级版主 2022-5-11

成功案例: 热烈祝贺: 非全日制卷王, 喜提2个心仪offer, 面3家过2家

1 卷王1号 超级版主 2022-4-21

成功案例: [693号卷王] 二线城市6年卷王喜提4大优质Offer, 含央企offer, 最高薪酬35W

1 卷王1号 超级版主 2022-4-16

成功案例: [85号卷王] 双非2本小伙, 春招大捷, 喜提9个offer, 最高薪酬近30万

1 卷王1号 超级版主 2022-4-14

成功案例: [741号卷王] 卷王逆袭! 6年小伙从很少面试机会到搞定35K*14薪Offer

1 卷王1号 超级版主 2022-4-12

成功案例: [642号卷王] 热烈祝贺, 6年卷王喜提优质国企offer

1 卷王1号 超级版主 2022-4-7

成功案例: [796号卷王] 热烈祝贺, 36岁卷王喜提52万优质offer

1 卷王1号 超级版主 2022-3-25

❑ 成功案例: [15号卷王] 小伙卷1年, 涨薪9K+, 喜收ebay等多个优质offer

① 卷王1号 超级版主 2022-3-24

❑ 成功案例: [821号卷王] 小伙狠卷3个月, 喜提10多个offer

① 卷王1号 超级版主 2022-3-21

❑ 成功案例: [736号卷王] 3年半经验收22k offer, 但是小伙志存高远, 冲击25k+

① 卷王1号 超级版主 2022-3-20

❑ 成功案例: 热烈祝贺1群小卷王offer拿到手软, 甚至拒了阿里offer

① 卷王1号 超级版主 2022-3-16

❑ 简历案例: 简历一改, 腾讯的邀请就来了! 热烈祝贺, 小伙收到一大堆面试邀请

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 祝贺我圈两大超级卷王, 一个过了阿里HR面, 一个过了阿里2面

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 小伙伴php转Java, 卷1.5年Java, 涨薪50%, 喜收多个优质offer

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 4年小伙狠卷半年, 拿到 移动、京东 两大顶级offer

 尼恩 超级版主 2022-3-5

❑ 成功案例: [267号卷王] 助力3年经验卷王, 拿到蜂巢的17k x 14薪的offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [143号卷王] 二本院校00后卷神, 毕业没到一年跳到字节, 年薪45W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [76号卷王] 2线城市卷王, 狠卷1.5年, 喜收22K offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [429号卷王] 小伙伴在社群卷5个月, 涨8k+

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [154号卷王] 双非学校毕业卷王, 连拿 京东到家&滴滴 两个大厂Offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [232号卷王] 涨薪10K, 继续卷向食物链顶端

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: 狠卷1年技术, 喜收 腾讯、阿里、微软三大Offer, 最高年薪56W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [449号卷王] 应届毕业卷王喜收 滴滴offer, 年薪33W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [551号卷王] 小伙伴学完后, 成功进入大厂, 并且推荐自己的朋友加VIP学习

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [214号卷王] 助力2年经验卷王, 成功拿到17K月薪

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: 社群卷王小伙伴成功过了滴滴三面 获滴滴Offer

① 卷王1号 超级版主 2022-2-10

❑ [612号卷王]滴滴小伙伴, 蹲点考察半年, 觉得靠谱后加入 疯狂创客圈

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [732号卷王] 尼恩助力3年经验卷王收获 京东offer, 年薪35W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [558号卷王] 2年经验卷王, 喜收 网易和阿里子公司两个优质offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [569号卷王] 双非应届生卷王, 喜收字节跳动实习offer

① 卷王1号 超级版主 2022-2-25

❑ 成功案例: [420号卷王] 狠卷1年, 卷王涨薪80%, 涨薪12000元!

① 卷王1号 超级版主 2022-2-25

❑ 成功案例: [76号卷王] 通过尼恩1年半的指导, 专科学历小伙伴从0.8K涨到22K

① 卷王1号 超级版主 2022-2-10

简历优化后的成功涨薪案例（VIP含免费简历优化）

6年专科，2年翻4倍

2年从8K涨到 35K

2021年从8K涨到22K

高并发 VIP76

老师，求助。

现在有两个满意的 offer，不知道怎么抉择。

一个是吉利，17k，大数据与 ai 部门。

另一个是一个平台，从零开始用 java 重写现在的项目，分布式架构，带团队，自己招人。22k，我觉得我说少了，我自己提的，然后今天发了 offer

呵呵，你太牛了

我也不好说

工资高的是个小公司，不到 50 人

感觉好事都被你占了

这一年半，真的谢谢您。

呵呵，相互交流，相互成长。

您写的书本，解决了我项目上很多问题。您在群里不厌其烦地告诉我们学习，也是我能坚持下来的重要因素，还有每次提问您都能解答疑惑，让我始终能戒骄戒躁。恩师，

**秘诀：
简历指导+ 狠狠卷**

2022年涨到35K

VIP76

解决了，限制 ip 频率。

谢谢老师

中午12:43

调整到了 35,加上这个月加班费，38

中午12:43

老师，我隔壁来了。

晚上8:23

大大的赞

老师你这路子是对的。我就跟着你学习思路和方法，还有教程走的。

我和你一样的兴奋和喜悦

记得咱们去年改简历的时候，还是 10k

这种提升，已经太令人震撼啦

是 8K...

20 年 4 月份转行，就一路跟着你学习

9年 小伙伴拿到 年薪90W offer

9月11日改简历 11月29日晒offer

秘诀:
简历指导+ 狠狠卷

薪资高 稳

这个是给你微调的

省略号的地方, 需要你再补充一点

9月11日 下午17:38

上面你留着这个就行

Java 开发 - 9 年-修改.docx 34.1 KB

主要的工作是啥?

提升了自己的实力, 就不用怕

易所 关于数字货币的

po 也有 打盹的时候, 该裁员, 照样一个不少

年包比 po 多 19w

这么多

po 估计有 70 万

那你不是有 90 个 W?

po 给我 68

就是吗

小伙8年经验 年薪60w

7月12日改简历 8月10日晒offer

秘诀:
改简历+ 狠狠卷

涨幅多少呀?

或者, 幅度多少呀?

之前 36*15, 现在这个 39*15

今年行情不太好, 还有一些 offer 基本都是平薪, 或降薪的。

OK

这个马上来

刚在指导简历

哈哈

等等哈

辛苦啦

这次找工作, 您的指导真的起到效果了。

我这次复习基本看的都是咱们课程的

6年小伙伴 年薪40w

9月6日改简历 9月21日晒offer

秘诀:
简历指导+ 狠狠卷

张这多

今年这行情, 也算可以了

总包多少呀, 让我也了解一下

大概 40w 吧

谢谢恩师的指导和鼓励

是在深圳

深圳的行情尤其难

能有面试电话就不错啦

是的

太牛啦

哈哈哈哈哈, 恩师的鼓励指导也很重要

给你前面调整了一下

Java - 6年.docx 24.7 KB

简历电话

恩师, 简历我改好了, 您再帮我看看一下

9月6日 下午14:47

Java - 6年(1).docx 24.5 KB

简历电话

恩师, 看第二个吧

9月6日 晚上19:41

5年小伙喜提3个offer 年薪 35个W

5月22日改简历 11月29日晒offer

恩恩老师晚上好, 汇报下最近的 offer 情况。最近面试收到了三个 offer。两个是平薪, 一个是跳槽电商公司的 offer, 涨幅暂时不到 20%。通过这次面试也让我知道自己距离高级开发还有一点距离, 还要再多卷才能突破。

最后结合自己的情况, 先选择去跨境电商的公司再提升下

之前是 20k*13.5, 跨境电商这个说是 23k* (14-15)

恭喜恭喜

毕业两年, 能拿到 3 个 offer, 已经很不错了。

很多小伙伴, 面试电话一个都接不到, 简历海投 7000 份, 只收到 3 次面试机会, 没有一个机会拿到最终 offer

您这个年薪, 算下来也有 35W 了吧

在待岗部分还要确认下, 大部分人说只有两个月

这个时间点, 拿到这个水平, 挺不错的啦

持续加油卷哈

嘿嘿! 看看明年自己有没有能力冲击离开

把你视频都吃透

辛苦老师再帮忙指导下哈

秘诀:
简历指导+ 狠狠卷

算法方面, 可以找点英语题, 刷刷, 当作复习

项目的话, 再好好改造一下

好的 谢谢老师

恩恩老师晚上好, 结合老师的视频, 我对自己的简历改了多次。看到大家经过老师的修改, 有醍醐灌顶的感觉, 我也想请老师也帮忙看看给一些建议~ 希望在当前环境有 offer 收获~

看量级 java 开发-本科统招-2022-5年经验.docx 35.9 KB

简历电话

OK

我排好号, 回头找您哈

1.5年小伙搞定15K offer

就业寒冬涨100%

5月7日改简历 11月21日晒offer

秘诀:
简历指导+ 狠狠卷

他那个不止那啥... 我才是两...
原先7k, 现在15k

那也很牛...
都翻倍了, 都是牛人...
正常就30%...
慢点来...
下一步可以瞄准25k...
其实我工作一年...
那你就更牛逼拉...
一年经验, 搞定15k offer, 舍你其谁...

卷王逆袭成功案例

6年小伙从很少面试机会到搞定35K*14薪

3月5日改简历 4月11日拿offer

一个月拿到了理想的offer

面试邀请少 小伙很苦恼

面试法宝 rocketmq四部曲

理想很丰满 目标35K

6年 经验小伙伴喜收25K offer

3月12日改简历 12月1日晒offer

秘诀:
简历指导+ 狠狠卷

咱们以这个项目为模板改哈

项目有多少人, 你带领多少人?

你工作几年了?

6年了

改简历脚窝, 20 涨到 25 k, 高级开发, P7 帮的后端团队

任务重, 道路远, 并不是没有方向

7年经验卷王 薪酬涨30%

7月11日改简历 9月1日晒offer

秘诀:
改简历+ 狠狠卷

只要本事好, 拿 offer 比较容易的

入职了, 最终并没那么涨薪, 还是旧的太... 只涨了30%

4年经验卷王逆袭 被毕业后，反涨24W

7月改简历

8月30日晒offer



小伙5月份"被毕业"，改简历后 斩获顶级央企Offer 涨薪7000+

5月29日改简历

7月5日晒offer



卷王逆袭成功案例 武汉6年喜收4个优质offer 最高的年薪35W

2月9日改简历

4月15日晒offer

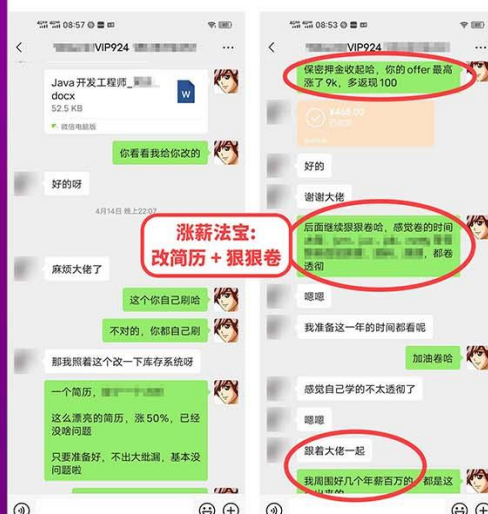


卷王逆袭成功案例

6年小伙喜提4个Offer 最高涨9k，年薪35W

4月14日改简历

5月17日晒offer



卷王逆袭成功案例

5年经验小伙收2个offer 最高涨薪8k，年薪42W

5月9日改简历

5月30日晒offer

秘诀:
简历指导+ 狠卷3高

以此为样
大家狠狠卷
打造最卷IT社群

卷王逆袭成功案例

非全日制 6年经验卷王 喜提3个Offer，年包30W

5月9日改简历

5月18日晒offer

面试法宝:
改简历+ 狠狠卷

卷王逆袭成功案例

寒五冻六之际卷王大逆袭 收3大offer，涨30%

5月17日改简历

5月27日晒offer

秘诀:
简历指导+ 狠卷3高

卷王逆袭成功案例

4年卷王入职微软，涨50%

3月7日改简历

5月12日晒offer

涨薪法宝:
改简历+ 狠狠卷

4年小伙喜收百度、Boss直聘等N个顶级Offer 最高涨幅100%

6月27日改简历 9月19日晒offer

**秘诀：
改简历 + 狠狠卷**

有个offer选择问题

boss直聘和小满之间，boss那边给的offer，工资比小满高，但是小满那边给的offer，福利比boss高，不知道该怎么选，如果你应该怎么选呀。

还有其他很多offer的，还有一个小满的，总体上看boss和小满之间选一个

了

boss比小满年薪包多7w以上，我这涨幅都接近百分之百了

太牛啦

按照你那个思路结合上次指导的内容改了几个点，发给你确实大上了很多，加上一些指标的说明确实会让人眼前一亮的感觉。

明天难时候有没空和我看下哈，一是看两个点我实在不知道该怎么改了，二是我在推荐表达这方面确实不行，还得让你帮忙把把关。

卷王逆袭成功案例 4年卷王入收2个offer，涨50%

3月23日改简历 5月12日晒offer

尼恩老师，我的简历你看了吗，有什么建议？

ok

咱们开始吧

工作经历 这里不需要写描述

下边请描述，每段经历写什么

这个不用加粗

都是一样，加粗了反而没有效果

**涨薪法宝：
改简历 + 狠狠卷**

offer决策圈

n+1 电商erp，部门成熟，人数多，加班少。

n+2 做业务中台 内部系统，新部门，人数少，加班多。

又搞到个offer

能搞定两个offer，不尴尬

现在很多小伙伴面试机会都没有

涨了多少钱

地点在哪里

涨50%的样子

忘记你工作几年啦，大概工作几年啦

小伙大三暑期很焦虑 跟着尼恩卷一年 校招斩获顶级央企Offer

去年5月19日加入VIP群 今年7月5日晒offer

**秘诀：
狠狠卷书+视频**

邀请你加入群聊

尼恩老师

我校招去华润电力控股有限公司了

跟着你卷了一年 大学顺便拿了几个国奖

不错不错，这是央企

放空中心

这太牛啦

其实拿到手的也就一个a类国一

尼恩大佬 大三的暑期实习找不到，现在准备秋招应该没关系吧

看身边总是很焦虑，自己算法这一块卡住

网盘里边有算法视频

去刷一刷吧

秋招来得及

谢谢大佬 不过经过几个月练习，看您写的书比之前轻松多了

趁着还是学生这段时间，慢慢把知识吃透

嗯嗯，我的书，比较深

期待大佬的下一本书，已经迫不及待去学习了

小伙高中学历 薪酬涨120%

5月6日改简历 7月22日晒offer

你这块估计要送你668

模块的开发工作，就这个三个哈

哈哈，不用了老师，您真的讲了就行，光今天晚上辅导就值几千了

老弟很感谢您

还有很多其他的模块

面试官提问

就是其他人做的

嗯嗯，好

**秘诀：
改简历 + 狠狠卷**

之前你的工资是多少来的

翻了一翻

我得送你多少奖金来的

不知道，原价给老弟就行了

哈哈，0.00

咱们得说这话呀

老弟拿着您的那个高开发改造亮点，所向披靡。

ok

从从到尾给面试官讲明白

后面继续狠狠卷哈

卷王逆袭成功案例

非全日制卷王 面试3家
收2个offer 涨薪30%

4月13日改简历

4月21日晒offer

5年卷王喜收2大Offer

最高涨5K

5月19日改简历

9月13日晒offer

面试法宝:
改简历 + 面试题

卷王逆袭成功案例

3年经验卷王，涨60%

4月16日改简历

5月11日晒offer

卷王逆袭成功案例

双非二本小伙春招大翻身
喜提9大offer

2月22日改简历

4月13日晒offer

面试法宝:
改简历 + IM实操

9大offer 最高年薪30万

序号	公司	部门	岗位	薪资结构	总包
1	公司	数据产品部	java后端开发	18.5k+14.5k+5k+200k/每月+500k/期权	>30w
2	公司	交易研发部	java后端开发	16k+14k+15k+餐补+100k/每月	22.4w
3	公司	待定	java游戏开发	15k+15k+餐补+100k/每月	>22.5w
4	公司	待定	java游戏开发	11k+13k+餐补+100k/每月	14.2w
5	公司	待定	java游戏开发	11k+13k+餐补+100k/每月	>16.8w
6	公司	待定	java游戏开发	9k+餐补+餐补+餐补	9k
7	公司	待定	java游戏开发	9k+餐补+餐补+餐补	9k
8	公司	待定	java游戏开发	9k+餐补+餐补+餐补	9k
9	公司	待定	java游戏开发	9k+餐补+餐补+餐补	9k

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>